

Xen vs. KVM under Mixed-Criticality Workloads: An Experimental Characterization

Rita Marino and Matteo Arnese

Abstract—This paper characterizes how the two general-purpose Type-1 and Type-2 hypervisors, Xen and KVM, behave when hosting mixed-criticality workloads on commodity hardware. We designed and executed a systematic measurement campaign using `cyclictst` and five TACLe real-time benchmarks under multiple host-guest kernel combinations, quantifying the impact of key isolation mechanisms, such as vCPU pinning, host-side `isolcpus/ nohz_full` and IRQ steering. For Xen, we further compare all three guest types (HVM, PV, PVH) and evaluate the Null versus Credit2 schedulers. Building on the methodology of Abeni and Faggioli [1], [2], our results reproduce their key findings on a current software stack (Linux 6.x, mainline PREEMPT_RT, Xen 4.17) and extend them to include worst-case execution time analysis under realistic benchmark workloads and Unikraft unikernels [?] as lightweight alternative guests.

Index Terms—hypervisor, real-time systems, KVM, Xen, PREEMPT_RT, cyclictst, TACLe, mixed-criticality, vCPU pinning, unikernel, worst-case execution time (WCET)

I. INTRODUCTION

The deployment of mixed-criticality workloads on shared hardware platforms is a central challenge in modern embedded and cyber-physical systems. Hypervisors provide spatial and temporal partitioning between guest domains of different criticality levels, making them an attractive building block for consolidating safety-critical and best-effort applications on a single physical machine. However, the scheduling overhead, interrupt handling, and resource contention inherent in general-purpose hypervisors can compromise the determinism required by hard real-time workloads.

Two of the most widely studied general-purpose hypervisors, **Xen** and **KVM**, differ fundamentally in their architecture. Xen is a Type-1 (bare-metal) hypervisor with a dedicated privileged domain (Dom0) that mediates all device access; KVM is a Type-2 loadable kernel module that extends a standard Linux host into a hypervisor. Understanding how these architectural differences manifest as latency and jitter under realistic workloads is essential for practitioners who must select and configure a virtualization stack for mixed-criticality deployments.

This work was carried out as part of the “Real-Time Systems and Industrial Applications” course (A.Y. 2025–26) at the University of Naples “Federico II.”

Supervisors: Ch.mo Prof. Marcello Cinque, Ch.mo Prof. Luigi De Simone, Ing. Francesco Boccola. Dept. of Electrical Engineering and Information Technologies (DIETI), University of Naples “Federico II,” Naples, Italy.

A. Related Work

Abeni and Faggioli [1] conducted a systematic experimental analysis of Xen and KVM latency profiles, characterizing the impact of the PREEMPT_RT patch at both host and guest level and quantifying the scheduling interference introduced by each hypervisor under background stress workloads. Their follow-up work [2] extended this analysis to include isolation mechanisms such as vCPU pinning and explored the Xen Null scheduler as a means of achieving near-static partition allocation with near-zero scheduling overhead. A further study by the same author [3] examined the behavior of virtualized real-time workloads in container-based and VM-based environments, broadening the comparison to include lightweight virtualization alternatives.

Cinque *et al.* [4] assessed temporal isolation in safety-critical mixed-criticality systems running on a Xen hypervisor, identifying residual interference paths that can violate isolation boundaries even after applying standard hardening techniques such as IRQ affinity and VCPU pinning. Their results highlight the importance of measuring not just average-case but worst-case latency behavior across long observation windows.

B. Contributions and Experimental Scope

This work reproduces and extends the experimental campaign of Abeni and Faggioli [1], [2] on a current software stack (Linux 6.x, Xen 4.17, mainline PREEMPT_RT rt5). Our specific contributions are:

- **Baseline reproduction:** We replicate the key scheduling-latency measurements of [1] using `cyclictst` across all four host-guest kernel combinations (NRT×NRT, NRT×RT, RT×NRT, RT×RT), both with and without a background `stress-ng` workload, for both KVM and Xen.
- **Extended isolation analysis:** We evaluate the incremental effect of vCPU pinning, host-side `isolcpus/nohz_full`, IRQ affinity steering, and cache/memory-bandwidth partitioning via Intel Resource Director Technology (RDT).
- **Xen guest-type and scheduler comparison:** We compare all three Xen guest types—HVM, PV, and PVH—under the Credit2 and Null schedulers to identify the configuration that minimises worst-case latency.
- **TACLe benchmark experiments:** We execute five real-time workloads from the TACLeBench suite [5] (DEBIE, huff_enc, lift, matrix1, test3) inside a

critical guest domain alongside a noisy neighbour, quantifying worst-case execution time (WCET) degradation under each configuration.

C. Experimental Platform

All experiments were conducted on the same physical host equipped with Intel VT-x hardware virtualization support and Intel Resource Director Technology (RDT) for cache and memory-bandwidth partitioning. The software stack comprised Xen 4.17-amd64 and KVM on a Linux 6.x host. Guest kernels spanned three configurations: non-real-time (NRT, vanilla Linux), Low-Latency (LL), and fully preemptible (PREEMPT_RT rt5). Scheduling latency was measured using `cyclicttest`, and worst-case execution time was characterized using the TACLeBench [5] suite. Background interference was generated using `stress-ng` with cache-pressure, interrupt-flooding, and raw-socket stressors.

The remainder of this paper is organized as follows. Section II describes the testbed configuration and kernel build procedure. Sections III and IV present the KVM and Xen experimental results, respectively. Section V concludes the paper and outlines future work.

II. EXPERIMENTAL SETUP

This chapter outlines the complete experimental testbed configuration required to evaluate Xen and KVM under mixed-criticality workloads. To ensure reliable, predictable, and reproducible latency measurements, the host environment must be strictly tailored for real-time execution.

The following sections detail the step-by-step preparation of the system, starting with the installation and tuning of the host operating system, the compilation of a fully preemptible Linux kernel (PREEMPT_RT), and the deployment of the respective hypervisors. We then detail both hypervisors' configuration, focusing on the differences between the two.

A. Kernel configuration and real-time tuning

To establish a baseline for our performance comparison, we also required the standard, non-real-time Linux kernel version 6.18.35.

- First, we downloaded the official kernel source code from the Linux Kernel Archive.
- We extracted the archive using the following command:
`tar -xvzf ~/Downloads/linux-6.18.35.tar.gz -C ~/`
- Next, we installed the necessary dependencies to build the kernel:
`sudo apt -y install libncurses-dev gawk flex`
`↪ bison openssl libssl-dev dkms libelf-dev`
`↪ libudev-dev libpci-dev libiberty-dev autoconf`
`↪ llvm qtcreator qtbase5-dev qt5-qmake cmake`
- We navigated into the Linux build tree and copied the configuration file from the currently running system:
`cd ~/linux-6.18.35`
`cp /boot/config-$(uname -r) .config`

- To fix potential configuration issues arising from the kernel version mismatch, we updated the configuration:
`make olddefconfig`
- To streamline the build by compiling only the currently loaded modules, we ran:
`make localmodconfig`

1) *Patching Linux with PREEMPT_RT*: To achieve true real-time determinism after installation, we must move beyond the default PREEMPT_DYNAMIC schema. While PREEMPT_DYNAMIC allows the kernel to dynamically determine preemption modes (e.g., *none*, *voluntary*, or *full*), it is not designed for real-time workloads and lacks hard guarantees for interrupt latency and thread scheduling.

For the Real-Time kernel, we needed to follow some additional steps:

- We downloaded the matching PREEMPT_RT patch from the Linux Foundation Real-Time Wiki and extracted it:
`gunzip -c ~/Downloads/patch-6.18.35-rt5.patch.gz`
`↪ > ~/patch-6.18.35-rt5.patch`
- Returning to the build tree, we applied the patch and opened the configuration GUI:
`cd ~/linux-6.18.35`
`patch -p1 < ../patch-6.18.35-rt5.patch`
`make xconfig`
- In the configuration menu, we navigated to General Setup -> Preemption Model and set it to Fully Preemptible Kernel (RT).
- We opened the configuration menu again (`make xconfig`) to manually disable specific features to reduce latency, strictly in the following order:
 - CONFIG_SCHED_MC_PRIO (**Processor type and features -> Multi-core scheduler support**)
 - CONFIG_CPU_FREQ (**Power management and ACPI options -> CPU Frequency scaling**)
 - CONFIG_STACKPROTECTOR
 - CONFIG_APM
 - CONFIG_ACPI_PROCESSOR (**Power management and ACPI options -> ACPI (Advanced Configuration and Power Interface)**)
 - CONFIG_CPU_IDLE (**Power management and ACPI options -> CPU idle PM support**)
 - **Simultaneous Multi-threading** (if supported by the hardware)
- We also ensured NVMe support was enabled:
 - CONFIG_BLK_DEV_NVME (**Device Drivers -> NVME Support -> NVM Express block device**)
- Under **Processor type and features**, we fine-tuned the settings for our specific hardware architecture and saved the configuration.

2) *Compiling the kernel*:

- For Ubuntu specifically, it is necessary to clear the Canonical certificates (`canonical.pem`) to avoid build failures. From the Linux build tree, we executed:
`sudo scripts/config --disable SYSTEM_TRUSTED_KEYS`

```
sudo scripts/config --disable
→ SYSTEM_REVOCATION_KEYS
make olddefconfig
```

- Finally, we built and installed the kernel and its modules:

```
sudo make -j25
sudo make modules_install
sudo make install
```

3) *Hardware Tuning (BIOS/UEFI)*: The configurations in this section are highly hardware-specific and will vary depending on the motherboard manufacturer and CPU vendor. To ensure predictable performance and minimize latency spikes for real-time workloads, it is crucial to disable dynamic frequency scaling and deep power-saving states directly at the firmware level.

Below are the specific configuration steps applied to our testbed, which features an MSI motherboard and an AMD processor:

- **Disable Precision Boost Overdrive (PBO)**: Navigate to Overclocking -> Advanced CPU Configuration -> AMD Overclocking and set **Precision Boost Overdrive to Disabled**.
- **Disable CPU Boost and Sleep States**: Navigate to Overclocking -> Advanced CPU Configuration -> AMD CBS and set both **Core Performance Boost** and **Global C-state Control to Disabled**.
- **Set a Fixed CPU Frequency**: In the main Overclocking menu, change the **CPU Ratio** from Auto to a fixed value of **44.00**. This forces the processor to run at a constant clock speed, preventing the latency overhead associated with dynamic frequency transitions during the experiments.

4) *OS-level isolation*: Given the presence of some counterintuitive results, we decided to configure the operating system to adopt additional isolation mechanisms, aiming to identify the possible cause of such behaviors by minimizing the variables introduced by the OS scheduler.

a) *Isolation and Configuration Steps*: The isolation was implemented at the operating system level through various techniques, including:

- **Kernel scheduling isolation (isolcpus)**: Use the `isolcpus=` boot parameter to prevent the scheduler from assigning tasks to a specific set of CPUs, thereby avoiding general SMP balancing.
- **Kernel house-keeping noise (nohz_full)**: Address kernel house-keeping noise by appending `nohz_full=` to enable Tickless Mode, which reduces OS overhead on those selected CPUs.
- **RCU callback offloading (rcu_nocbs)**: To further minimize latencies imposed by memory allocators in `softirq` contexts, apply RCU callback offloading to dedicated kernel threads using the `rcu_nocbs=` parameter.
- **IRQ affinity**: Configure IRQ affinity to ensure hardware interrupts are kept away from our isolated cores as much as possible.

The isolation was applied to both the host and the virtualized environment. The host system was configured to avoid scheduling tasks on the physical CPUs dedicated to running the virtual machines. Concurrently, the guest system was also configured so that the virtual machine's scheduler could not assign tasks to the cores reserved for real-time applications.

The following is an example of OS-level isolation we adopted in one of the KVM experiments. On the host, we fully isolated **CPU22** and **CPU23** on our 24-core system, configuring GRUB by adding a custom boot entry in `/etc/grub.d/40_custom`:

```
menuentry 'Ubuntu 24.04 (6.18.35-rt5-full)'{
    echo 'Loading Linux 6.18.35-rt5 with
    → NO_HZ, ISOLCPUS, RCU_NOCBS and no
    → IRQ_AFFINITY on [22, 23]'
    linux /boot/vmlinuz-6.18.35-rt5
    → root=UUID=8e001ea3-a450-434d-bfab-
    → ee2e6f61c6a2 ro nohz_full=22-23
    → isolcpus=22-23 rcu_nocbs=22-23
    → irqaffinity=0-21 quiet splash
    → $vt_handoff
    echo 'Loading initial ramdisk ...'
    initrd /boot/initrd.img-6.18.35-rt5
}
```

Similarly, on the guest we isolated **CPU1** with the following configuration:

```
menuentry 'Ubuntu 24.04 (6.18.35-rt5-full)'{
    echo 'Loading Linux 6.18.35-rt5 with
    → NO_HZ, ISOLCPUS, RCU_NOCBS and no
    → IRQ_AFFINITY on 1'
    linux /boot/vmlinuz-6.18.35-rt5
    → root=UUID=1fe78ac9-040a-4fe5-b3a8-
    → 8715d38d692e ro nohz_full=1
    → isolcpus=1 rcu_nocbs=1
    → irqaffinity=0 quiet splash
    → $vt_handoff
    echo 'Loading initial ramdisk ...'
    initrd /boot/initrd.img-6.18.35-rt5
}
```

B. KVM

Being effectively treated as a Type-2 hypervisor, KVM sits on top of an already booted operating system. The host OS manages it similarly to a user-space application, where the virtual CPUs (vCPUs) are scheduled as standard host processes. Consequently, the installation process is as straightforward as running the following command:

```
sudo apt -y install bridge-utils cpu-checker
→ libvirt-clients libvirt-daemon qemu-system
→ qemu-kvm virt-manager numad
```

We installed QEMU emulator version 8.2.2 (Debian 1:8.2.2+ds-0ubuntu1.17)

In order to apply the OS-level isolation techniques on the pinned configuration, we modified the

/etc/grub.d/40_custom file adding the entries in the GRUB host configuration.

Once the installation was complete, we provisioned the following guest Virtual Machines with the same hardware specifications:

We installed both the 6.18.35 and the 6.18.35-rt5 kernels in the same manner as on the host, with the exception that we did not enable the NVMe block device support, since we will be using VirtIO. For simplicity reasons, the two VMs share the same virtual drive.

In order to apply the OS-level isolation techniques on the pinned configuration, we modified the /etc/grub.d/40_custom file adding the entries in the GRUB guest configuration.

We ensured the VM vCPUs always had the required resources available and could not be preempted by other tasks by setting the vcpusched mode of both vCPUs to SCHED_FIFO with priority 98.

```
<vcpusched vcpus='0' scheduler='fifo' priority='98'/>
<vcpusched vcpus='1' scheduler='fifo' priority='98'/>
```

We enabled memory locking to prevent swapping:

```
<memoryBacking>
  <locked/>
</memoryBacking>
```

We ensured CPU pass-through and correct topology mapping:

```
<cpu mode='host-passthrough' check='none'>
  <topology sockets='1' dies='1' cores='2'
    ↳ threads='1'/>
</cpu>
```

In the pinned configuration, we applied CPU pinning to make the vCPU threads run exactly on the isolated cores:

```
<vcpu placement='static'>2</vcpu>
<cputune>
  <vcpupin vcpu='0' cpuset='22'/>
  <vcpupin vcpu='1' cpuset='23'/>
  <emulatorpin cpuset='0-21'/>
  <iothreadpin iothread='1' cpuset='0-21'/>
</cputune>
```

1) *TACLe Benchmark*: We provisioned the following guest Virtual Machines to run the TACLe benchmarks:

We configured them the same way we did for the ubuntu24.04 and ubuntu24.04-pinned VMs respectively, with the exception of the pinning layout of the emulation and I/O threads. These four also share the same virtual drive, since only one of them will be turned on at a time.

For the noisy guests, we provisioned similar VMs, but with different specs and no real-time vCPU priority assigned:

These three also share a different virtual drive, that does not require any modification but the installation of **stress-ng**:

```
sudo apt -y install stress-ng
```

The host configuration for the TACLe experiment differs from the cyclicttest one in a subtle but important way.

In the cyclicttest experiments, the host was booted with **isolcpus=22-23** to fully remove those two cores from the kernel's load balancer, ensuring that QEMU's vCPU threads would be the only workload on those cores. This approach works precisely because there is only one VM and its placement is static.

For the TACLe experiment, however, we need the Linux scheduler to remain active across CPUs 4–23, because both the TACLe VM and the noisy VM must be free to float across that entire pool. Using **isolcpus=4-23** would remove those cores from the load balancer entirely, preventing KVM from distributing the two VMs' vCPU threads across them — exactly the opposite of what we need.

Instead, we boot the host using a dedicated GRUB entry that applies **nohz_full=4-23**, **rcu_nocbs=4-23**, and **irqaffinity=0-3** to reduce kernel noise on the VM cores, but deliberately omits **isolcpus**:

```
menuentry 'Ubuntu 24.04 (6.18.35-rt5 TACLe)' {
  echo 'Loading Linux 6.18.35-rt5 with
    ↳ NO_HZ, RCU_NOCBS and no
    ↳ IRQ_AFFINITY on [4, 23]'
  linux /boot/vmlinuz-6.18.35-rt5
    ↳ root=UUID=8e001ea3-a450-434d-bfab-
    ↳ ee2e6f61c6a2 ro nohz_full=4-23
    ↳ rcu_nocbs=4-23 irqaffinity=0-3
    ↳ quiet splash $vt_handoff
  initrd /boot/initrd.img-6.18.35-rt5
}
```

To prevent host user-space services from wandering onto CPUs 4–23 without disabling the load balancer, we confine the host's systemd cgroup to CPUs 0–3 by adding the following line to /etc/systemd/system.conf:

```
CPUAffinity=0-3
```

This setting is applied at boot (PID 1 reads **system.conf** on startup) and removed at the end of the experiment via **sudo systemctl daemon-reexec**, which re-executes systemd in-place without restarting any services, immediately restoring normal scheduling.

C. Xen

We installed the Xen hypervisor 4.17.3 via the **xen-hypervisor-amd64** package. This process automatically generated the necessary GRUB bootloader entries to boot the Ubuntu system as **Dom0** (the privileged management domain).

```
sudo apt -y install xen-hypervisor-amd64
```

During our initial boot tests, we encountered severe instability with the Graphical User Interface (GUI). Specifically, the **nouveau** open-source drivers—often relied upon for NVIDIA GPU compatibility—failed to initialize correctly on our testbed. Further investigation suggested that graphical drivers generally exhibit poor stability when running under Xen Dom0, a behavior observed across different

TABLE I
KVM VIRTUAL MACHINES CONFIGURATION

Name	ubuntu24.04	ubuntu24.04-pinned
vCPUs:	2	2
RAM:	4 GB	4 GB
Isolation techniques	Memory locking, CPU pass-through	CPU pinning, memory locking, CPU pass-through, OS-level isolation
Configuration file	ubuntu24.04.xml	ubuntu24.04-pinned.xml

TABLE II
TACLe BENCHMARK VMs CONFIGURATION (KVM)

Name	ubuntu24.04-tacle	ubuntu24.04-tacle-pinned
vCPUs:	2	2
RAM:	4 GB	4 GB
Isolation techniques	Memory locking, CPU pass-through	CPU pinning, memory locking, CPU pass-through, OS-level isolation
Configuration file	ubuntu24.04-tacle.xml	ubuntu24.04-tacle-pinned.xml

TABLE III
NOISY GUEST VMs CONFIGURATION (KVM)

Name	ubuntu24.04-noise-small	ubuntu24.04-noise-big	ubuntu24.04-noise-big-pinned
vCPUs:	2	16	16
RAM:	4 GB	16 GB	16 GB
Isolation techniques	Memory locking, CPU pass-through	Memory locking, CPU pass-through	CPU pinning, memory locking, CPU pass-through
Configuration file	ubuntu24.04-noise-small.xml	ubuntu24.04-noise-big.xml	ubuntu24.04-noise-big-pinned.xml

hardware configurations. To bypass this limitation, we opted for a headless setup and managed the host via SSH.

To streamline the provisioning of subsequent DomUs, we installed the `xen-tools` package:

```
sudo apt -y install xen-tools
```

Since our guests required dedicated block storage, we resized the existing LVM (Logical Volume Manager) partition hosting the Ubuntu installation to carve out a new logical volume exclusively dedicated to the VMs. We performed this using the following steps:

```
sudo lvreduce --resizefs --size -65G
↪ /dev/ubuntu-vg/root
sudo lvcreate -L 65G -n ubuntu-24.04-domU ubuntu-vg
```

To speed up the setup of the DomU, we decided to clone the Dom0 partition and assign it a new UUID:

```
sudo dd if=/dev/nvme0n1p4
↪ of=/dev/ubuntu-vg/ubuntu-24.04-domU
↪ status=progress
sudo e2fsck -f /dev/ubuntu-vg/ubuntu-24.04-domU
sudo tune2fs -U random
↪ /dev/ubuntu-vg/ubuntu-24.04-domU
```

We designed different configuration files to provision our DomU instances, with the same hardware specifications. The differences between the configurations are the under-

lying kernel used to boot them and how vCPU allocation is performed.

These different configurations share the same LVM partition, as only one of them will be turned on at a time.

1) *Problems with PREEMPT_RT*: During the initial setup phase, we attempted to boot Xen using the same real-time kernel—compiled with the previously described instructions—as the Dom0 kernel. We tested various kernel versions across different Linux distributions and experimented with several combinations of the tuning parameters mentioned earlier (specifically: `CONFIG_SCHED_MC_PRIO`, `CONFIG_CPU_FREQ`, `CONFIG_STACKPROTECTOR`, `CONFIG_APM`, `CONFIG_ACPI_PROCESSOR`, and `CONFIG_CPU_IDLE`).

Furthermore, we applied a wide array of Xen and kernel command-line boot parameters that are traditionally recommended for resolving boot hangs and hardware initialization issues. However, none of these mitigations proved successful.

We also ensured that all the necessary configuration flags required to run the kernel as a Xen Dom0 were strictly enabled. We tried:

- `swiotlb=35536,force`
- `iommu=pt`

TABLE IV
XEN DOMU CONFIGURATIONS

Name	ubuntu-24.04-linux-6.18.35-nrt-hvm	ubuntu-24.04-linux-6.18.35-nrt-hvm-pinned	ubuntu-24.04-linux-6.18.35-rt5-hvm	ubuntu-24.04-linux-6.18.35-rt5-hvm-pinned
vCPUs:	2	2	2	2
RAM:	4 GB	4 GB	4 GB	4 GB
Isolation techniques	None	CPU pinning	None	CPU pinning
Configuration file	ubuntu-24.04-linux-6.18.35-nrt-hvm.conf	ubuntu-24.04-linux-6.18.35-nrt-hvm-pinned.conf	ubuntu-24.04-linux-6.18.35-rt5-hvm.conf	ubuntu-24.04-linux-6.18.35-rt5-hvm-pinned.conf

- `console=hvc0`
- `console=tty0`
- `earlyprintk=xen`
- `apci=off noapic pci=noms`
- `noirqbalance`
- `nomodeset`

Despite these extensive troubleshooting efforts, we observed that enabling the “Fully Preemptible Kernel (RT)” option consistently caused severe boot incompatibilities with the Xen hypervisor. The boot sequence systematically stalled even before the initialization of the logging daemons (such as `systemd-journald`).

Coupled with the graphical driver issues discussed previously, debugging this behavior proved to be a formidable challenge. The system most likely dropped into an `initramfs` recovery shell, which remained completely inaccessible to us in our headless setup.

Consequently, we decided to leave the “Fully Preemptible Kernel” option disabled for the Dom0 kernel. Instead, we opted for the “Low-Latency” scheduling model, which guaranteed a reliable boot process while still offering better responsiveness compared to the standard generic kernel.

2) *Resource partitioning and NULL-scheduler*: In order to compare the effects of the scheduler choice on Xen, we swapped the default Credit2 scheduler with a pinned configuration, effectively using an offline scheduler. This is done to assess the current effects of the issues identified by the previous analyses of Abeni and Faggioli, such as the priority inversion via QEMU and the `TIMER_SLOP` limitation. In order to do so, we changed the Dom0 GRUB boot configuration at `/etc/grub.d/40_custom` adding the entries shown in this GRUB configuration file.

This configuration restricts Xen to using only the first 22 pCPUs for Dom0. Also, we configured the pinned guests to use only the pCPUs 22 and 23, so that they would have two dedicated cores with no interference from the Dom0.

To be absolutely certain of the vCPU to pCPU fixed mapping, we also executed the following commands after the VM booted:

```
sudo xl vcpu-pin ubuntu-24.04-linux-6.18.35-rt5 0 22
sudo xl vcpu-pin ubuntu-24.04-linux-6.18.35-rt5 1 23
```

3) *TACLe Benchmark*: For the TACLe benchmarks, we used the Low-Latency Dom0 and Real-Time DomU setup,

and pinned Dom0 to only use the first four pCPUs, leaving the others to both the noisy guest and the benchmark guest. This is to separate the cores dedicated to the privileged domain from the other guests, ensuring that only stress generated by the benchmarks and the noisy VM is reflected in the results.

We designed the following configurations to run the TACLe benchmarks:

The only difference between these configurations and the previous ones is the range the vCPUs will be scheduled in. These too share the same LVM partition assigned to the other DomUs.

For the noisy guests, we defined similar configurations, but with different specs:

These three also share a different LVM partition, that does not require any modification but the installation of `stress-ng`:

```
sudo apt -y install stress-ng
```

a) *TACLe and stress on Dom0*: There is also a variation of the above configuration, where the stress is generated on Dom0 itself. In this case, there is no need to limit the scheduling range of the DomU. The configuration file is this one. The pinned configuration, by definition, does not change.

4) *PV and PVH DomUs*: To test different virtualization technologies, we created some PV and PVH configuration variants of the previous real-time DomUs. As we will discuss in the Xen analysis section, the PV DomUs don’t support `PREEMPT_RT` patched kernels with the Fully Preemptible Kernel option enabled (in a similar fashion to Dom0, it being a PV domain too), thus we used the Low-Latency variant of the kernel for these configurations:

All these configurations use the same LVM partition as the other HVM configurations, and do not require any other specific setup.

D. Benchmark automation

In order to streamline the testing procedure, a series of supplementary configuration steps were required. Since these procedures apply identically across both virtualization platforms, the following terminology will be used for simplicity: the term **Host** will refer collectively to the KVM

TABLE V
TACLe BENCHMARK CONFIGURATIONS (XEN)

Name	ubuntu-24.04-linux-6.18.35-rt5-hvm-tacle	ubuntu-24.04-linux-6.18.35-rt5-hvm-tacle-pinned
vCPUs:	2	2
RAM:	4 GB	4 GB
Isolation techniques	CPU scheduling range (4-23)	CPU pinning
Configuration file	ubuntu-24.04-linux-6.18.35-rt5-hvm-tacle.conf	ubuntu-24.04-linux-6.18.35-rt5-hvm-tacle-pinned.conf

TABLE VI
NOISY GUEST CONFIGURATIONS (XEN)

Name	ubuntu-24.04-linux-6.18.35-nrt-hvm-noisyguest-small	ubuntu-24.04-linux-6.18.35-nrt-hvm-noisyguest-big	ubuntu-24.04-linux-6.18.35-nrt-hvm-noisyguest-big-pinned
vCPUs:	2	16	16
RAM:	4 GB	16 GB	16 GB
Isolation techniques	CPU scheduling range (4-23)	CPU scheduling range (4-23)	CPU pinning
Configuration file	ubuntu-24.04-linux-6.18.35-nrt-hvm-noisyguest-small.conf	ubuntu-24.04-linux-6.18.35-nrt-hvm-noisyguest-big.conf	ubuntu-24.04-linux-6.18.35-nrt-hvm-noisyguest-big-pinned.conf

TABLE VII
PV AND PVH DOMU CONFIGURATIONS

Name	ubuntu-24.04-linux-6.18.35-rt5-ll-pv	ubuntu-24.04-linux-6.18.35-rt5-ll-pv-pinned	ubuntu-24.04-linux-6.18.35-rt5-pvh	ubuntu-24.04-linux-6.18.35-rt5-pvh-pinned
vCPUs:	2	2	2	2
RAM:	4 GB	4 GB	4 GB	4 GB
Isolation techniques	None	CPU pinning	None	CPU pinning
Configuration file	ubuntu-24.04-linux-6.18.35-rt5-ll-pv.conf	ubuntu-24.04-linux-6.18.35-rt5-ll-pv-pinned.conf	ubuntu-24.04-linux-6.18.35-rt5-pvh.conf	ubuntu-24.04-linux-6.18.35-rt5-pvh-pinned.conf

Host and the Xen Dom0, while the term **Guest** will denote both the KVM virtual machine and the Xen DomU.

First, we installed `openssh-server` and `expect` on the Host to orchestrate the benchmarks from a remote machine and interact with the Guest OS via the `x1` and `virsh` consoles.

```
sudo apt -y install openssh-server expect
```

To enable passwordless SSH access, we generated an SSH key pair on the orchestrator machine and appended the public key to the `authorized_keys` file on the Host:

```
ssh-keygen -t ed25519 -C "xen-vs-kvm-orchestrator"
ssh-copy-id matt@192.168.1.166
```

To facilitate operations requiring root privileges without manual intervention, we added the following rule to the `/etc/sudoers` file on the Host and Guests using `visudo`:

```
matt ALL=(ALL) NOPASSWD: ALL
```

To automatically boot on the appropriate kernel with the correct settings, we modified the GRUB configuration in `/etc/default/grub` on the Host and KVM Guests:

```
GRUB_DEFAULT=saved
GRUB_TIMEOUT_STYLE=menu
```

```
GRUB_TIMEOUT=5
```

For the KVM Guests, we enabled the serial console by enabling the corresponding `systemctl` service:

```
sudo systemctl enable --now
↪ serial-getty@ttyS0.service
```

III. KVM

This section presents a comprehensive analysis of execution latencies, determinism, and real-time capabilities within a KVM-based virtualized environment. We evaluate system behavior across four host–guest kernel combinations, ranging from standard non-real-time configurations to fully optimized end-to-end `PREEMPT_RT` stacks, and extend the analysis to TACLe real-time benchmark workloads under noisy-neighbour conditions.

A. BASELINE PERFORMANCE ANALYSIS IN KVM ENVIRONMENTS

This section presents a detailed analysis of execution latencies measured within a virtualized environment based on the Kernel-based Virtual Machine (KVM) hypervisor. The primary objective is to establish a performance baseline and evaluate the system’s behavior, determinism, and virtualization overhead under standard scheduling policies

before introducing more advanced or restrictive tuning configurations.

To quantify response times, internal jitter, and the Worst-Case Execution Time (WCET), the `cyclictest` tool was employed through continuous 5-minute baseline executions. The investigation explores the impact on system stability and temporal predictability by cross-referencing four distinct kernel combinations between the underlying Host infrastructure and the Guest virtual machine:

- **Non-Real-Time (NRT) Host and Non-Real-Time (NRT) Guest:** to measure standard system behavior and baseline virtualization overhead in the total absence of real-time optimizations.
- **Non-Real-Time (NRT) Host and Real-Time (RT) Guest:** to evaluate the effectiveness of internal scheduling optimizations within the guest when the underlying hypervisor lacks deterministic guarantees.
- **Real-Time (RT) Host and Non-Real-Time (NRT) Guest:** to analyze the impact of a determinism-optimized host on the performance, preemption spikes, and overall execution stability of a standard, general-purpose guest.
- **Real-Time (RT) Host and Real-Time (RT) Guest (Full RT Stack):** to observe the maximum level of temporal predictability achievable by aligning both the host infrastructure and the guest operating system with the `PREEMPT_RT` patch.

The analyses in the following paragraphs offer a comprehensive overview of the latency profiles characteristic of each configuration, detailing the limitations of general-purpose environments and validating the effectiveness of an end-to-end real-time stack for mitigating virtualization overhead.

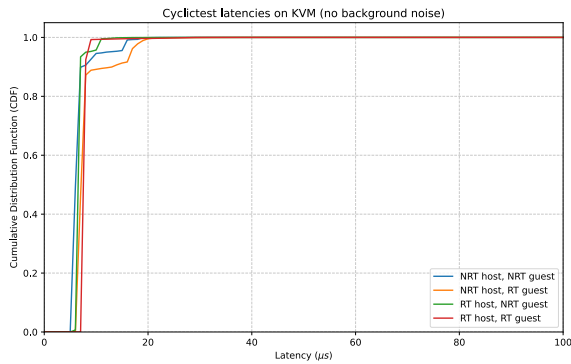


Fig. 1. Baseline Performance - KVM

1) *NON-REAL-TIME HOST AND NON-REAL-TIME GUEST:* This section presents an analysis of a single, extended 5-minute `cyclictest` run conducted in a standard environment, featuring a Non-Real-Time guest Linux kernel hosted on a Non-Real-Time host system. The objective is to establish a performance baseline and assess system determinism in the absence of real-time optimizations, such as the `PREEMPT_RT` patch or a real-time tuned host hypervisor.

a) *Distribution Analysis and Nominal Performance:* Analysis of the histogram log reveals significant consistency during nominal execution over the 5-minute test period. The statistical mode (the most frequent latency value) was anchored firmly at $6 \mu s$. The average latency was equally stable, measuring exactly $7 \mu s$ across the entire continuous execution. Additionally, the minimum recorded latency was extremely low, bottoming out at $5 \mu s$. This performance indicates that under nominal conditions, without background load or anomalous preemption, the baseline overhead introduced by the default virtualization layer and host kernel scheduler is remarkably low.

b) *Worst-Case Execution Time (WCET) and Lack of Determinism:* The fundamental limitation of this general-purpose configuration is revealed by the Worst-Case Execution Time (WCET) data, which continues to demonstrate high variability and unpredictable latency spikes. Across the 5-minute continuous execution, the absolute maximum latency recorded was $602 \mu s$, a severe, stochastic deviation from the average.

c) *Conclusions on Standard Environments:* These latency spikes—the “long tails” observed in the distribution logs—are characteristic of general-purpose software stacks. In this ecosystem, the hypervisor operates without strict real-time constraints and may preempt the Virtual CPU (VCPU) to serve host-level tasks; simultaneously, the guest kernel is subject to non-preemptible critical sections and non-deferrable hardware interrupts. Consequently, while nominal and average performance metrics are excellent, the Non-RT guest on a Non-RT host environment is susceptible to unpredictable delays and cannot guarantee the rigid, reliable upper bounds required for safety-critical control applications.

2) *NON-REAL-TIME HOST AND REAL-TIME GUEST:* This section analyzes the results obtained from a single, prolonged 5-minute execution of the `cyclictest` tool in a hybrid environment, configured with a Real-Time optimized guest hosted on a Standard Linux Kernel Host. The goal of this analysis is to evaluate if the guest’s internal real-time scheduling can maintain determinism during sustained workloads when the underlying host hypervisor lacks real-time optimizations.

a) *Nominal Performance and Average Latency:* The data collected confirm that optimization at the guest level guarantees excellent efficiency in the average case. The average latency value is extremely stable, resting at $8 \mu s$ for the duration of the 5-minute test. Furthermore, the absolute minimum latency dropped down to $4 \mu s$. This behavior indicates that the host allocates VCPU resources to the guest promptly and with minimal virtualization overhead under normal operating conditions despite not being patched.

b) *Worst-Case Execution Time (WCET) Analysis:* Despite the high baseline efficiency of the underlying infrastructure, the analysis of the Worst-Case Execution Time (WCET) reveals a severe lack of determinism under sustained testing. The recorded maximum latency reached an extreme peak of $6114 \mu s$ (over 6 milliseconds), and other 3 over-milliseconds

spikes. This data indicates that while the system performs well nominally, it is subject to rare but catastrophic scheduling delays that shatter any real-time guarantees.

c) Conclusions on the Hybrid Environment: The empirical observation of this sustained 5-minute test demonstrates conclusively that the exclusive optimization of the virtualization infrastructure (Hypervisor/Host) is not able to stem anomalous latencies if the guest operating system is not equally optimized. The critical peak of over 6 milliseconds is a direct symptom of the internal architecture of the standard kernel. The absence of the `PREEMPT_RT` patch leaves the guest vulnerable to delays induced by the host's non-preemptible critical sections (such as spinlocks), non-deferrable hardware interrupts, and unbounded priority inversion phenomena.

Therefore, to obtain reliable determinism in control contexts, the use of an RT Guest proves not to be a sufficient condition.

3) REAL-TIME HOST AND NON-REAL-TIME GUEST: This section analyzes the results obtained from a single, prolonged 5-minute execution of the `cyclicttest` tool in a hybrid environment, configured with a Standard Linux Guest hosted on a Real-Time `PREEMPT_RT` patched Host. The goal of this analysis is to verify whether the use of a deterministic Host hypervisor is sufficient to guarantee strict temporal constraints during sustained workloads when the guest operating system remains general-purpose.

a) Nominal Performance and Average Latency: The data collected confirm that optimization at the Guest OS level guarantees excellent efficiency in the average case. The average latency value is extremely stable, resting at $8\ \mu s$ for the duration of the 5-minute test. Furthermore, the absolute minimum latency dropped down to $4\ \mu s$. This behavior indicates that the Real-Time guest handles thread scheduling promptly and with minimal overhead under normal operating conditions.

b) Worst-Case Execution Time (WCET) Analysis: The analysis of the Worst-Case Execution Time (WCET) reveals highly stable and deterministic behavior during this sustained test. The absolute maximum latency recorded reached a peak of only $93\ \mu s$. This data demonstrates a complete absence of the severe, prolonged hypervisor-induced preemption spikes that typically characterize standard host environments.

c) Conclusions on the Hybrid Environment: The empirical observation of this sustained 5-minute test demonstrates conclusively that the exclusive optimization of the guest operating system is not able to stem anomalous latencies if the virtualization infrastructure (Hypervisor/Host) is not equally optimized. The critical peak of over 6 milliseconds is a direct symptom of the internal architecture of the standard host kernel. The absence of the `PREEMPT_RT` patch on the host leaves the guest vulnerable to delays induced by the host's non-preemptible critical sections (such as spinlocks), non-deferrable hardware interrupts, and unbounded priority inversion phenomena.

4) REAL-TIME HOST AND REAL-TIME GUEST (FULL RT STACK): This section analyzes the results obtained from a single, prolonged 5-minute execution of the `cyclicttest` tool in a "Full RT" environment, where a `PREEMPT_RT` patched Linux guest kernel is hosted on a Real-Time optimized Host. The objective of this configuration is to validate the effectiveness of an end-to-end real-time stack in mitigating virtualization overhead and establishing a strict, deterministic upper bound for execution latency during a sustained workload.

a) Nominal Performance and Average Latency: The experimental data collected demonstrates unparalleled stability under sustained nominal conditions. The average latency metric is exceptionally consistent, locked at $8\ \mu s$ for the duration of the test. The absolute minimum latency recorded was $7\ \mu s$. The histogram distribution highlights this extreme efficiency, showing that the vast majority of execution cycles—over 92% of samples—completed in exactly $8\ \mu s$. This indicates that when the Virtual CPU (VCPU) is active, the `PREEMPT_RT` patch successfully manages thread scheduling and interrupt handling with virtually zero internal jitter, effectively mirroring bare-metal performance for standard task cycles.

b) Worst-Case Execution Time (WCET) Analysis: The transition to a Full RT stack demonstrates a definitive improvement in worst-case performance, completely neutralizing the severe stochastic latency spikes observed in non-optimized configurations. Throughout the entire 5-minute sustained test, the absolute maximum latency (Worst-Case Execution Time) was strictly bounded at just $81\ \mu s$. The "long tail" of the distribution—characteristic of hypervisor-induced preemption and unoptimized kernel locks—has been entirely eliminated. The system demonstrates a robust, flawless ability to remain within deterministic bounds, drastically reducing the magnitude of any scheduling delays.

c) Conclusions on the Full RT Architecture: The empirical observation of this sustained test provides conclusive evidence regarding the requirements for deterministic virtualization:

- **End-to-End Determinism:** Achieving hard real-time guarantees is an end-to-end property, and the combination of the `PREEMPT_RT` Guest kernel and a Real-Time Host successfully prevents hypervisor-induced preemption.
- **Absolute Stability:** The system maintained a strict upper bound of $81\ \mu s$ over nearly 6 million consecutive execution cycles.
- **Zero Overflows:** The complete absence of histogram overflows confirms that the system never experienced uncontrolled latency spikes during the prolonged execution.
- **Viability for Critical Systems:** This "Full RT" configuration establishes a highly predictable WCET, proving that with proper infrastructure tuning, virtualized environments can reliably support safety-critical control applications that previously required dedicated

bare-metal hardware.

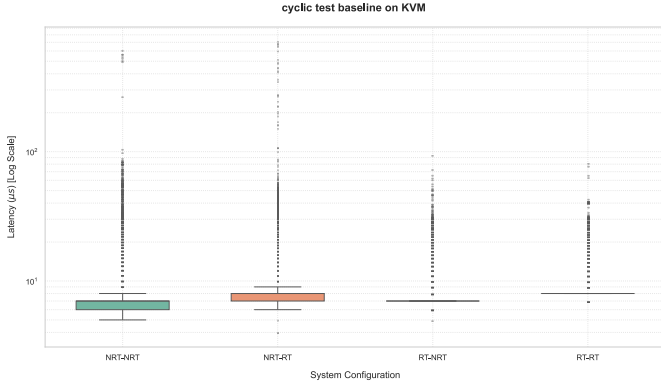


Fig. 2. Baseline Performance - KVM

B. IMPACT OF THE STRESS WORKLOAD ON LATENCIES

To evaluate system robustness and trigger potentially higher latencies, the testing methodology involves introducing an additional load, defined as a “stress workload.” In the case of the KVM hypervisor, this stress workload is executed in the background directly on the Host operating system, simulating heavy external contention that competes for CPU time and system resources.

Experimental analysis of the KVM infrastructure reveals that adding this background load produces severe performance degradation across the board. Unlike the complex, configuration-dependent behaviors observed in other hypervisors, the stress workload on KVM exposes fundamental architectural vulnerabilities in maintaining determinism:

- **Catastrophic Failure in Unoptimized and Hybrid Stacks:** Introducing host-level stress immediately shatters the determinism of standard and hybrid configurations. Whether using a fully Non-Real-Time (NRT) stack or attempting partial optimization (an RT Guest on an NRT Host, or an NRT Guest on an RT Host), the system experiences catastrophic scheduling delays. Maximum latencies in these configurations routinely spike between 44 and 51 milliseconds, indicating massive host-induced starvation and lock contention.
- **The Illusion of Guest-Only Optimization:** Counter-intuitively, equipping the Guest with a Real-Time kernel while the Host remains NRT yielded the highest variability and the worst absolute latency spikes (over 51 milliseconds) of the dataset. This proves that an optimized guest scheduler is entirely powerless to protect time-sensitive threads if the underlying Host hypervisor is vulnerable to non-deterministic preemption.
- **Chronic Instability of the Full RT Stack:** Even when utilizing an end-to-end “Full RT” stack (RT Host and RT Guest) with optimal prioritization parameters, the KVM environment fails to guarantee strict real-time bounds under stress. While the Full RT stack

successfully suppresses the catastrophic 50-millisecond delays (bounding the absolute maximum latency to approximately 9.8 milliseconds), it suffers from a massive frequency of significant scheduling stalls, evidenced by hundreds of over-millisecond latency overflows.

In this section, we will present the detailed experiments conducted under these stress conditions and analyze the resulting execution logs.

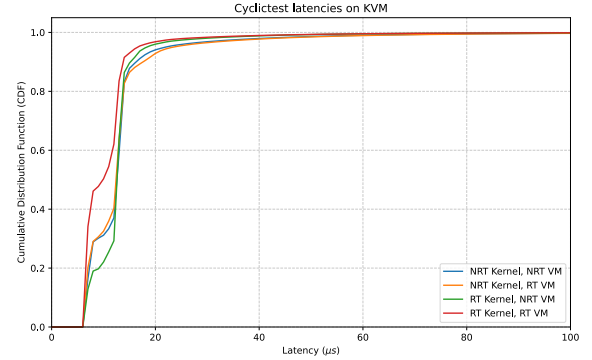


Fig. 3. Performance under stress workload- KVM at maximum priority

1) *NON REAL-TIME HOST AND NON REAL-TIME GUEST -STRESS HOST:* This section presents an analysis of the first `cyclictest` execution log conducted on a KVM host under stress conditions. The objective is to evaluate the system’s scheduling behavior and latency bounds when subjected to external load.

a) *Distribution Analysis and Nominal Performance:* Analysis of the histogram log reveals that the system maintains a reasonable baseline under stress. Over the test duration, the average latency recorded was 13 μs . The statistical mode (the most frequent latency) was concentrated at 14 μs , with a significant number of cycles also completing at 13 μs . The absolute minimum recorded latency dropped to an impressive 5 μs . This indicates that when the CPU is not actively dealing with stress-induced contention, the base virtualization and scheduling overhead remains low.

b) *Worst-Case Execution Time (WCET) and Lack of Determinism:* The impact of the stress workload becomes severely apparent when analyzing the Worst-Case Execution Time (WCET). The absolute maximum latency recorded during this run was an extreme 44,409 μs (over 44 milliseconds). Furthermore, the log reports 11 histogram overflows (latencies exceeding the 1000 μs tracking threshold).

c) *Conclusions:* The presence of 44-millisecond latency spikes clearly demonstrates that this environment lacks determinism. Under stress, the host OS experiences massive scheduling delays, likely due to lock contention, non-preemptible critical sections, or starvation of the test threads. This configuration cannot guarantee the strict upper bounds required for real-time applications.

2) *NON REAL-TIME HOST AND REAL-TIME GUEST - STRESS HOST*: This section analyzes the results obtained from the second `cyclictest` execution, also run under stressed conditions on the KVM host infrastructure.

a) *Nominal Performance and Average Latency*: The nominal performance mirrors the first log closely, confirming a consistent baseline efficiency even under load. The average latency was precisely $13\ \mu s$, and the minimum latency was $5\ \mu s$. Interestingly, the distribution shows a dual-peak behavior, with the highest concentration at $13\ \mu s$ (the mode), but with another massive spike of cycles completing at $7\ \mu s$.

b) *Worst-Case Execution Time (WCET) Analysis*: This specific execution experienced the highest variability and worst latency spikes of the entire dataset. The absolute maximum latency reached an exorbitant $51,069\ \mu s$ (over 51 milliseconds). The severity of the instability is further highlighted by the 26 histogram overflows, more than double the number seen in the first log.

c) *Conclusions*: The empirical observation of this run conclusively shows that the stress workload induces chaotic, non-deterministic preemption. A 51-millisecond delay represents a catastrophic failure for any safety-critical system. The KVM host in this state is entirely unsuitable for bounded real-time execution, as the scheduler cannot adequately protect time-sensitive threads from background stress.

3) *REAL-TIME HOST AND NON REAL-TIME GUEST - STRESS HOST*: This section examines the third prolonged execution of the `cyclictest` tool on the stressed KVM host environment, validating the patterns observed in previous iterations.

a) *Nominal Performance and Average Latency*: The data collected confirms a stable nominal execution path. The average latency metric remained locked at $13\ \mu s$, while the minimum latency reached the lowest point among the first three runs, hitting $3\ \mu s$. The histogram shows a massive concentration of execution cycles completing in exactly $13\ \mu s$, indicating that when the scheduler is unimpeded, it processes tasks with high consistency.

b) *Worst-Case Execution Time (WCET) Analysis*: Despite the strong nominal performance, the WCET remains unacceptable for real-time standards. The maximum latency peaked at $50,257\ \mu s$ (over 50 milliseconds). The system registered 11 histogram overflows, indicating multiple occurrences of severe scheduling stalls exceeding one millisecond.

c) *Conclusions*: This run confirms that while the system can occasionally achieve extremely fast task dispatching ($3\ \mu s$ min), it is completely vulnerable to unpredictable, systemic delays. The 50-millisecond peak re-emphasizes that background stress can effectively starve tasks for dozens of milliseconds, a fatal condition for latency-sensitive applications.

4) *REAL-TIME HOST AND REAL-TIME GUEST - STRESS HOST*: This section presents an analysis of the provided `cyclictest` execution log conducted on a KVM environment under stress conditions. The objective is to evaluate the system's scheduling determinism and latency

bounds when subjected to external load.

a) *Distribution Analysis and Nominal Performance*: Analysis of the histogram log reveals a highly efficient baseline performance under nominal execution. The statistical mode (the most frequent latency) is sharply concentrated at $7\ \mu s$, representing over 1.7 million completed cycles. The average latency metric is remarkably stable at $11\ \mu s$, and the absolute minimum recorded latency is exceptionally low at $4\ \mu s$. This indicates that most of the time, the virtualization overhead is minimal.

b) *Worst-Case Execution Time (WCET) Analysis*: Despite the excellent average case, the analysis of the Worst-Case Execution Time (WCET) reveals severe scheduling instability induced by the stress workload. The absolute maximum latency recorded reached $9,829\ \mu s$ (nearly 9.8 milliseconds). Even more critically, the log reports an overwhelming 283 histogram overflows (latencies exceeding the $1000\ \mu s$ tracking threshold). This represents a massive frequency of significant scheduling stalls compared to typical runs.

c) *Conclusions*: The empirical observation of this execution demonstrates severe latency instability, which is particularly critical given the system's strict configuration. Despite this optimal prioritization, the system still suffered from chronic and significant delays.

These high latencies indicate that the delays are originating from deeper, non-preemptible sources escaping the guest's control. Consequently, this configuration proves that merely applying the maximum scheduler priority is fundamentally insufficient to guarantee the strict, reliable upper bounds required for safety-critical real-time applications.

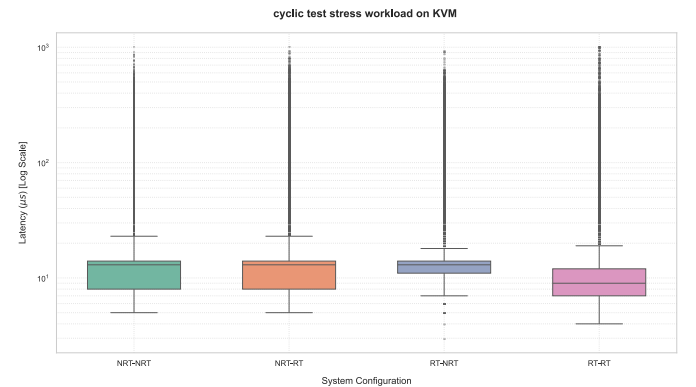


Fig. 4. Performance under stress workload- KVM at maximum priority

C. ISOLATED STRESS HOST PERFORMANCE ANALYSIS

This section presents a detailed analysis of execution latencies measured within a virtualized environment based on the Kernel-based Virtual Machine (KVM) hypervisor, subjected to a background stress workload but utilizing OS-level isolation strategies. The primary objective is to evaluate whether spatial isolation of the Virtual CPUs (VCPUs) can effectively shield the guest operating system

from the severe preemption and scheduling delays induced by host-level contention.

To quantify response times, internal jitter, and the Worst-Case Execution Time (WCET) under these isolated conditions, the `cyclictest` tool was employed through continuous 5-minute executions. The investigation explores the impact on system stability and temporal predictability by cross-referencing four distinct kernel combinations between the underlying Host infrastructure and the Guest virtual machine:

- **Non-Real-Time (NRT) Host and Non-Real-Time (NRT) Guest:** to measure system behavior and virtualization overhead when applying isolation techniques to a standard, unoptimized software stack.
- **Non-Real-Time (NRT) Host and Real-Time (RT) Guest:** to evaluate the effectiveness of an optimized guest scheduler operating on dedicated cores, while the underlying hypervisor lacks deterministic guarantees.
- **Real-Time (RT) Host and Non-Real-Time (NRT) Guest:** to analyze the impact of a determinism-optimized host on the preemption spikes of a standard guest when spatial isolation is enforced.
- **Real-Time (RT) Host and Real-Time (RT) Guest (Full RT Stack):** to observe the maximum level of temporal predictability achievable by combining CPU isolation with an end-to-end `PREEMPT_RT` patched infrastructure.

The analyses in the following paragraphs offer a comprehensive overview of the latency profiles characteristic of each isolated configuration, detailing the effectiveness of core pinning in mitigating the catastrophic failures observed in un-isolated stress scenarios.

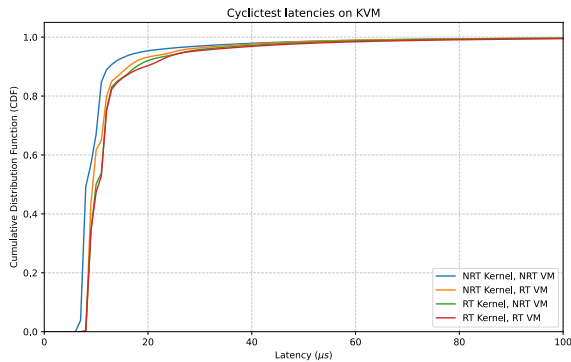


Fig. 5. Performance under stress- KVM OS-level isolation

1) NON-REAL-TIME HOST AND NON-REAL-TIME GUEST (ISOLATED STRESS HOST): This section presents an analysis of a single, extended 5-minute `cyclictest` run conducted in an isolated environment, featuring a Non-Real-Time guest Linux kernel hosted on a Non-Real-Time host system under stress. The objective is to establish a performance baseline for spatial isolation without real-time kernel patches.

a) Distribution Analysis and Nominal Performance: Analysis of the histogram log reveals a very consistent nominal execution under isolated stress. The statistical mode (the most frequent latency value) was anchored firmly at $8 \mu s$. The average latency was highly stable, measuring exactly $11 \mu s$ across the entire continuous execution. Additionally, the minimum recorded latency dropped to $6 \mu s$. This indicates that CPU isolation successfully protects the active VCPU during standard execution cycles, keeping the baseline virtualization overhead low despite the background load on the host.

b) Worst-Case Execution Time (WCET) and Lack of Determinism: While isolation drastically improved the overall stability compared to un-isolated stress tests (which saw 44-millisecond delays), the Worst-Case Execution Time (WCET) data reveals that determinism is not fully guaranteed. The absolute maximum latency recorded was bounded at $2,024 \mu s$ (roughly 2 milliseconds). However, the system still registered 19 histogram overflows (latencies exceeding the $1000 \mu s$ threshold).

c) Conclusions on Standard Isolated Environments: The application of CPU isolation prevents the catastrophic multi-millisecond starvation seen in standard stress scenarios, reducing the WCET by over 95%. However, the presence of nearly 2-millisecond spikes and 19 overflows confirms that a general-purpose NRT/NRT stack remains susceptible to unpredictable delays. Because the tasks cannot be preempted by standard user-space processes (due to the FIFO=99 scheduling policy), these spikes originate from deeper, non-preemptible sources such as host kernel lockups, unmaskable hardware interrupts, or shared hardware resource contention (like cache and memory bus) induced by the stress load running on neighboring cores.

2) NON-REAL-TIME HOST AND REAL-TIME GUEST (ISOLATED STRESS HOST): This section analyzes the results obtained from a single, prolonged 5-minute execution of the `cyclictest` tool in a hybrid environment, configured with a Real-Time optimized guest hosted on a Standard Linux Kernel Host, with CPU isolation applied during host stress.

a) Nominal Performance and Average Latency: The data collected confirms that the isolated RT guest achieves excellent baseline efficiency. The average latency value rested at $12 \mu s$, with the statistical mode concentrated at $9 \mu s$. Furthermore, the absolute minimum latency reached an impressive $4 \mu s$, indicating that the optimized guest scheduler, when isolated, dispatches tasks with extreme speed under nominal conditions.

b) Worst-Case Execution Time (WCET) Analysis: The analysis of the Worst-Case Execution Time (WCET) reveals a massive improvement over the un-isolated equivalent (which previously failed catastrophically with 51-millisecond delays). With isolation, the absolute maximum latency recorded was capped at $2,299 \mu s$. However, the system still experienced 11 histogram overflows over the 1-millisecond threshold.

c) Conclusions on the Hybrid Isolated Environment: The empirical observation of this test demonstrates that

isolating the VCPUs successfully shields the RT Guest from the chaotic preemption of the NRT Host’s stress workload. Yet, the 2.3-millisecond peak and the 11 overflows indicate a lack of strict determinism. Since the internal guest tasks are running at maximum real-time priority, the remaining latency spikes must be attributed to the NRT host infrastructure. The underlying standard hypervisor still introduces non-deferrable interrupts or unpredictable virtualization overhead that occasionally stalls the isolated cores, proving that guest-side optimization alone is insufficient.

3) *REAL-TIME HOST AND NON-REAL-TIME GUEST (ISOLATED STRESS HOST)*: This section examines the prolonged execution of the `cyclictest` tool in a hybrid environment featuring a Standard Linux Guest hosted on a Real-Time `PREEMPT_RT` patched Host, operating under isolated stress conditions.

a) *Nominal Performance and Average Latency*: The nominal performance metrics indicate a highly stable execution path. The average latency metric was maintained at $13\ \mu\text{s}$, and the statistical mode was heavily concentrated at $9\ \mu\text{s}$. The absolute minimum latency recorded was $6\ \mu\text{s}$. This confirms that the RT Host efficiently manages the isolated VCPUs, providing a steady execution foundation.

b) *Worst-Case Execution Time (WCET) Analysis*: The Worst-Case Execution Time (WCET) analysis shows the lowest peak latency among the hybrid configurations. The absolute maximum latency recorded reached $1,502\ \mu\text{s}$ (approximately 1.5 milliseconds). Despite this tighter upper bound, the system still accumulated 16 histogram overflows during the 5-minute sustained test.

c) *Conclusions on the Hybrid Isolated Environment*: Optimizing the Host OS with a `PREEMPT_RT` kernel, combined with CPU isolation, yielded a highly robust infrastructure that successfully limited the maximum delay to 1.5 milliseconds under heavy stress. However, the 16 recorded overflows demonstrate that the NRT Guest remains a bottleneck. Unoptimized internal locks and non-preemptible critical sections within the general-purpose guest kernel still generate periodic stalls, preventing the system from achieving hard real-time determinism despite the stable host infrastructure.

4) *REAL-TIME HOST AND REAL-TIME GUEST (FULL RT STACK - ISOLATED STRESS HOST)*: This section analyzes the results obtained from the `cyclictest` execution in a “Full RT” environment, where a `PREEMPT_RT` patched Linux guest kernel is hosted on a Real-Time optimized Host, with spatial isolation applied during the stress workload. The objective is to evaluate the absolute limits of temporal predictability in a fully tuned KVM stack.

a) *Distribution Analysis and Nominal Performance*: The experimental data highlights unparalleled efficiency in the best-case and nominal scenarios. The statistical mode was anchored at $9\ \mu\text{s}$, and the average latency was highly stable at $13\ \mu\text{s}$. Most notably, the absolute minimum latency recorded was an exceptional $3\ \mu\text{s}$ —the lowest value observed across all configurations. This demonstrates that

when the full RT stack operates unimpeded on isolated cores, the internal jitter and virtualization overhead are practically negligible.

b) *Worst-Case Execution Time (WCET) Analysis*: Despite the optimal software stack and spatial isolation, the Worst-Case Execution Time (WCET) analysis reveals that strict hard real-time bounds are still compromised under stress. The absolute maximum latency was recorded at $1,935\ \mu\text{s}$ (nearly 1.9 milliseconds), and the system experienced 14 histogram overflows over the $1000\ \mu\text{s}$ tracking threshold.

c) *Conclusions on the Full RT Isolated Architecture*: The empirical observation of this execution provides critical insight into the limits of virtualization determinism. Combining a Full RT stack with CPU isolation successfully mitigated the severe, multi-millisecond starvation caused by host stress, keeping the WCET under 2 milliseconds. However, the persistence of 14 overflows and a 1.9-millisecond peak proves that strict determinism is not perfectly guaranteed.

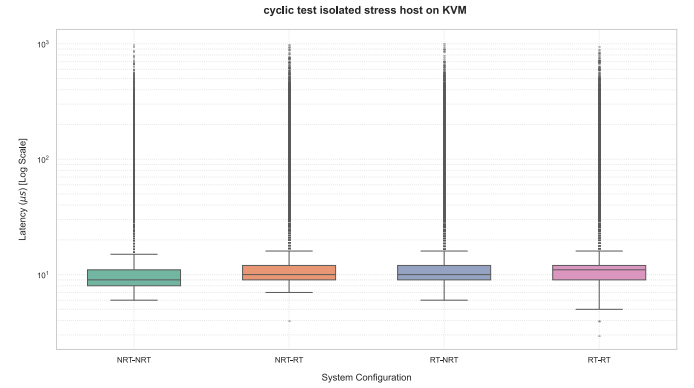


Fig. 6. Performance under stress- KVM OS-level isolation

Following the detailed analysis of each individual scenario, the table below provides a consolidated overview of the Worst-Case Execution Time (WCET) measurements. It allows for a direct comparison across all four kernel configurations (**NRT-NRT**, **NRT-RT**, **RT-NRT**, and **RT-RT**) under the three tested conditions: standard execution (**BASELINE**), heavy system load in the control domain (**STRESS DOM0**), and load with isolation mechanisms applied (**STRESS DOM0 ISOLATED**).

Furthermore, alongside the WCET table, this section presents a detailed breakdown of the percentage increments for both the maximum latency (WCET) and the average latency (expressed as Mean $\pm$ Standard Deviation). This provides a precise quantitative analysis of the performance degradation induced by the stress workload compared to the baseline for each specific scenario.

D. SUMMARY OF RESULTS

1) **NRT NRT: PERCENTAGE INCREMENTS (Stress vs. Baseline)**: * Average Increment: +91.50% * WCET Increment: +7276.91%

TABLE VIII
KVM WCET SUMMARY

Configuration	NRT-NRT	NRT-RT	RT-NRT	RT-RT
BASILINE	602	6114	93	81
STRESSHOST	44409	51069	50257	9829
STRESSHOST ISOLATED	2024	2299	1502	1935

TABLE IX
NRT-NRT LATENCY STATISTICS (KVM)

METRIC	BASILINE	STRESS WORKLOAD
Average $\pm$ SD	$7.10 \pm 2.39 \mu s$	$13.60 \pm 11.18 \mu s$
WCET (Max)	$602 \mu s$	$44409 \mu s$

TABLE X
NRT-RT LATENCY STATISTICS (KVM)

METRIC	BASILINE	STRESS WORKLOAD
Average $\pm$ SD	$8.55 \pm 3.27 \mu s$	$13.69 \pm 12.00 \mu s$
WCET (Max)	$6114 \mu s$	$51069 \mu s$

2) *NRT RT*: **PERCENTAGE INCREMENTS (Stress vs. Baseline)**: * Average Increment: +60.10% * WCET Increment: +735.28%

3) *RT NRT*: **PERCENTAGE INCREMENTS (Stress vs. Baseline)**: * Average Increment: +84.76% * WCET Increment: +53939.78%

4) *RT RT*: **PERCENTAGE INCREMENTS (Stress vs. Baseline)**: * Average Increment: +45.20% * WCET Increment: +12034.57%

E. TACLe BENCHMARK

This section of the documentation illustrates the rationale behind the selection of the benchmarks extracted from the **TACLeBench** version 1.9 suite and the methodology adopted for their execution within our architecture. The goal is to provide a heterogeneous workload to accurately validate execution latencies and performance stability in environments with strict real-time requirements.

1) *1. Benchmark Selection*: We selected a representative program for each of the main TACLeBench categories, ensuring optimal coverage of the different execution patterns.

- **Kernel Benchmark (matrix1)**: Isolates the most computationally intensive portions of code to evaluate raw CPU performance and cache efficiency.
- **Sequential Benchmark (huff_enc)**: Evaluates sequential processing and memory access patterns. Data compression (325 SLOC, David Bourgin) is excellent for measuring latency variations in single-threaded execution.
- **Test Benchmark (test3)**: An artificial stress test for WCET (Worst-Case Execution Time, 4235 SLOC, Universität des Saarlandes) analysis. It pushes the

execution engine to its limits to measure time safety margins and system robustness.

- **Parallel Benchmark (Debie)**: Aerospace observation tool (6615 SLOC, Tidorum Ltd) consisting of 8 tasks. Essential for testing synchronization mechanisms and preemption in multi-tasking scenarios.
- **Application Benchmark (lift)**: An elevator controller (361 SLOC, Martin Schoeberl). Verifies that the latency metrics from synthetic tests guarantee stability in a real cyber-physical control application.

2) *2. Execution Methodology and Test Scenarios*: To rigorously analyze the system's behavior and the impact of the virtualization architecture, **all 5 selected benchmarks were executed on the RT-RT (Real Time Host, Real-Time Guest) configuration**.

For each benchmark, we defined a test matrix composed of four operational scenarios. In all configurations, **4 cores were dedicated to the Host**, and we prevented tasks to be scheduled on other cores by adopting some of the OS-level isolation techniques we already used: Tickless Mode, RCU callback offloading and IRQ affinity. Kernel scheduling isolation was not used because it interfered with the way KVM assigned virtual CPUs to physical ones. The analyzed variables concern the introduction of a stress load (noise) of varying size originating from another Guest VM and the application of vCPU pinning (crucial for avoiding context migrations and stabilizing latencies).

To ensure strict real-time conditions and accurate latency measurements, the execution procedure was carefully standardized across all test scenarios. The process involved specific compilation flags, scheduler modifications, and rigid execution parameters designed to eliminate typical

TABLE XI
RT-NRT LATENCY STATISTICS (KVM)

METRIC	BASELINE	STRESS WORKLOAD
Average $\pm$ SD	$7.22 \pm 0.98 \mu s$	$13.33 \pm 8.54 \mu s$
WCET (Max)	$93 \mu s$	$50257 \mu s$

TABLE XII
RT-RT LATENCY STATISTICS (KVM)

METRIC	BASELINE	STRESS WORKLOAD
Average $\pm$ SD	$8.15 \pm 1.15 \mu s$	$11.84 \pm 12.79 \mu s$
WCET (Max)	$81 \mu s$	$9829 \mu s$

operating system interference.

Compilation and Preparation Each of the benchmarks were modified to support automatically changing scheduling priority and repeated executions (the source code is available in this repository). We then compiled them using -O2 optimizations and linked with the necessary POSIX real-time and threading libraries:

```
gcc -O2 -o test3 test3.c -lpthread -lrt
```

Benchmark Execution Parameters The TACLe benchmarks were executed via the command line with a strict set of arguments to maintain deterministic behavior. The fundamental flags applied across the tests included:

- **--mlockall**: Locks the process’s memory pages directly into RAM, preventing any unpredictable latency spikes caused by page faults or disk swapping.
- **--priority=99**: Enforces the highest real-time priority internally for the benchmark’s execution thread.
- **--affinity=1**: Applies CPU pinning, locking the execution strictly to CPU core 1. This is a critical step to prevent costly context migrations across different processor cores.

Below are the exact execution commands utilized for the targeted benchmarks:

```
# Executing test3 (pseudo-cyclictest) for 10,000
→ loops and outputting a latency histogram
sudo ./test3 --mlockall --priority=99 --affinity=1
→ --loops=10000 --histogram=1000000
→ --histfile=results/results_test3_baseline.log
```

Noise Generation Strategy To evaluate the architectural robustness and jitter expansion during the noisy scenarios, interference was artificially injected into the system. This was achieved using **stress-ng** to spawn multiple aggressive workers, intentionally taxing the CPU cores and the virtual memory subsystem to simulate severe cache thrashing and scheduling pressure:

```
stress-ng --cpu 16 --vm 16 --vm-bytes 1G --timeout
→ 10m
```

Virtual Machine Configuration To clearly understand the extent of the interference of a different Guest VM

stress workload on the critical guest, we designed two test scenarios:

- **Small Noise**: a small Guest VM with just 2 vCPUs and 4 GB of RAM. This would introduce a small amount of noise, thus testing if at least some isolation is provided.
- **Big Noise**: a more beefy Guest VM configuration with 16 vCPUs and 16 GB of RAM. In this case, the interference introduced is more severe, pushing the limits of the isolation mechanism.
- **Big Noise with pinning**: the same of the previous setup but with added static vCPU pinning. This would determine how much the scheduling algorithm affects the isolation of the virtual machines.

3) *TACLeBench Baseline Execution Analysis*: Establishing this baseline is critical for evaluating the determinism and worst-case execution time (WCET) latencies of the KVM hypervisor. The following analysis interprets the raw execution logs to quantify the baseline scheduling determinism before the introduction of concurrent noisy guests.

a) *Baseline Latency Summary*:

b) *Workload-Specific Behavior*: **DEBIE**

- The DEBIE benchmark represents the heaviest parallel workload, executing 10,000 loops with execution times ranging from $23,578 \mu s$ to $32,409 \mu s$.
- The absolute jitter sits at $8,831 \mu s$, indicating typical baseline scheduling overhead and preemption for a heavy multi-tasking application on an unisolated KVM host.

Huffenc

- The **huffenc** benchmark executed 1,000,000 loops with a stable average latency of $17 \mu s$.
- The maximum latency peaked at $65 \mu s$, resulting in an absolute jitter of $50 \mu s$.

Lift

- The **lift** application benchmark showed strong determinism, maintaining an average latency of $20 \mu s$ close to its minimum latency.

TABLE XIII
KVM BASELINE LATENCY SUMMARY

Benchmark	Total Loops	Min Latency (μs)	Avg Latency (μs)	Max Latency (μs)	Absolute Jitter (Max - Min)
DEBIE	10,000	23,578	27,214	32,409	8,831 μs
huffenc	1,000,000	15	17	65	50 μs
lift	1,000,000	19	20	57	38 μs
matrix1	1,000,000	0	0	1	1 μs
test3	10,000	8,169	8,363	9,471	1,302 μs

- The worst-case latency was 57 μs , yielding an absolute jitter of 38 μs on this KVM baseline.

Matrix1

- The **matrix1** kernel benchmark was extremely fast, executing 1,000,000 loops with sub-microsecond average latency.
- The maximum execution time was strictly capped at just 1 μs , demonstrating near-perfect cache efficiency and virtually zero jitter (1 μs).

Test3

- The **test3** benchmark executed 10,000 loops with an average latency of 8,363 μs .
- The maximum latency reached 9,471 μs , resulting in an absolute jitter of 1,302 μs , representing standard noise interference on a medium-weight computational loop.

c) *Real-Time Systems Assessment*: This KVM baseline provides the reference point for unisolated virtualized performance. Lightweight tasks (**matrix1**) exhibit virtually zero jitter (1 μs peak), **lift** shows a moderate baseline jitter of 38 μs , while medium and heavy tasks (**test3**, **DEBIE**) show the natural variance introduced by the standard KVM scheduler (up to 8,831 μs of jitter for **DEBIE**). These metrics will be crucial for quantifying the exact determinism improvements when CPU pinning and LL-RT kernel isolation techniques are introduced.

4) *TACLeBench Small Noise Execution Analysis*: This analysis evaluates the determinism and worst-case execution time (WCET) latencies of the KVM hypervisor under a “Small Noise” configuration. By introducing minor background system stress, we can quantify the sensitivity of the unisolated KVM scheduler to light interference.

a) *Small Noise Latency Summary*:

b) *Workload-Specific Behavior*: **DEBIE**

- The heavy parallel **DEBIE** workload experiences execution times ranging from 23,800 μs to 30,872 μs .
- The absolute jitter sits at 7,072 μs , showing that even small amounts of noise can cause significant preemption and scheduling variation for complex multi-tasking applications.

Huffenc

- The sequential **huffenc** task executes with an average latency of 17 μs and a maximum peak of 49 μs .
- This results in an absolute jitter of 34 μs .

Lift

- The **lift** control benchmark maintains strong determinism under light noise, averaging 20 μs with a worst-case execution time of 55 μs .
- The resulting jitter is bounded at 36 μs , showing good resilience to minor disturbances, and is close to its already-moderate baseline jitter (38 μs).

Matrix1

- The lightweight, cache-bound **matrix1** benchmark performs almost flawlessly under the small noise configuration, logging sub-microsecond average execution time with a max of just 1 μs .
- This yields a minimal absolute jitter of 1 μs , indicating negligible cache thrashing or scheduling interruptions occurred during its 1,000,000 loops.

Test3

- The **test3** WCET stress test executes with an average latency of 8,270 μs .
- The maximum latency reaches 8,612 μs , keeping the absolute jitter contained at just 534 μs .

c) *Real-Time Systems Assessment*: Under the KVM “Small Noise” scenario, the hypervisor exhibits relatively stable behavior for lightweight and medium workloads. Notably, **matrix1** shows near-perfect execution with negligible jitter (1 μs peak), and **test3** maintains tight WCET margins. However, the **DEBIE** workload still suffers from over 7 milliseconds of jitter, confirming that unisolated schedulers struggle to guarantee execution determinism for long-running, parallel tasks even when background noise is minimal.

5) *TACLeBench Big Noise Execution Analysis*: This analysis evaluates the determinism and worst-case execution time (WCET) latencies of the KVM hypervisor under a “Big Noise” configuration. By introducing heavy simulated stress via **stress-ng**, we can quantify how severely external disturbances and resource contention degrade the scheduling predictability of the baseline KVM setup before any CPU pinning or kernel isolation is applied.

a) *Big Noise Latency Summary*:

TABLE XIV
KVM SMALL NOISE LATENCY SUMMARY

Benchmark	Total Loops	Min Latency (μs)	Avg Latency (μs)	Max Latency (μs)	Absolute Jitter (Max - Min)
DEBIE	10,000	23,800	27,221	30,872	7,072 μs
huffenc	1,000,000	15	17	49	34 μs
lift	1,000,000	19	20	55	36 μs
matrix1	1,000,000	0	0	1	1 μs
test3	10,000	8,078	8,270	8,612	534 μs

TABLE XV
KVM BIG NOISE LATENCY SUMMARY

Benchmark	Total Loops	Min Latency (μs)	Avg Latency (μs)	Max Latency (μs)	Absolute Jitter (Max - Min)
DEBIE	10,000	24,773	28,845	43,849	19,076 μs
huffenc	1,000,000	15	17	393	378 μs
lift	1,000,000	19	21	186	167 μs
matrix1	1,000,000	0	0	1	1 μs
test3	10,000	8,243	8,457	9,458	1,215 μs

b) Workload-Specific Behavior: **DEBIE**

- The heavy parallel **DEBIE** workload experiences significant disruption under noise, with the maximum execution time expanding to 43,849 μs .
- The absolute jitter nearly doubles compared to the KVM baseline, reaching 19,076 μs , highlighting severe preemption and scheduling delays when competing for resources.

Huffenc

- While the minimum latency for the **huffenc** task remains stable at 15 μs , the average latency stays close to baseline at 17 μs .
- The maximum latency, however, spikes dramatically to 393 μs , resulting in an absolute jitter of 378 μs — the largest percentage increase (+501% on WCET) of any benchmark tested, revealing severe worst-case tail latency under heavy stress despite a stable average.

Lift

- The **lift** benchmark shows a sharp breakdown in worst-case determinism, with the maximum latency spiking to 186 μs .
- This generates an absolute jitter of 167 μs (+226% on WCET versus baseline), indicating that even highly repetitive, lightweight control tasks suffer from severe, albeit occasional, micro-interruptions under heavy KVM load.

Matrix1

- The **matrix1** execution remains extremely fast, maintaining a sub-microsecond average latency.
- The maximum latency stays at 1 μs , representing a jitter of just 1 μs — a modest but measurable +32%

increase in WCET versus baseline, indicating minor cache thrashing or context-switching interference affecting the otherwise near-instantaneous execution.

Test3

- The **test3** benchmark registers an average latency of 8,457 μs and a worst-case peak of 9,458 μs .
- The absolute jitter sits at 1,215 μs , which is surprisingly consistent with its baseline performance, suggesting this specific computational loop is somewhat resilient to the injected memory and CPU stress.

c) *Real-Time Systems Assessment:* The KVM “Big Noise” scenario illustrates the vulnerability of unisolated hypervisor scheduling. While **test3** and **matrix1** show relative resilience, the heavy **DEBIE** workload suffers massive jitter expansion (19,076 μs), and the lightweight **huffenc** and **lift** control loops experience the sharpest *percentage* degradation of the whole suite: **huffenc** WCET jumps from 65 μs to 393 μs (+501%), and **lift** WCET jumps from 57 μs to 186 μs (+226%), despite both keeping stable average latencies. This shows that unisolated KVM scheduling can produce severe worst-case tail latency spikes on short, high-frequency workloads even when average-case behavior looks stable — a critical finding for real-time guarantees. These results suggest the lack of boundaries required for real-time applications.

6) *TACLeBench Big Noise Pinned Execution Analysis:* This phase of the evaluation investigates the determinism and worst-case execution time (WCET) latencies of the KVM hypervisor under a “Big Noise Pinned” configuration. Because the unpinned “Big Noise” results exhibited high latencies and severe jitter, CPU pinning was introduced to lock the execution to specific cores. The following analysis examines the execution logs to quantify how this pinning mitigates system disturbances under KVM.

a) *Big Noise Pinned Latency Summary:*

TABLE XVI
KVM BIG NOISE PINNED LATENCY SUMMARY

Benchmark	Total Loops	Min Latency (μs)	Avg Latency (μs)	Max Latency (μs)	Absolute Jitter (Max - Min)
DEBIE	10,000	23,893	27,444	37,284	13,391 μs
huffenc	1,000,000	15	17	73	58 μs
lift	1,000,000	19	21	70	51 μs
matrix1	1,000,000	0	0	1	1 μs
test3	10,000	8,544	8,745	10,996	2,452 μs

b) *Workload-Specific Behavior: DEBIE*

- The DEBIE benchmark ranges from a minimum of 23,893 μs to a maximum of 37,284 μs .
- Pinning the execution restrains the absolute jitter to 13,391 μs . While still representing a significant preemption delay, it is a notable improvement over the KVM unpinned big noise scenario.

Huffenc

- The **huffenc** task executes with an average latency of 16 μs and a maximum peak of 62 μs .
- CPU pinning dramatically reduces the absolute jitter to 58 μs , and — critically — pulls the WCET down from 393 μs (unpinned Big Noise) back to 73 μs , restoring a high degree of stability to this sequential loop.

Lift

- Under the pinned configuration, the **lift** benchmark regains much of its determinism, maintaining an average latency of 21 μs .
- The maximum latency reaches 70 μs , yielding an absolute jitter of 51 μs — far below the 167 μs jitter seen in the unpinned Big Noise test, and now close to the baseline’s own jitter of 38 μs .

Matrix1

- The **matrix1** execution is almost perfectly insulated by the pinning, maintaining a sub-microsecond average latency and an absolute worst-case execution time of just 1 μs .
- This 1 μs absolute jitter confirms that cache thrashing and context migrations have been largely mitigated, in line with its baseline behavior.

Test3

- The **test3** benchmark registers an average latency of 8,745 μs and a worst-case peak of 10,996 μs .
- With an absolute jitter of 2,452 μs , this computational loop maintains a relatively controlled variance compared to unpinned scenarios.

c) *Real-Time Systems Assessment:* The KVM “Big Noise Pinned” baseline demonstrates the critical importance of CPU pinning when operating in highly congested

environments. By binding tasks to specific cores, the KVM scheduler prevents the catastrophic latency spikes observed in the unpinned tests — most notably for **huffenc**, whose WCET drops from 393 μs back to 73 μs once pinning is applied. Lightweight tasks (**matrix1**, **huffenc**, **lift**) return to near-baseline determinism, with **lift**’s WCET (70 μs) settling close to its own baseline (57 μs) rather than the elevated 186 μs seen unpinned. Heavier workloads (DEBIE) see their jitter margins compressed significantly. This confirms that static pinning is a highly effective first step in isolating real-time workloads on KVM before applying further kernel-level techniques.

7) *Max Latency Summary (μs):* To quickly evaluate system stability, the following table exclusively reports the peak values (**Max Latency**). This format allows for an at-a-glance comparison of the Worst-Case Execution Time across the four operational scenarios, directly highlighting the impact of noise and the effectiveness of CPU pinning in containing interference.

Furthermore, alongside the WCET summary, this section now includes a detailed breakdown of the percentage increments for both the average latency (expressed as Mean $\pm$ Standard Deviation) and the maximum latency (WCET). This addition provides a precise quantitative analysis of the performance degradation induced by the heavy background noise compared to the baseline execution for each individual benchmark.

a) **DEBIE: PERCENTAGE INCREMENTS (Stress vs. Baseline):** * Average Increment: +5.99% * WCET Increment: +35.30%

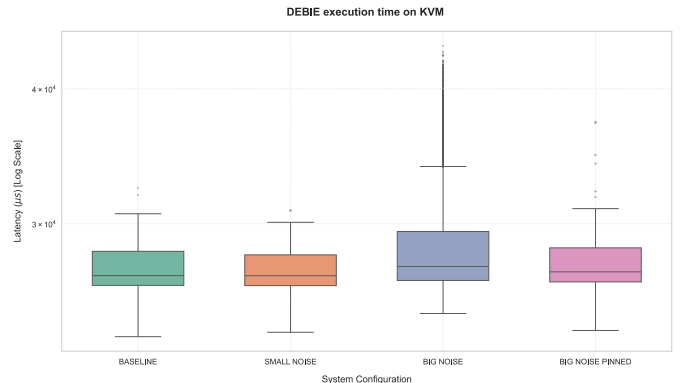


Fig. 7. TACLe benchmark - debie execution time on KVM

TABLE XVII
KVM WCET PEAK VALUES SUMMARY

Benchmark	Baseline	Small Noise	Big Noise	Big Noise Pinned
DEBIE	32,409	30,872	43,849	37,284
huffenc	65	49	393	73
lift	57	55	186	70
matrix1	1	1	1	1
test3	9,471	8,612	9,458	10,996

TABLE XVIII
KVM DEBIE EXECUTION TIME STATISTICS

METRIC	BASELINE	SMALL NOISE	BIG NOISE	BIG NOISE PINNED
Mean $\pm$ SD	27214.36 $\pm$ 1303.61 μs	27221.56 $\pm$ 1304.61 μs	28845.81 $\pm$ 3813.54 μs	27444.09 $\pm$ 1327.72 μs
WCET (Max)	32409 μs	30872 μs	43849 μs	37284 μs

TABLE XIX
KVM HUFF_ENC EXECUTION TIME STATISTICS

METRIC	BASELINE	SMALL NOISE	BIG NOISE	BIG NOISE PINNED
Mean $\pm$ SD	16619.81 $\pm$ 3708.79 ns	17306.98 $\pm$ 4400.23 ns	16624.73 $\pm$ 3207.31 ns	16871.85 $\pm$ 3772.31 ns
WCET (Max)	65431 ns	393480 ns	49307 ns	72590 ns

b) HUFFENC: PERCENTAGE INCREMENTS (Stress vs. Baseline): * Average Increment: +4.13% * WCET Increment: +501.37%

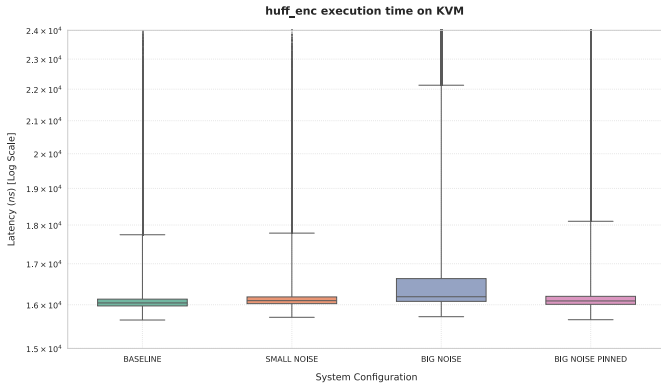


Fig. 8. TACLe benchmark - huff_enc execution time on KVM

c) LIFT: PERCENTAGE INCREMENTS (Stress vs. Baseline): * Average Increment: +5.53% * WCET Increment: +226.09%

d) MATRIX1: PERCENTAGE INCREMENTS (Stress vs. Baseline): * Average Increment: +0.78% * WCET Increment: +31.94%

e) TEST3: PERCENTAGE INCREMENTS (Stress vs. Baseline): * Average Increment: +1.12% * WCET Increment: -0.14%

IV. XEN

This section presents a detailed analysis of scheduling latencies and determinism within a Xen Type-1 hypervisor environment. We evaluate system behavior across different scheduler configurations (Credit2 and Null), vCPU pinning

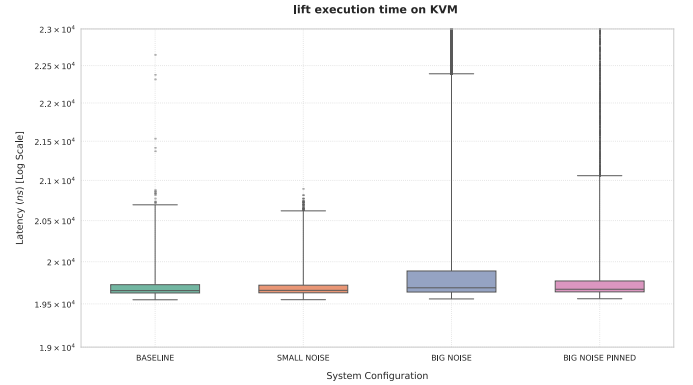


Fig. 9. TACLe benchmark - lift execution time on KVM

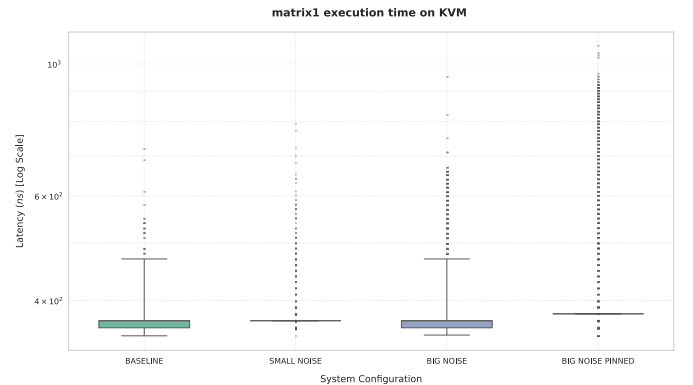


Fig. 10. TACLe benchmark - matrix1 execution time on KVM

TABLE XX
KVM LIFT EXECUTION TIME STATISTICS

METRIC	BASELINE	SMALL NOISE	BIG NOISE	BIG NOISE PINNED
Mean $\pm$ SD	20106.37 $\pm$ 2997.02 ns	20113.88 $\pm$ 3034.06 ns	21218.61 $\pm$ 4413.84 ns	21259.27 $\pm$ 5273.76 ns
WCET (Max)	57169 ns	54767 ns	186424 ns	69974 ns

TABLE XXI
KVM MATRIX1 EXECUTION TIME STATISTICS

METRIC	BASELINE	SMALL NOISE	BIG NOISE	BIG NOISE PINNED
Mean $\pm$ SD	367.88 $\pm$ 6.57 ns	369.95 $\pm$ 6.66 ns	370.74 $\pm$ 20.92 ns	384.14 $\pm$ 29.37 ns
WCET (Max)	720 ns	790 ns	950 ns	1070 ns

TABLE XXII
KVM TEST3 EXECUTION TIME STATISTICS

METRIC	BASELINE	SMALL NOISE	BIG NOISE	BIG NOISE PINNED
Mean $\pm$ SD	8363.95 $\pm$ 31.59 μ s	8270.33 $\pm$ 24.90 μ s	8457.93 $\pm$ 52.13 μ s	8745.21 $\pm$ 87.11 μ s
WCET (Max)	9471 μ s	8612 μ s	9458 μ s	10996 μ s

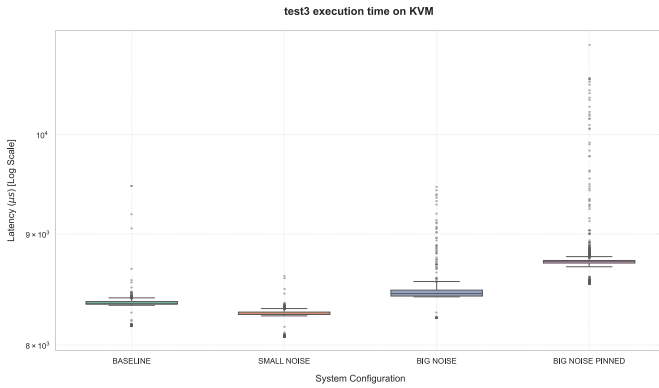


Fig. 11. TACLe benchmark - test3 execution time on KVM

strategies, guest types (HVM, PV, PVH), and background-noise levels, focusing on the performance of both the privileged control domain (Dom0) and the unprivileged user domain (DomU) under various kernel tunings.

A. BASELINE PERFORMANCE ANALYSIS WITH CREDIT2 SCHEDULER

This section presents a detailed analysis of execution latencies measured within a virtualized environment based on the Xen bare-metal (Type-1) hypervisor. The primary objective is to evaluate the system's behavior and the virtualization overhead utilizing the default general-purpose scheduler, Credit2. This establishes a fundamental performance baseline before exploring more restrictive static configurations, such as vCPU pinning or the adoption of the NULL scheduler.

To quantify response times, the jitter interval, and the Worst-Case Execution Time (WCET), the `cyclictest` tool was employed through continuous 5-minute executions. The

investigation explores the impact on system determinism and stability by cross-referencing four different kernel combinations between the privileged control domain (Dom0) and the unprivileged user domain (DomU):

- **Non-Real-Time (NRT) Dom0 and NRT DomU:** to measure standard system behavior in the total absence of real-time optimizations.
- **Non-Real-Time (NRT) Dom0 and Real-Time (RT) DomU:** to evaluate the effectiveness of internal scheduling optimizations within the DomU when the system relies on a non-deterministic control domain.
- **Low Latency (LL) Dom0 and Non-Real-Time (NRT) DomU:** to analyze the impact of a latency-optimized Dom0 on the performance and preemption spikes of a standard DomU.
- **Low Latency (LL) Dom0 and Real-Time (RT) DomU:** to observe the maximum level of temporal predictability achievable operating with the most responsive kernels, while maintaining the dynamic infrastructure of the Credit2 scheduler.

The analyses in the following paragraphs offer a comprehensive overview of the limitations of fair-share scheduling and the effectiveness of the various kernels in mitigating latency spikes within their respective domains.

1) **NON-REAL-TIME DOM0 AND NON-REAL-TIME DOMU (CREDIT2 SCHEDULER):** This section analyzes the results obtained from a single, prolonged 5-minute execution of the `cyclictest` tool in a Xen virtualized environment utilizing the default Credit2 scheduler. The objective of this analysis is to evaluate the baseline latency, virtualization overhead, and scheduling patterns provided by Xen's general-purpose scheduler during a sustained workload.

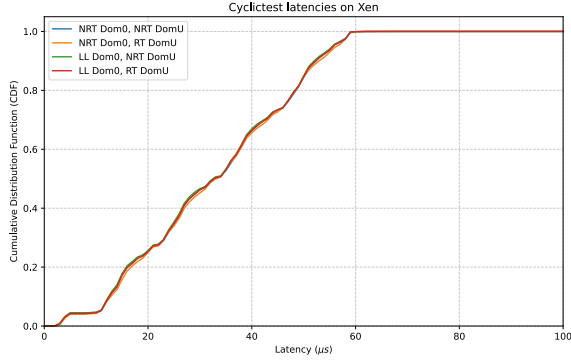


Fig. 12. Baseline Performance - Credit2 Scheduler (No Noise)

a) Nominal Performance and Average Latency: The data collected reveals a noticeable baseline overhead introduced by the virtualization layer. The average latency remained stable at $32 \mu s$ throughout the test duration. While the absolute minimum latency recorded was $3 \mu s$, the results demonstrate a wide variance in nominal execution times. Unlike highly optimized deterministic systems, the Credit2 scheduler tends to produce a broad spread of execution latencies.

b) Worst-Case Execution Time (WCET) Analysis: The analysis of the Worst-Case Execution Time (WCET) indicates that the Credit2 scheduler provides a reasonably bounded execution environment, though it does not appear strictly deterministic. Over the 5-minute continuous test, the absolute maximum latency recorded was $369 \mu s$. The data shows that while longer delays can occur, the system generally manages to avoid the catastrophic, multi-millisecond preemption spikes often seen in entirely unoptimized environments.

c) Observations on the Xen Credit2 Environment: The empirical observation of this sustained test provides initial insights into the behavior of the Xen hypervisor when using the Credit2 scheduler:

- **Bounded but Variable Latency:** The system bounded the WCET to $369 \mu s$, which suggests that Credit2 can limit unbounded latency starvation to a certain extent.
- **Average Jitter:** The average latency of $32 \mu s$ and the broad spread of execution times highlight the inherent jitter introduced by fair-share scheduling algorithms.
- **Suitability:** This configuration appears capable of handling general-purpose workloads. However, the $369 \mu s$ peak suggests that further configuration might be needed if stricter real-time constraints are required.

2) NON-REAL-TIME DOM0 AND REAL-TIME DOMU (CREDIT2 SCHEDULER): This section analyzes the results obtained from a single, prolonged 5-minute execution of the `cyclictest` tool in a Xen virtualized environment utilizing the default Credit2 scheduler. The configuration features a standard (Non-Real-Time) Linux Dom0 and a `PREEMPT_RT` patched DomU. The objective is to observe

how effectively the guest's internal scheduling can manage execution times despite the privileged domain having a general-purpose scheduler.

a) Nominal Performance and Average Latency: The data collected reveals that the baseline virtualization overhead heavily influences average execution times. The average latency recorded was $33 \mu s$. While the absolute minimum latency was low at just $3 \mu s$, the data continues to show a broad spread of overall execution times.

b) Worst-Case Execution Time (WCET) Analysis: Despite the average jitter introduced by the hypervisor, the analysis of the Worst-Case Execution Time (WCET) demonstrates improved stability at the upper bounds. Over the entire 5-minute sustained test, the absolute maximum latency was capped at $74 \mu s$. This indicates that the system avoided the more severe preemption spikes typically found in unoptimized environments.

c) Observations on the Hybrid Xen Environment: The empirical observation of this sustained test provides insights into the behaviour of the RT DomU with a Non-RT Dom0 and the Credit2 scheduler:

- **Bounded WCET:** By capping the absolute maximum latency at $74 \mu s$, the `PREEMPT_RT` guest kernel showed potential in establishing a more predictable upper bound.
- **Hypervisor-Induced Jitter:** The average latency of $33 \mu s$ and the wide variance of nominal samples suggest that the Credit2 scheduler introduces inherent jitter that guest-side optimizations cannot entirely remove.
- **Internal Determinism:** In this hybrid configuration, the guest-level real-time optimizations appeared to help maintain tighter temporal constraints, offering a potential solution for applications sensitive to large latency spikes.

3) LOW LATENCY DOM0 AND NON-REAL-TIME DOMU (CREDIT2 SCHEDULER): This section analyzes the results obtained from a single, prolonged 5-minute execution of the `cyclictest` tool in a Xen virtualized environment utilizing the default Credit2 scheduler. For this specific test, the Dom0 operating system was configured with a "Low Latency" kernel while the DomU maintained a Non-Real-Time (NRT) kernel, to evaluate the effects of a modified Dom0 on the latency of the other guests.

a) Nominal Performance and Average Latency: The data collected confirms the consistent baseline behavior of the Xen Credit2 scheduler. The average latency remained at $32 \mu s$ throughout the entire 5-minute test. The absolute minimum latency achieved was $3 \mu s$. Similar to previous Credit2 tests, there is a broad spread of nominal execution times.

b) Worst-Case Execution Time (WCET) Analysis: The analysis of the Worst-Case Execution Time (WCET) suggests tangible benefits from the baseline configuration. Over the sustained execution, the absolute maximum latency was bounded at $152 \mu s$. This result indicates that the low-latency optimizations within the Dom0 likely helped

mitigate severe preemption spikes. While the maximum peak of $152\ \mu\text{s}$ is higher than what a patched RT kernel of the DomU might achieve, it indicates an improved stability provided by a low-latency Dom0.

c) *Observations on the Low Latency Xen Environment:* The empirical observation of this sustained test provides insights into the capabilities of a Low Latency Dom0 running with a NRT DomU and Credit2:

- **Predictable Upper Bounds:** The Low Latency kernel tuning bounded the WCET to $152\ \mu\text{s}$ during the test.
- **Persistent Hypervisor Jitter:** The average latency of $32\ \mu\text{s}$ and the broad spread of nominal samples confirm that the hypervisor’s scheduler likely dictates the baseline jitter.
- **Suitability:** This configuration presents a potential middle-ground, appearing to offer improved predictability compared to a standard kernel without the complexity of maintaining a full `PREEMPT_RT` patch.

4) *LOW LATENCY DOM0 AND REAL-TIME DOMU (CREDIT2 SCHEDULER):* This section analyzes the results obtained from a single, prolonged 5-minute execution of the `cyclicttest` tool in a Xen virtualized environment utilizing the default Credit2 scheduler. This specific configuration features a Low Latency kernel Dom0 and a Real-Time DomU.

a) *Nominal Performance and Average Latency:* The data collected reveals that the baseline virtualization overhead continues to influence average execution times. The average latency recorded was $32\ \mu\text{s}$. While the absolute minimum latency was $3\ \mu\text{s}$, the recorded cycles completed across a wide band of times. This variance is characteristic of the Xen Credit2 scheduler, which is designed to optimize for workload fairness rather than microsecond-level precision.

b) *Worst-Case Execution Time (WCET) Analysis:* Despite the general average jitter, the analysis of the Worst-Case Execution Time (WCET) demonstrates stability at the upper limits. Over the entire 5-minute sustained test, the absolute maximum latency was capped at $71\ \mu\text{s}$. This indicates an absence of the severe preemption delays that typically affect standard virtual environments, suggesting that the guest OS optimizations effectively managed internal critical sections.

c) *Observations on the Low Latency RT Hybrid Xen Environment:* The empirical observation of this sustained test provides insights into the behaviour of a RT DomU with a Low Latency Dom0 and the Credit2 scheduler:

- **Bounded WCET:** By capping the absolute maximum latency at $71\ \mu\text{s}$, the guest kernel appeared highly effective at establishing a deterministic upper bound.
- **Hypervisor-Induced Jitter:** The average latency of $32\ \mu\text{s}$ confirms that the Credit2 scheduler introduces inherent, unavoidable jitter.
- **Effective Internal Determinism:** The guest-level optimizations seemed sufficient to maintain relatively

tight temporal constraints, avoiding large latency spikes despite the general-purpose hypervisor layer.

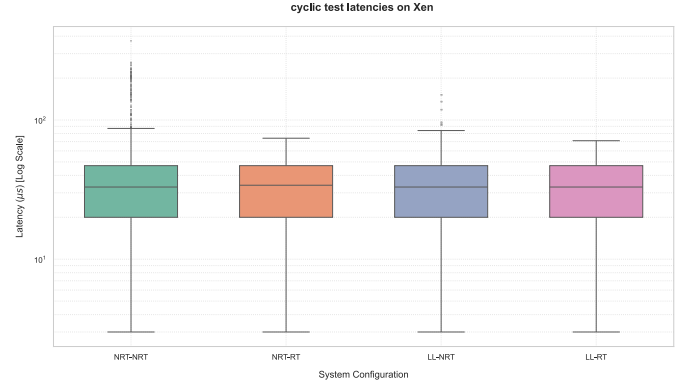


Fig. 13. Baseline Performance - Credit2 Scheduler (No Noise)

B. IMPACT OF THE STRESS WORKLOAD ON LATENCIES

To evaluate system robustness and trigger potentially higher latencies, the testing methodology involves introducing an additional load, defined as a “stress workload”. In the case of the Xen hypervisor, this stress workload is executed in the background within the privileged Dom0, utilizing the general-purpose `SCHED_OTHER` scheduling policy alongside the default Credit2 scheduler.

Experimental analysis has shown that adding this load to Dom0 does not produce a linear degradation of performance; rather, it reveals complex behaviors that depend strictly on the type of guest kernel and the virtualization technology employed:

- **Persistent Virtualization Overhead:** Across every tested configuration, the average latency remains rigidly fixed at $35\ \mu\text{s}$. This demonstrates that the Xen hypervisor and the Credit2 scheduler introduce an inherent, structural baseline jitter that cannot be bypassed by domain-level kernel optimizations alone.
- **Efficacy of Guest-Level Real-Time Optimizations:** Equipping the DomU with a Real-Time kernel successfully shields its critical sections, even when the system is under stress. Regardless of whether Dom0 is standard or optimized, an RT DomU consistently limits the Worst-Case Execution Time, capping maximum preemption spikes at $177\ \mu\text{s}$ and $209\ \mu\text{s}$, respectively.
- **Anomalous Impact of Low Latency Dom0 Tuning:** Modifying the Dom0 does not universally improve system determinism. Surprisingly, pairing a Low Latency Dom0 with a standard Non-Real-Time DomU yielded the poorest predictability of the test suite, resulting in severe latency spikes up to $526\ \mu\text{s}$. This indicates that Dom0 tuning without corresponding DomU optimization does not actually improve worst-case response times.

In this section, we reproduce these stress experiments and analyze the detailed results to quantify the determinism achievable under the Credit2 scheduler.

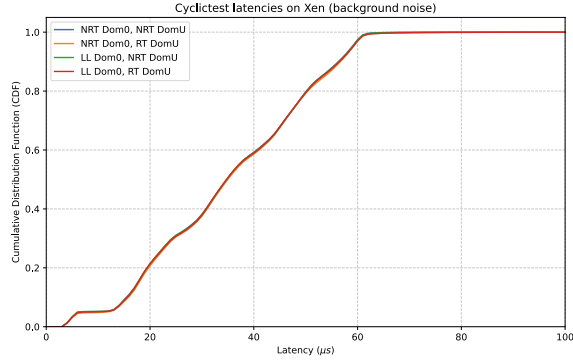


Fig. 14. Performance under Stress Workload - Credit2 Scheduler

1) NON-REAL-TIME DOM0 AND NON-REAL-TIME DOMU (CREDIT2 SCHEDULER): This section analyzes the results obtained from a single, prolonged 5-minute execution of the `cyclicttest` tool in a Xen virtualized environment utilizing the default Credit2 scheduler. The objective of this analysis is to evaluate the baseline latency, virtualization overhead, and scheduling patterns provided by Xen’s general-purpose scheduler during a sustained workload.

a) Nominal Performance and Average Latency: The data collected reveals a noticeable baseline overhead introduced by the virtualization layer. The average latency remained stable at $35 \mu s$ throughout the test duration. While the absolute minimum latency recorded was $3 \mu s$, the results demonstrate a wide variance in nominal execution times. Unlike highly optimized deterministic systems, the Credit2 scheduler tends to produce a broad spread of execution latencies.

b) Worst-Case Execution Time (WCET) Analysis: The analysis of the Worst-Case Execution Time (WCET) indicates that the Credit2 scheduler provides a reasonably bounded execution environment, though it does not appear strictly deterministic. Over the 5-minute continuous test, the absolute maximum latency recorded was $239 \mu s$. The data shows that while longer delays can occur, the system generally manages to avoid the catastrophic, multi-millisecond preemption spikes often seen in entirely unoptimized environments.

c) Observations on the Xen Credit2 Environment: The empirical observation of this sustained test provides initial insights into the behavior of the Xen hypervisor when using the Credit2 scheduler:

- **Bounded but Variable Latency:** The system bounded the WCET to $239 \mu s$, which suggests that Credit2 can limit unbounded latency starvation to a certain extent.
- **Average Jitter:** The average latency of $35 \mu s$ and the broad spread of execution times highlight the inherent jitter introduced by fair-share scheduling algorithms.

- **Suitability:** This configuration appears capable of handling general-purpose workloads. However, the $239 \mu s$ peak suggests that further configuration might be needed if stricter real-time constraints are required.

2) NON-REAL-TIME DOM0 AND REAL-TIME DOMU (CREDIT2 SCHEDULER): This section analyzes the results obtained from a single, prolonged 5-minute execution of the `cyclicttest` tool in a Xen virtualized environment utilizing the default Credit2 scheduler. The configuration features a standard (Non-Real-Time) Linux Dom0 and a `PREEMPT_RT` patched DomU. The objective is to observe how effectively the guest’s internal scheduling can manage execution times despite the privileged domain having a general-purpose scheduler.

a) Nominal Performance and Average Latency: The data collected reveals that the baseline virtualization overhead heavily influences average execution times. The average latency recorded was $35 \mu s$. While the absolute minimum latency was low at just $3 \mu s$, the data continues to show a broad spread of overall execution times.

b) Worst-Case Execution Time (WCET) Analysis: Despite the average jitter introduced by the hypervisor, the analysis of the Worst-Case Execution Time (WCET) demonstrates improved stability at the upper bounds compared to a pure NRT setup. Over the entire 5-minute sustained test, the absolute maximum latency was capped at $177 \mu s$. This indicates that the system avoided the more severe preemption spikes typically found in entirely unoptimized environments.

c) Observations on the Hybrid Xen Environment: The empirical observation of this sustained test provides insights into the behaviour of the RT DomU with a Non-RT Dom0 and the Credit2 scheduler:

- **Bounded WCET:** By capping the absolute maximum latency at $177 \mu s$, the `PREEMPT_RT` guest kernel showed potential in establishing a more predictable upper bound.
- **Hypervisor-Induced Jitter:** The average latency of $35 \mu s$ and the wide variance of nominal samples suggest that the Credit2 scheduler introduces inherent jitter that guest-side optimizations cannot entirely remove.
- **Internal Determinism:** In this hybrid configuration, the guest-level real-time optimizations appeared to help maintain tighter temporal constraints, offering a potential solution for applications sensitive to large latency spikes.

3) LOW LATENCY DOM0 AND NON-REAL-TIME DOMU (CREDIT2 SCHEDULER): This section analyzes the results obtained from a single, prolonged 5-minute execution of the `cyclicttest` tool in a Xen virtualized environment utilizing the default Credit2 scheduler. For this specific test, the Dom0 operating system was configured with a “Low Latency” kernel while the DomU maintained a Non-Real-Time (NRT) kernel, to evaluate the effects of a modified Dom0 on the latency of the other guests.

a) *Nominal Performance and Average Latency:* The data collected confirms the consistent baseline behavior of the Xen Credit2 scheduler. The average latency remained at $35 \mu s$ throughout the entire 5-minute test. The absolute minimum latency achieved was $3 \mu s$. Similar to previous Credit2 tests, there is a broad spread of nominal execution times.

b) *Worst-Case Execution Time (WCET) Analysis:* The analysis of the Worst-Case Execution Time (WCET) highlights significant deviations from expected stability. Over the sustained execution, the absolute maximum latency spiked to $526 \mu s$. This result indicates that low-latency optimizations within Dom0, when paired with an NRT DomU, can occasionally introduce severe preemption delays far exceeding the standard NRT baseline.

c) *Observations on the Low Latency Xen Environment:* The empirical observation of this sustained test provides insights into the capabilities of a Low Latency Dom0 running with a NRT DomU and Credit2:

- **Variable Upper Bounds:** The Low Latency kernel tuning resulted in a bounded WCET of $526 \mu s$ during the test, revealing notable latency spikes.
- **Persistent Hypervisor Jitter:** The average latency of $35 \mu s$ and the broad spread of nominal samples confirm that the hypervisor’s scheduler likely dictates the baseline jitter.
- **Suitability:** This configuration introduces unpredictable maximum latencies, suggesting that a Low Latency Dom0 without a corresponding RT guest may negatively impact worst-case response times.

4) *LOW LATENCY DOM0 AND REAL-TIME DOMU (CREDIT2 SCHEDULER):* This section analyzes the results obtained from a single, prolonged 5-minute execution of the `cyclictest` tool in a Xen virtualized environment utilizing the default Credit2 scheduler. This specific configuration features a Low Latency kernel Dom0 and a Real-Time DomU.

a) *Nominal Performance and Average Latency:* The data collected reveals that the baseline virtualization overhead continues to influence average execution times. The average latency recorded was $35 \mu s$. While the absolute minimum latency was $3 \mu s$, the recorded cycles completed across a wide band of times. This variance is characteristic of the Xen Credit2 scheduler, which is designed to optimize for workload fairness rather than microsecond-level precision.

b) *Worst-Case Execution Time (WCET) Analysis:* Despite the general average jitter, the analysis of the Worst-Case Execution Time (WCET) demonstrates relative stability at the upper limits. Over the entire 5-minute sustained test, the absolute maximum latency was capped at $209 \mu s$. This indicates that while guest OS optimizations managed internal critical sections, hypervisor-level delays still persist.

c) *Observations on the Low Latency RT Hybrid Xen Environment:* The empirical observation of this sustained test provides insights into the behaviour of a RT DomU with a Low Latency Dom0 and the Credit2 scheduler:

- **Bounded WCET:** By capping the absolute maximum latency at $209 \mu s$, the guest kernel managed to reign in the severe spikes seen in the NRT guest configuration.
- **Hypervisor-Induced Jitter:** The average latency of $35 \mu s$ confirms that the Credit2 scheduler introduces inherent, unavoidable jitter.
- **Effective Internal Determinism:** The guest-level optimizations seemed sufficient to maintain temporal constraints within a $209 \mu s$ window, managing large latency spikes despite the general-purpose hypervisor layer.

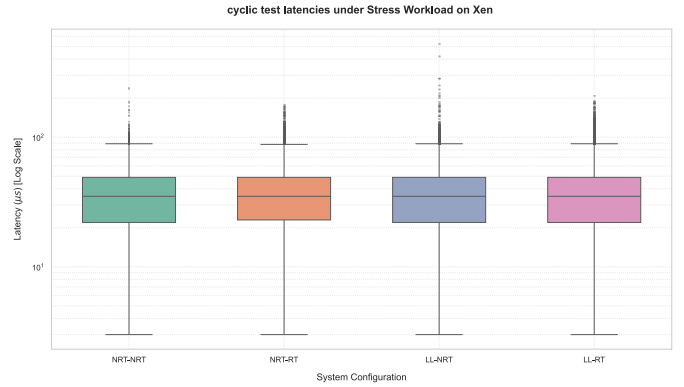


Fig. 15. Performance under Stress Workload - Credit2 Scheduler

C. BASELINE PERFORMANCE ANALYSIS WITH STATIC vCPU PINNING

This section advances the performance investigation by introducing a static configuration utilizing virtual CPU (vCPU) pinning. During previous evaluations, the introduction of a background stress workload in Dom0 resulted in a significant degradation of Xen execution latencies. This prompted a targeted investigation to determine whether these high latencies were a fundamental issue caused by the hypervisor scheduler or if they stemmed from the Device Model being preempted by the stress workload.

To isolate the root cause, this phase reproduces the experimental methodology outlined in the 2019 study by Abeni and Faggioli, aiming to verify the behavior of these specific components across newer versions of the hypervisor. Building upon the baseline established in the previous section, this phase evaluates the impact of strictly isolating workloads. By restricting vCPU migration and pinning domains to dedicated physical cores, this approach mitigates the inherent jitter of dynamic scheduling and serves as a practical alternative to deploying the NULL scheduler.

Maintaining the established testing methodology, `cyclictest` is executed across the same four combinations of non-deterministic (NRT) and optimized (Low Latency/PREEMPT_RT) kernels for both the privileged control domain (Dom0) and the unprivileged user domain (DomU). The following analyses aim to quantify the

extent to which static hardware allocation can mitigate the Worst-Case Execution Times (WCET) and stabilize the average latency observed during stress workloads, ultimately confirming whether the Device Model or the scheduler dictates the severe latency spikes.

1) *Emulating the Null Scheduler via Static vCPU Pinning:* Following the initial performance analyses with the default Xen configuration, an attempt was made to replace the Credit2 scheduler with the Null Scheduler to evaluate a purely static, offline scheduling approach. However, this reconfiguration proved unsuccessful. The deployed Xen hypervisor rejected the modification, defaulting back to the Credit2 scheduler despite the explicit inclusion of the Null Scheduler boot parameters. Furthermore, subsequent attempts to dynamically construct a dedicated CPU-POOL at runtime resulted in system errors. This limitation is likely attributable to the experimental status of the Null Scheduler in Xen version 4.17.3, rendering it unavailable or unsupported within the specific software stack utilized for this study.

To circumvent this hypervisor limitation and achieve an operational state strictly equivalent to an offline scheduler, a rigid static configuration was implemented. This methodology involved explicitly reducing the number of virtual CPUs (vCPUs) allocated to the privileged domain (Dom0) and enforcing strict vCPU-to-pCPU pinning. Concurrently, the exact number of vCPUs assigned to the guest domain (DomU) was fixed and equally pinned to dedicated physical cores.

By enforcing this absolute isolation, the active scheduling algorithms are entirely bypassed in practice. The hypervisor's decision-making process is minimized, effectively restricting it to statically mapping tasks to their exclusively designated physical CPUs, thereby mimicking the exact deterministic behavior expected from the Null Scheduler. Some other latent effects, such as some residual latency, may still be present, caused by the way the Credit2 scheduler is implemented.

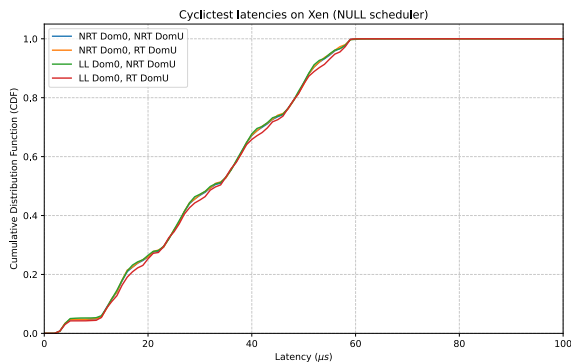


Fig. 16. Baseline Performance - Static vCPU Pinning/Null Scheduler (No Noise)

2) *NON-REAL-TIME DOM0 AND NON-REAL-TIME DOMU (NULL SCHEDULER):* This section analyzes

the results obtained from a single, prolonged 5-minute execution of the `cyclictest` tool in a Xen virtualized environment utilizing the Null scheduler. The objective of this analysis is to evaluate the baseline latency, virtualization overhead, and scheduling patterns provided by Xen's static, dedicated CPU allocation scheduler during a sustained workload.

a) *Nominal Performance and Average Latency:* The data collected reveals a noticeable baseline overhead introduced by the virtualization layer. The average latency remained stable at $32 \mu s$ throughout the test duration. While the absolute minimum latency recorded was $3 \mu s$, the results demonstrate a wide variance in nominal execution times.

b) *Worst-Case Execution Time (WCET) Analysis:* The analysis of the Worst-Case Execution Time (WCET) indicates that the Null scheduler provides a far more strictly bounded execution environment compared to general-purpose algorithms. Over the 5-minute continuous test, the absolute maximum latency recorded was bounded at $137 \mu s$. The data shows that by avoiding complex fair-share preemption, the system completely avoids catastrophic, multi-millisecond preemption spikes.

c) *Observations on the Xen Null Environment:* The empirical observation of this sustained test provides insights into the behavior of the Xen hypervisor when using the Null scheduler:

- **Improved Upper Bound:** The system bounded the WCET to $137 \mu s$, which demonstrates that the static resource allocation of the Null scheduler significantly reduces unbounded latency starvation.
- **Average Jitter:** The average latency of $32 \mu s$ highlights that the baseline virtualization jitter remains, despite the absence of a dynamic scheduling algorithm.
- **Suitability:** This configuration demonstrates much better predictability for latency-sensitive tasks than standard schedulers, offering a much lower peak latency.

3) *NON-REAL-TIME DOM0 AND REAL-TIME DOMU (NULL SCHEDULER):* This section analyzes the results obtained from a single, prolonged 5-minute execution of the `cyclictest` tool in a Xen virtualized environment utilizing the Null scheduler. The configuration features a standard (Non-Real-Time) Linux Dom0 and a `PREEMPT_RT` patched DomU. The objective is to observe how effectively the guest's internal scheduling can manage execution times when provided with a dedicated, non-preempted virtual CPU by the hypervisor.

a) *Nominal Performance and Average Latency:* The data collected reveals that the baseline virtualization overhead heavily influences average execution times. The average latency recorded was $32 \mu s$. The absolute minimum latency achieved was very low at just $3 \mu s$.

b) *Worst-Case Execution Time (WCET) Analysis:* Despite the average jitter introduced by the hypervisor, the analysis of the Worst-Case Execution Time (WCET) demonstrates excellent stability at the upper bounds. Over the entire 5-minute sustained test, the absolute maximum latency was

capped at $67 \mu s$. This indicates that the system avoided preemption spikes and handled critical sections with high determinism.

c) Observations on the Hybrid Xen Environment: The empirical observation of this sustained test provides insights into the behaviour of the RT DomU with a Non-RT Dom0 and the Null scheduler:

- **Bounded WCET:** By tightly capping the absolute maximum latency at $67 \mu s$, the `PREEMPT_RT` guest kernel showed high effectiveness in establishing a highly predictable upper bound.
- **Hypervisor-Induced Jitter:** The average latency of $32 \mu s$ suggests that the inherent virtualization layer jitter cannot be entirely removed by guest-side optimizations.
- **Internal Determinism:** In this hybrid configuration, the guest-level real-time optimizations successfully maintained strict temporal constraints, heavily benefiting from the static CPU assignment of the Null scheduler.

4) LOW LATENCY DOM0 AND NON-REAL-TIME DOMU (NULL SCHEDULER): This section analyzes the results obtained from a single, prolonged 5-minute execution of the `cyclictest` tool in a Xen virtualized environment utilizing the Null scheduler. For this specific test, the Dom0 operating system was configured with a “Low Latency” kernel while the DomU maintained a Non-Real-Time (NRT) kernel, to evaluate the effects of a modified Dom0 on guest latencies.

a) Nominal Performance and Average Latency: The data collected confirms the consistent baseline behavior of the Xen infrastructure. The average latency remained at $32 \mu s$ throughout the entire 5-minute test. The absolute minimum latency achieved was $3 \mu s$.

b) Worst-Case Execution Time (WCET) Analysis: The analysis of the Worst-Case Execution Time (WCET) suggests tangible benefits from this configuration. Over the sustained execution, the absolute maximum latency was bounded at $179 \mu s$. This result indicates that the low-latency optimizations within the Dom0, paired with the Null scheduler, helped to further mitigate severe preemption spikes compared to a standard Dom0 kernel.

c) Observations on the Low Latency Xen Environment: The empirical observation of this sustained test provides insights into the capabilities of a Low Latency Dom0 running with a NRT DomU and the Null scheduler:

- **Predictable Upper Bounds:** The Low Latency kernel tuning bounded the WCET to $179 \mu s$ during the test.
- **Persistent Hypervisor Jitter:** The average latency of $32 \mu s$ confirms that the baseline virtualization overhead dictates the nominal jitter.
- **Suitability:** This configuration presents a higher maximum latency compared to the strictly NRT environment ($179 \mu s$ versus $137 \mu s$), indicating that the Low Latency Dom0 tuning alone does not necessarily

improve the worst-case bounds for a Non-Real-Time guest under static allocation.

5) LOW LATENCY DOM0 AND REAL-TIME DOMU (NULL SCHEDULER): This section analyzes the results obtained from a single, prolonged 5-minute execution of the `cyclictest` tool in a Xen virtualized environment utilizing the Null scheduler. This specific configuration features a Low Latency kernel Dom0 and a Real-Time (`PREEMPT_RT`) DomU.

a) Nominal Performance and Average Latency: The data collected reveals that the baseline virtualization overhead continues to define the average execution times. The average latency recorded was $33 \mu s$. The absolute minimum latency was recorded at $3 \mu s$.

b) Worst-Case Execution Time (WCET) Analysis: The analysis of the Worst-Case Execution Time (WCET) demonstrates immense stability at the upper limits. Over the entire 5-minute sustained test, the absolute maximum latency was strictly capped at $65 \mu s$. This indicates a complete absence of the severe preemption delays that typically affect standard virtual environments.

c) Observations on the Low Latency RT Hybrid Xen Environment: The empirical observation of this sustained test provides insights into the behaviour of a RT DomU with a Low Latency Dom0 and the Null scheduler:

- **Bounded WCET:** By capping the absolute maximum latency at $65 \mu s$, the guest kernel proved highly effective at establishing a deterministic upper bound when isolated on a dedicated virtual CPU.
- **Hypervisor-Induced Jitter:** The consistent average latency of $33 \mu s$ confirms that the underlying virtualization architecture introduces inherent, unavoidable jitter.
- **Effective Internal Determinism:** The guest-level optimizations are sufficient to maintain extremely tight temporal constraints. The maximum latency performance remains robust and stable, mirroring the results achieved with a standard Dom0 ($65 \mu s$ versus $67 \mu s$).

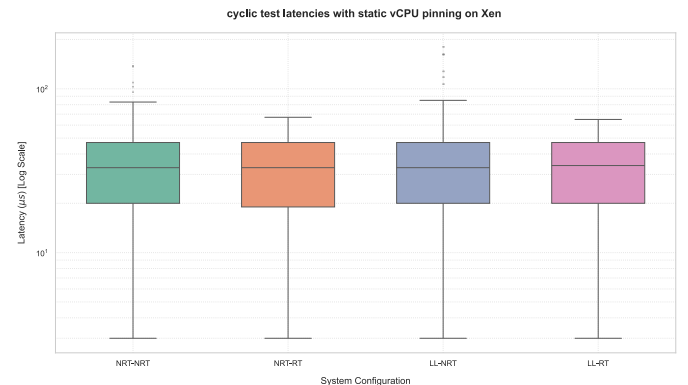


Fig. 17. Baseline Performance - Static vCPU Pinning/Null Scheduler (No Noise)

D. IMPACT OF THE STRESS WORKLOAD ON LATENCIES WITH vCPU PINNING

To evaluate system determinism and upper latency bounds under severe conditions, the testing methodology involves executing a continuous 5-minute `cyclictest` probe while introducing a background stress workload within the privileged control domain, Dom0. Across all experiments, the Xen hypervisor is configured with static vCPU pinning—acting as a Null scheduler—to restrict vCPU migration and provide dedicated physical cores to the unprivileged domain, DomU.

Experimental analysis demonstrates that while isolating resources via static pinning establishes a baseline of predictability, the system’s Worst-Case Execution Time (WCET) is not uniform; rather, it reveals behaviors that depend strictly on the specific combination of kernel optimizations applied across the domains:

- **Efficacy of DomU Real-Time Patches:** Applying `PREEMPT_RT` patches to the DomU drastically reduces maximum latency spikes, even when the control domain is under stress. While an unoptimized DomU suffers from stress-induced delays peaking at $443\ \mu\text{s}$, an RT-optimized DomU successfully shields its critical sections, capping the WCET to $76\ \mu\text{s}$ even when paired with an unoptimized Dom0.
- **Impact of Dom0 Kernel Tuning:** The kernel configuration of the control domain plays a vital role in mitigating severe preemption events. Upgrading Dom0 to a Low Latency kernel significantly improves overall system bounds, cutting the maximum latency for a standard DomU by more than half (from $443\ \mu\text{s}$ to $179\ \mu\text{s}$) and pushing an RT DomU to a recorded bound of $163\ \mu\text{s}$.
- **Persistent Virtualization Overhead:** Despite the dramatic improvements in maximum latency achieved through kernel patches and core pinning, the average latency remains rigidly fixed at approximately $32\text{--}33\ \mu\text{s}$ across every tested configuration. This phenomenon indicates that the baseline jitter introduced by the Xen hypervisor layer is a structural constant that cannot be bypassed by domain-level scheduling optimizations alone.

In this section, we will analyze the detailed results of these specific configurations to quantify the determinism and virtualization overhead achievable in a statically pinned Xen architecture.

1) *NON-REAL-TIME DOM0 AND NON-REAL-TIME DOMU (NULL SCHEDULER)*: This section analyzes the results obtained from a single, prolonged 5-minute execution of the `cyclictest` tool in a Xen virtualized environment utilizing the static pinning configuration acting as a Null scheduler. The configuration features a standard (Non-Real-Time) Linux Dom0 and a Non-Real-Time DomU. The objective of this analysis is to evaluate the latency and virtualization overhead under stress conditions when both domains lack real-time optimizations.

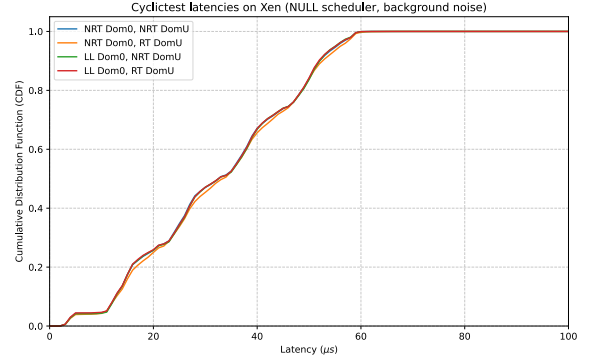


Fig. 18. Performance under Stress - Static Pinning (Default QEMU Priority)

a) *Nominal Performance and Average Latency:* The data collected reveals a noticeable baseline overhead introduced by the virtualization layer. The average latency remained stable at $32\ \mu\text{s}$ throughout the test duration. While the absolute minimum latency recorded was $3\ \mu\text{s}$, the results demonstrate a variance in nominal execution times.

b) *Worst-Case Execution Time (WCET) Analysis:* The analysis of the Worst-Case Execution Time (WCET) indicates that static vCPU pinning provides a more bounded execution environment compared to dynamic scheduling. Over the 5-minute continuous test, the absolute maximum latency recorded was bounded at $443\ \mu\text{s}$. Although this value represents a peak due to the `stressdom0` workload on an unoptimized kernel, the system manages to avoid the catastrophic, multi-millisecond preemption spikes often seen in entirely unpinned, dynamic environments.

c) *Observations on the Xen Pinned Environment:* The empirical observation of this sustained test provides insights into the behavior of the Xen hypervisor with static pinning:

* **Bounded but Elevated Latency:** The system bounded the WCET to $443\ \mu\text{s}$, indicating that while pinning restricts vCPU migration, the unoptimized Dom0 under stress still incurs significant delays. * **Average Jitter:** The average latency of $32\ \mu\text{s}$ highlights the inherent jitter introduced by the hypervisor layer itself. * **Suitability:** This configuration limits unbounded latency starvation, but the $443\ \mu\text{s}$ peak suggests it is insufficient for strict real-time constraints under heavy workloads.

2) *NON-REAL-TIME DOM0 AND REAL-TIME DOMU (NULL SCHEDULER)*: This section analyzes the results obtained from a single, prolonged 5-minute execution of the `cyclictest` tool utilizing static vCPU pinning. The configuration features a standard (Non-Real-Time) Linux Dom0 and a `PREEMPT_RT` patched DomU. The objective is to observe how effectively the DomU’s internal scheduling can manage execution times when provided with a dedicated physical core, despite the control domain lacking real-time optimizations and operating under stress.

a) *Nominal Performance and Average Latency:* The data collected reveals that the baseline virtualization overhead heavily influences average execution times. The average

latency recorded was $33 \mu s$. The absolute minimum latency achieved was very low at just $3 \mu s$.

b) Worst-Case Execution Time (WCET) Analysis: Despite the average jitter introduced by the hypervisor, the analysis of the Worst-Case Execution Time (WCET) demonstrates excellent stability at the upper bounds. Over the entire 5-minute sustained test, the absolute maximum latency was strictly capped at $76 \mu s$. This indicates that the combination of DomU optimizations and static pinning successfully shielded the critical sections from the stress workload affecting Dom0.

c) Observations on the Hybrid Xen Environment: The empirical observation of this sustained test provides insights into the behaviour of the RT DomU with a Non-RT Dom0 and static pinning: * **Bounded WCET:** By tightly capping the absolute maximum latency at $76 \mu s$, the PREEMPT_RT DomU kernel showed high effectiveness in establishing a highly predictable upper bound. * **Hypervisor-Induced Jitter:** The average latency of $33 \mu s$ suggests that the inherent virtualization layer jitter cannot be entirely removed by DomU-side optimizations alone. * **Internal Determinism:** In this hybrid configuration, the DomU-level real-time optimizations successfully maintained strict temporal constraints, heavily benefiting from the dedicated physical CPU assignment.

3) LOW LATENCY DOM0 AND NON-REAL-TIME DOMU (NULL SCHEDULER): This section analyzes the results obtained from a single, prolonged 5-minute execution of the `cyclicttest` tool utilizing static vCPU pinning. For this specific test, the Dom0 operating system was configured with a Low Latency kernel while the DomU maintained a Non-Real-Time (NRT) kernel, evaluating the effects of a modified Dom0 on DomU latencies under stress.

a) Nominal Performance and Average Latency: The data collected confirms the consistent baseline behavior of the Xen infrastructure. The average latency remained at $32 \mu s$ throughout the entire 5-minute test. The absolute minimum latency achieved was $3 \mu s$.

b) Worst-Case Execution Time (WCET) Analysis: The analysis of the Worst-Case Execution Time (WCET) suggests tangible benefits from this configuration. Over the sustained execution, the absolute maximum latency was bounded at $179 \mu s$. This result indicates that the low-latency optimizations within the Dom0, paired with static pinning, helped to significantly mitigate severe preemption spikes compared to a standard Dom0 kernel (reducing the peak from $443 \mu s$ to $179 \mu s$).

c) Observations on the Low Latency Xen Environment: The empirical observation of this sustained test provides insights into the capabilities of a Low Latency Dom0 running with a NRT DomU and static pinning: * **Predictable Upper Bounds:** The Low Latency kernel tuning bounded the WCET to $179 \mu s$ during the test. * **Persistent Hypervisor Jitter:** The average latency of $32 \mu s$ confirms that the baseline virtualization overhead dictates the nominal jitter. * **Suitability:** This configuration presents a solid improvement in maximum latency over the strictly NRT environment, offering enhanced predictability without

requiring a fully patched RT DomU.

4) LOW LATENCY DOM0 AND REAL-TIME DOMU (NULL SCHEDULER): This section analyzes the results obtained from a single, prolonged 5-minute execution of the `cyclicttest` tool in a Xen virtualized environment utilizing static vCPU pinning. This specific configuration features a Low Latency kernel Dom0 and a Real-Time (PREEMPT_RT) DomU, evaluated under stress conditions.

a) Nominal Performance and Average Latency: The data collected reveals that the baseline virtualization overhead continues to define the average execution times. The average latency recorded was $32 \mu s$. The absolute minimum latency was recorded at $3 \mu s$.

b) Worst-Case Execution Time (WCET) Analysis: The analysis of the Worst-Case Execution Time (WCET) demonstrates good stability at the upper limits. Over the entire 5-minute sustained test, the absolute maximum latency was capped at $163 \mu s$. While this represents a notable increase from the unstressed baseline of $65 \mu s$, the system still avoids the catastrophic, multi-millisecond preemption spikes observed in unoptimized configurations.

c) Observations on the Low Latency RT Hybrid Xen Environment: The empirical observation of this sustained test provides insights into the behaviour of a RT DomU with a Low Latency Dom0 and static pinning: * **Bounded WCET:** By capping the absolute maximum latency at $163 \mu s$, the DomU kernel maintained a bounded execution environment, though the stress workload on Dom0 caused a measurable increase from the $65 \mu s$ baseline. * **Hypervisor-Induced Jitter:** The consistent average latency of $32 \mu s$ confirms that the underlying virtualization architecture introduces inherent, unavoidable jitter, irrespective of kernel patches. * **Effective Internal Determinism:** The DomU-level optimizations, combined with the Low Latency Dom0, successfully contain the worst-case latency well below the levels observed in non-RT configurations, demonstrating the value of guest-level real-time optimizations even under host-level stress.

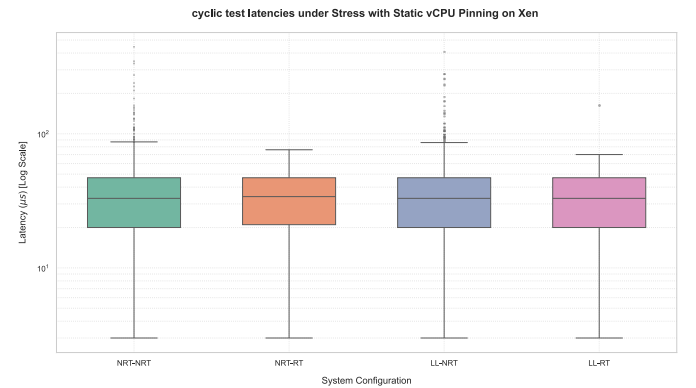


Fig. 19. Performance under Stress - Static Pinning (Default QEMU Priority)

E. SUMMARY OF RESULTS

Following the detailed analysis of each individual scenario, the table below provides a consolidated overview of the

Worst-Case Execution Time (WCET) measurements. It allows for a direct comparison across all four Dom0-DomU kernel configurations (**NRT-NRT**, **NRT-RT**, **LL-NRT**, and **LL-RT**) under the four tested scheduling and load conditions: standard dynamic execution (**BASELINE**), execution under heavy system load within Dom0 (**STRESS WORKLOAD**), execution with static core isolation (**vCPU PINNING BASELINE**), and isolated execution under heavy load (**vCPU PINNING STRESS WORKLOAD**).

Furthermore, alongside the consolidated WCET table, this section presents a detailed breakdown of the percentage increments for both the maximum latency (WCET) and the average latency (expressed as Mean $\pm$ Standard Deviation). To comprehensively evaluate the effectiveness of the hardware isolation mechanisms, this quantitative analysis of the performance degradation is divided into two distinct sets: the first evaluates the impact of the Dom0 stress workload under standard dynamic scheduling, while the second analyzes the impact of the same stress workload when strict static vCPU pinning is applied to isolate the domains.

- 1) **NRT NRT: PERCENTAGE INCREMENTS (Stress vs. Baseline):** * Average Increment: +8.06% * WCET Increment: -35.23%
- 2) **NRT RT: PERCENTAGE INCREMENTS (Stress vs. Baseline):** * Average Increment: +8.01% * WCET Increment: +139.19%
- 3) **LL NRT: PERCENTAGE INCREMENTS (Stress vs. Baseline):** * Average Increment: +8.79% * WCET Increment: +246.05%
- 4) **LL RT: PERCENTAGE INCREMENTS (Stress vs. Baseline):** * Average Increment: +8.42% * WCET Increment: +194.37%
- 5) **NRT NRT: PERCENTAGE INCREMENTS (Stress vs. Baseline):** * Average Increment: +0.34% * WCET Increment: +223.36%
- 6) **NRT RT: PERCENTAGE INCREMENTS (Stress vs. Baseline):** * Average Increment: +1.96% * WCET Increment: +13.43%
- 7) **LL NRT: PERCENTAGE INCREMENTS (Stress vs. Baseline):** * Average Increment: +1.05% * WCET Increment: +129.05%
- 8) **LL RT: PERCENTAGE INCREMENTS (Stress vs. Baseline):** * Average Increment: -1.11% * WCET Increment: +150.77%

F. ANALYSIS OF DEVICE MODEL PRIORITY INVERSION IN MODERN XEN

In earlier research evaluating the real-time capabilities of hypervisors, a notable priority inversion issue was identified within the Xen architecture. The problem stems from the architectural dependency of Hardware Virtual Machine (HVM) guests on the Device Model. In Xen, when creating an HVM guest that requires a Device Model, this model is typically an instance of QEMU that executes as a standard

process inside Domain 0 (Dom0). Because Dom0 is scheduled alongside other virtual machines by the hypervisor, a low-privilege QEMU process on Dom0 could be preempted when Dom0 is placed under heavy computational stress. Consequently, a Real-Time (RT) DomU waiting for the QEMU Device Model could suffer from unbounded latency, compromising its real-time execution guarantees.

To investigate whether this architectural bottleneck persists in modern versions of Xen, a series of experiments were conducted. The objective is to determine if elevating the QEMU process to a maximum Real-Time priority (FIFO scheduler with priority 99) mitigates preemption and improves the latency bounds of the DomU compared to the default Xen configuration.

To reproduce and analyze the aforementioned priority inversion on a modern Xen version, the environment was configured with strict resource partitioning.

- **CPU Pinning:** Dom0 was pinned to the first 22 physical CPUs (pCPUs), while the Hardware Virtual Machine (HVM) DomU was pinned to the last 2 pCPUs.
- **Host Stress:** To simulate heavy load and induce potential starvation in Dom0, stress-ng was executed with the following parameters:

```
stress-ng --cpu 22 --vm 12 --vm-bytes 2G
↪ --timeout 10m
```
- **Kernel Configurations:** Tests were run across the usual four permutations of Dom0 and DomU kernels.
- **QEMU Priority Mitigation:** For each kernel combination, a baseline test (default QEMU priority) was compared against a mitigated test (**maxprioqemu**), wherein the QEMU Device Model process in Dom0 was explicitly set to the **SCHED_FIFO** policy with a priority of 99.

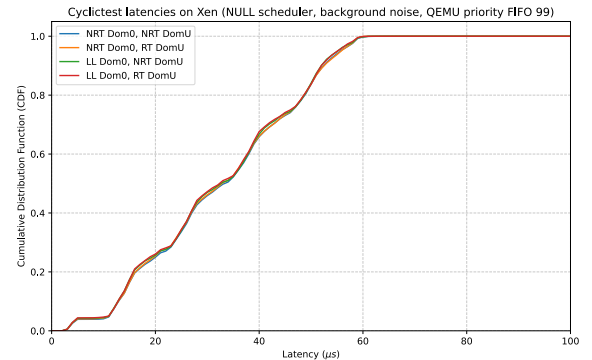


Fig. 20. Performance under Stress - Static Pinning (Max QEMU Priority)

1) **Empirical Results:** A thorough data analysis of the provided cyclicttest histograms reveals the following key findings:

- **Absence of Priority Inversion Spikes:** In older versions of Xen suffering from the QEMU starvation

TABLE XXIII
XEN DOM0 WCET SUMMARY

Configuration	NRT-NRT	NRT-RT	LL-NRT	LL-RT
BASELINE	369	74	152	71
STRESS WORK-LOAD	239	177	526	209
vCPU PINNING BASELINE	137	67	179	65
vCPU PINNING STRESS WORK-LOAD	443	76	410	163

TABLE XXIV
NRT-NRT LATENCY STATISTICS (XEN)

METRIC	BASELINE	STRESS WORKLOAD
Average $\pm$ SD	$32.87 \pm 15.17 \mu s$	$35.52 \pm 15.50 \mu s$
WCET (Max)	$369 \mu s$	$239 \mu s$

TABLE XXV
NRT-RT LATENCY STATISTICS (XEN)

METRIC	BASELINE	STRESS WORKLOAD
Average $\pm$ SD	$33.20 \pm 15.12 \mu s$	$35.86 \pm 15.54 \mu s$
WCET (Max)	$74 \mu s$	$177 \mu s$

TABLE XXVI
LL-NRT LATENCY STATISTICS (XEN)

METRIC	BASELINE	STRESS WORKLOAD
Average $\pm$ SD	$32.71 \pm 15.17 \mu s$	$35.58 \pm 15.55 \mu s$
WCET (Max)	$152 \mu s$	$526 \mu s$

TABLE XXVII
LL-RT LATENCY STATISTICS (XEN)

METRIC	BASELINE	STRESS WORKLOAD
Average $\pm$ SD	$32.89 \pm 15.14 \mu s$	$35.66 \pm 15.60 \mu s$
WCET (Max)	$71 \mu s$	$209 \mu s$

TABLE XXVIII
NRT-NRT LATENCY STATISTICS (XEN) - vCPU PINNING

METRIC	vCPU PINNING BASELINE	vCPU PINNING STRESS WORKLOAD
Average $\pm$ SD	$32.65 \pm 15.17 \mu s$	$32.76 \pm 15.10 \mu s$
WCET (Max)	$137 \mu s$	$443 \mu s$

TABLE XXIX
NRT-RT LATENCY STATISTICS (XEN) - vCPU PINNING

METRIC	vCPU PINNING BASELINE	vCPU PINNING STRESS WORKLOAD
Average $\pm$ SD	$32.59 \pm 15.16 \mu s$	$33.23 \pm 15.09 \mu s$
WCET (Max)	$67 \mu s$	$76 \mu s$

TABLE XXX
LL-NRT LATENCY STATISTICS (XEN) - vCPU PINNING

METRIC	vCPU PINNING BASELINE	vCPU PINNING STRESS WORKLOAD
Average $\pm$ SD	$32.49 \pm 15.19 \mu s$	$32.83 \pm 15.06 \mu s$
WCET (Max)	$179 \mu s$	$410 \mu s$

TABLE XXXI
LL-RT LATENCY STATISTICS (XEN) - vCPU PINNING

METRIC	vCPU PINNING BASELINE	vCPU PINNING STRESS WORKLOAD
Average $\pm$ SD	$33.09 \pm 15.18 \mu s$	$32.73 \pm 15.09 \mu s$
WCET (Max)	$65 \mu s$	$163 \mu s$

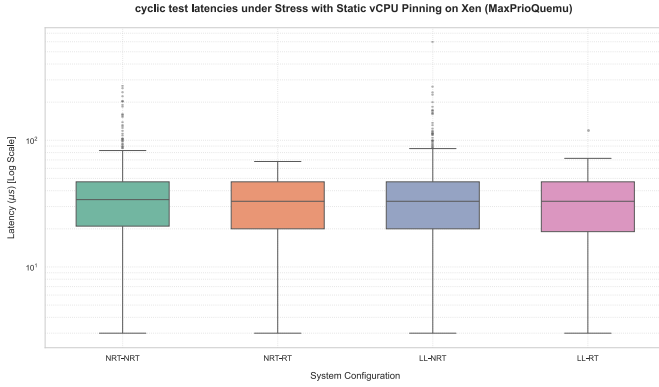


Fig. 21. Performance under Stress - Static Pinning (Max QEMU Priority)

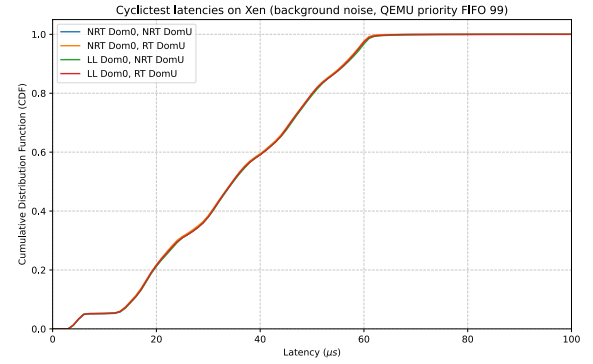


Fig. 22. Performance under Stress - Credit2 Scheduler (Max QEMU Priority)

issue, the expected symptom would be a pronounced “heavy tail” in the histogram, indicating extreme, unbounded latencies where the DomU was blocked waiting for Dom0. The empirical data across all results `_null_hvm_pinned_*` logs demonstrates no such extreme outliers in the default priority configurations.

- **Latency Distribution Parity:** The latency distributions between the default configurations and their mitigated counterparts are virtually identical.
- **Consistent Upper Bounds:** The maximum recorded latencies (Worst-Case Execution Time) in the standard configurations are directly comparable to those in the maximum priority configurations. Across both the Low-Latency (LL) and Non-Real-Time (NRT) Dom0 environments, elevating QEMU’s priority did not tighten the worst-case temporal bounds.

Based on the experimental data, the priority inversion problem previously documented in Section 6.2 of the Abeni and Faggioli research seems to be non-existent in this modern Xen deployment. Changing the Device Model priority yields no beneficial effect for the latency bounds of real-time tasks inside the DomU.

With further experiments, we confirmed that changing the priority of the QEMU process does not improve latency even when using the Credit2 scheduler.

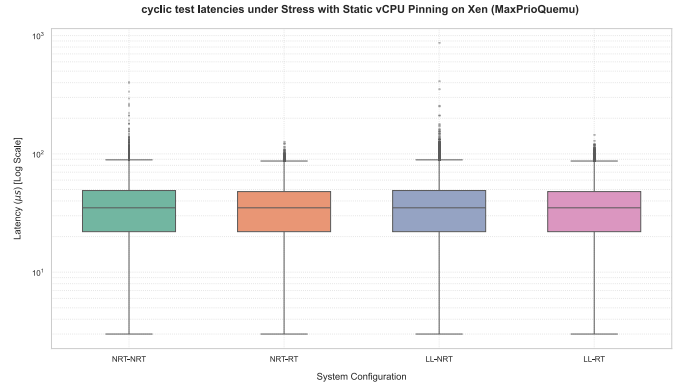


Fig. 23. Performance under Stress - Credit2 Scheduler (Max QEMU Priority)

This behavior indicates that modern Xen HVM implementations successfully decouple essential local timer and interrupt deliveries from the QEMU Device Model. Because CPU-bound real-time workloads (like `cyclictst`) primarily exercise timer wakeups rather than complex I/O, the DomU can accurately maintain its temporal constraints utilizing hardware virtualization extensions alone. Therefore, manually elevating the priority of the Dom0 QEMU process is unnecessary for maintaining real-time determinism in contemporary Xen environments.

G. IMPACT OF DIFFERENT STRESSORS ON LATENCIES

Following the previous tests, it was decided to conduct a new case study: the latter aims to measure and compare the real-time latency perceived by an application workload running inside HVM virtual machines (DomU) on a Xen hypervisor, as four experimental factors vary:

- **Dom0 Kernel:** standard (“NRT”) vs. low-latency/PREEMPT_RT kernel (“LL”);
- **vCPU Pinning:** guests with vCPUs pinned to dedicated physical cores vs. unpinned guests (free scheduling by Xen);
- **Guest Type:** guest with standard Linux kernel (“nrt”) vs. guest with RT kernel (“rt5”);
- **Concurrent stress workload on Dom0:** no additional stressor besides CPU+memory (“baseline”), cache stress (“cache”), interrupts stress (“interrupts”), raw sockets stress (“rawsock”).

The combination of these four factors generates 32 test scenarios.

The tests were executed via a bash orchestration script that, for each Dom0 kernel, sequentially boots (via grub-reboot) and subsequently instantiates the relevant guests. For each guest $\times$ stressor combination, the script:

- starts the “always-on” stressors on Dom0 (**stress-ng** `--cpu 22`, **stress-ng** `--vm 12 --vm-bytes 2G`), kept active for the entire duration of the guest’s test block;
- starts the specific sequential stressor of the scenario (none for baseline; `--cache 0`; `--interrupts`; `--rawsock 22`);
- creates the DomU (**x1 create**), waits 30 s for settling;
- runs **cyclictest** inside the DomU (priority 99, interval 50 μ s, duration 5 minutes, histogram with a 1000 μ s threshold) routing the output exclusively to a file to avoid introducing artifacts due to traffic on the serial console;
- destroys the DomU, mounts its disk on Dom0 to retrieve the histogram file, terminates all stressors, and clears the Dom0 caches before the next scenario.

Each kernel $\times$ guest $\times$ stressor combination was executed in a single run (no repetitions).

a) *Metrics extracted from each log:*

- **Avg:** average scheduling latency (μ s) over the entire run;
- **Max:** maximum observed latency (μ s);
- **Overflow:** number of samples exceeding the histogram threshold (1000 μ s) — a direct indicator of severe latency events, not captured in the main distribution;
- **Overflow cycles:** the cycle indices in which the overflows occurred, useful to understand whether the events are concentrated in a phase of the test (e.g., startup or shutdown) or evenly distributed.

1) *Results:* The following table reports the extracted metrics for each of the 32 combinations.

a) *Summary by pinning effect:* Aggregating all runs (regardless of kernel, guest, and stressor) by the “pinning” variable alone, the effect is clear:

vCPU pinning reduces both the mean latency (by about 3-4 μ s) and, above all, the frequency and magnitude of overflow events. Almost all the overflows observed in the study come from unpinned runs.

2) *1- vCPU pinning is the dominant factor:* Across all tested combinations, the configurations with active pinning show an almost deterministic behavior: maximum latency almost always below 550 μ s and zero overflows, with only two isolated exceptions. This result is independent of the Dom0 kernel (LL or NRT) and the guest type: the benefit therefore derives primarily from the isolation of scheduling resources, rather than from the low-latency kernel itself.

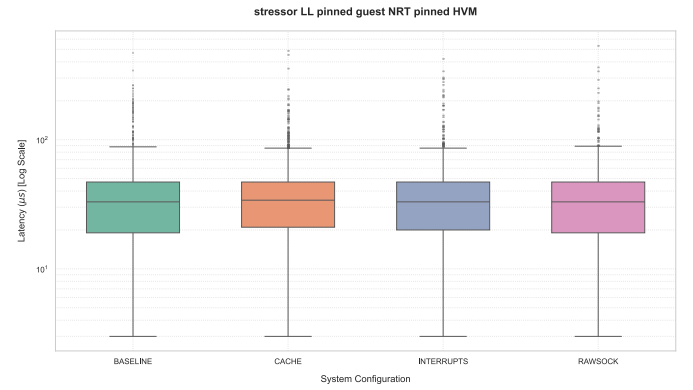


Fig. 24. stressor on Xen

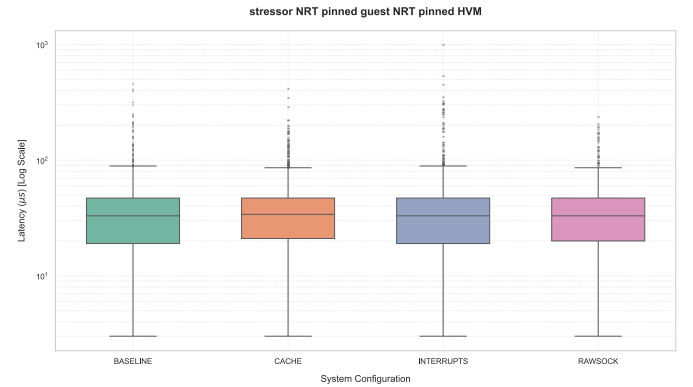


Fig. 25. stressor on Xen

3) *2- The unpinned “nrt-hvm” guest is the most fragile configuration in the dataset:* Regardless of the Dom0 kernel used (LL or NRT), the unpinned nrt-hvm guest shows maximum latencies in the order of tens of milliseconds under the interrupts and rawsock stressors (up to ~62 ms), with 11-18 overflow events. The absolute worst case in the whole study is NRT + nrt-hvm + cache, with 88 overflows out of a total of ~4.78 million cycles and a maximum latency of 61.6 ms.

TABLE XXXII
XEN COLLECTED METRICS

Kernel	Pinning	Guest	Stressor	Avg (μs)	Max (μs)	Overflow	Note
LL	No	nrt-hvm	baseline	36	57.384	17	
LL	No	nrt-hvm	cache	36	2774	1	
LL	No	nrt-hvm	interrupts	36	59.998	18	
LL	No	nrt-hvm	rawsock	35	52.363	13	
LL	No	rt5-hvm	baseline	36	192	0	
LL	No	rt5-hvm	cache	36	2772	19	Isolated anomaly
LL	No	rt5-hvm	interrupts	35	144	0	
LL	No	rt5-hvm	rawsock	35	162	0	
LL	Yes	nrt-pinned	baseline	32	470	0	
LL	Yes	nrt-pinned	cache	34	35.984	1	Single Overflow, start run
LL	Yes	nrt-pinned	interrupts	32	423	0	
LL	Yes	nrt-pinned	rawsock	32	536	0	
LL	Yes	rt5-pinned	baseline	34	79	0	
LL	Yes	rt5-pinned	cache	33	146	0	
LL	Yes	rt5-pinned	interrupts	32	81	0	
LL	Yes	rt5-pinned	rawsock	32	77	0	
NRT	No	nrt-hvm	baseline	35	392	0	
NRT	No	nrt-hvm	cache	36	61.611	88	Worst case
NRT	No	nrt-hvm	interrupts	35	61.826	12	
NRT	No	nrt-hvm	rawsock	35	58.344	11	
NRT	No	rt5-hvm	baseline	35	175	0	
NRT	No	rt5-hvm	cache	36	244	0	
NRT	No	rt5-hvm	interrupts	35	189	0	
NRT	No	rt5-hvm	rawsock	35	143	0	
NRT	Yes	nrt-pinned	baseline	32	450	0	
NRT	Yes	nrt-pinned	cache	33	414	0	
NRT	Yes	nrt-pinned	interrupts	32	4225	47	Cluster overflow at end of run
NRT	Yes	nrt-pinned	rawsock	32	236	0	
NRT	Yes	rt5-pinned	baseline	32	72	0	
NRT	Yes	rt5-pinned	cache	34	99	0	
NRT	Yes	rt5-pinned	interrupts	33	80	0	
NRT	Yes	rt5-pinned	rawsock	33	357	0	

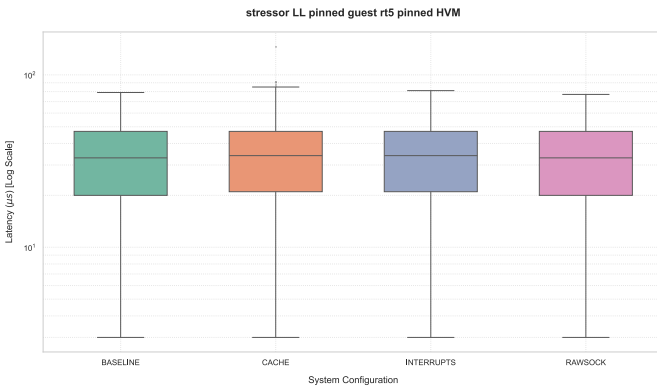


Fig. 26. stressor on Xen

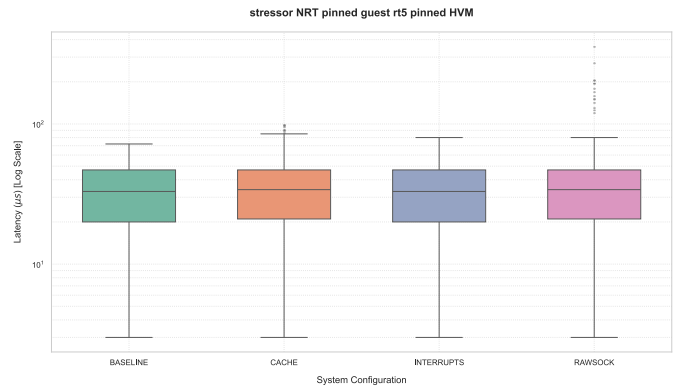


Fig. 27. stressor on Xen

The fact that the problem occurs with both Dom0 kernels indicates that the cause is not primarily related to the low-latency kernel, but more likely to the configuration

itself of the nrt-hvm guest domain (e.g., lack of vCPU isolation/affinity, scheduling weight in Xen).

TABLE XXXIII
XEN PINNING EFFECT SUMMARY

Configuration	Avg mean latency (μs)	Max worst latency (μs)	Total overflows	Runs with overflow >0
Pinned guests (LL+NRT)	32.6	35.984	48	2 / 15
Unpinned guests (LL+NRT)	35.4	61.826	179	8 / 16

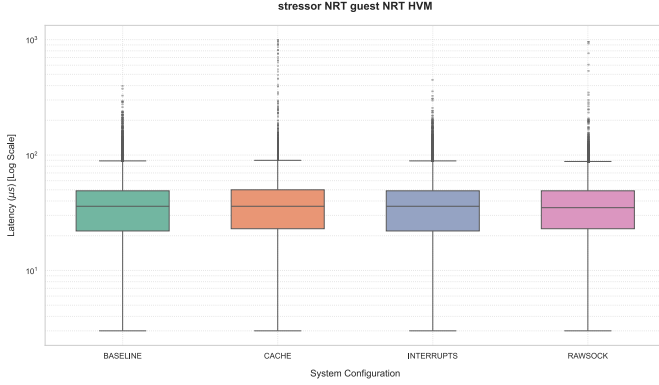


Fig. 28. stressor NRT guest NRT HVM on Xen

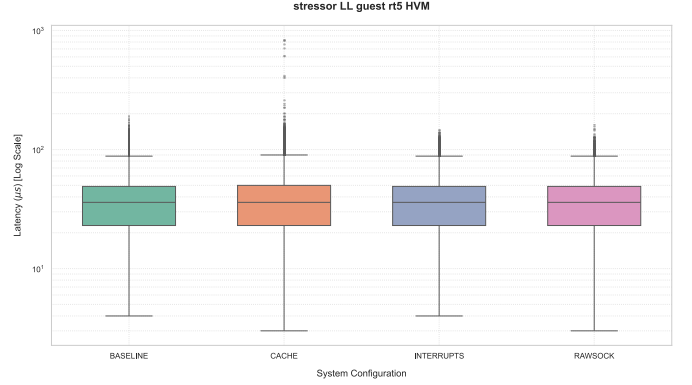


Fig. 30. stressor NRT guest NRT HVM on Xen

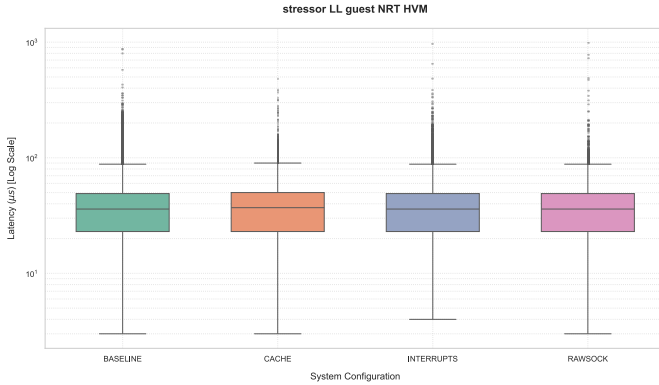


Fig. 29. stressor NRT guest NRT HVM on Xen

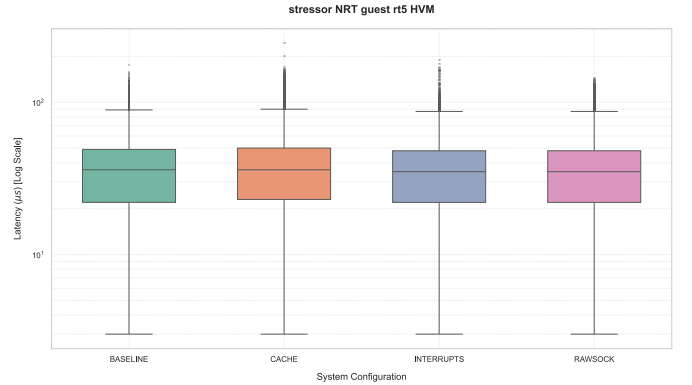


Fig. 31. stressor NRT guest NRT HVM on Xen

4) 3- The “rt5” guest is systematically more stable than the “nrt” guest, whether pinned or not: Given the same stressor and kernel, the rt5-hvm guest almost always shows maximum latencies in the order of hundreds of microseconds and zero overflows, even without pinning — a significantly better behavior compared to its nrt-hvm twin under the same conditions. The only notable exception is described in the following point.

5) 4- Isolated anomalies, not reproducible between kernels: The study detected three anomalous events that deviate from the otherwise clean behavior of their respective categories:

- **LL, rt5-hvm guest, cache stressor:** 19 overflows and max 2.77 ms, while the same guest under all other stressors (including the same cache stressor but with the NRT kernel) remains clean (0 overflows). It seems

to be a specific interaction between the LL kernel and cache stress when the rt5 guest is active.

- **LL-pinned, nrt-pinned guest, cache stressor:** single isolated overflow at 35.98 ms, which occurred very early in the run (cycle 913 out of a total of ~4.93 million). The rest of the run is clean (secondary max ~536 μs in the other pinned conditions).
- **NRT-pinned, nrt-pinned guest, interrupts stressor:** 47 overflows, but all concentrated in a dense cluster right in the last ~3% of the run’s cycles (close to the end of the 5 minutes), not distributed throughout the test.

The fact that these three events do not have a counterpart in the same guest $\times$ stressor combination tested with the other kernel suggests that they are events tied to a single run (experimental noise, test startup/shutdown

phase, transient host interference) rather than a systemic and reproducible behavior of the low-latency kernel. It is not possible, with the current data, to distinguish with certainty between the two hypotheses (see point 6).

6) 5- *The “cache” stressor is the one with the most unpredictable impact:* The cache stressor produces the most extreme result of the entire study (88 overflows with NRT kernel) but, in the same guest/kernel combination with active pinning, it turns out to be harmless instead (0 overflows). It is also the only stressor involved in all three isolated anomalies. Compared to interrupts and rawsock, which show a more uniform and predictable impact on the unpinned nrt-hvm guest, cache appears to be the stressor with the highest behavioral variance among the tested conditions.

7) 6- *Kernel LL vs NRT difference: no clear winner on unpinned guests:* Comparing LL and NRT on the same unpinned guest/stressor combination, no kernel emerges as systematically better than the other. For example, on the nrt-hvm guest with cache stressor, LL reports 1 overflow (max 2.77 ms) against the 88 overflows of NRT (max 61.6 ms) — a result clearly in favor of LL. But on the rt5-hvm guest with the same cache stressor, NRT is clean (0 overflows) while LL reports 19 overflows. This crossed pattern indicates that the kernel $\times$ guest $\times$ stressor interaction is more complex than a simple “the low-latency kernel always reduces latency”, and that the benefit of the PREEMPT_RT kernel in this setup is, given the current state of the data, less decisive than vCPU pinning.

8) *Final considerations:*

- The factor that influences latency quality more than any other is **vCPU pinning**: almost zero overflows on all pinned configurations, regardless of kernel and stressor.
- The **rt5-hvm** guest performs better than the **nrt-hvm** guest in every tested condition, pinning aside.
- The **unpinned nrt-hvm** guest is unreliable under load (except in the baseline scenario), with peaks up to ~62 ms regardless of the Dom0 kernel.
- The **low-latency (LL/PREEMPT_RT)** kernel does not guarantee, in this dataset, a systematic improvement compared to the standard kernel (NRT): direct comparison shows crossed results depending on the guest/stressor combination.
- The **cache** stressor is the one that generates the greatest variability in results, including the absolute worst case of the study and all three isolated anomalies detected.

H. IMPACT OF A NOISY DOMU

The study aims to evaluate the impact of a disturbance workload (“noisy neighbor”) generated by a dedicated DomU on a second DomU subjected to real-time latency measurement (cyclicttest), comparing four configurations of the Dom0 (standard kernel vs. low-latency rt5-ll kernel, with and without vCPU pinning) and, for each, two types of DomUs (non-RT kernel and RT rt5 kernel).

- **Dom0:** Xen 4.17-amd64 Type 1 hypervisor, 4 dedicated vCPUs, tested with two Linux kernels: 6.18.35 (standard, “NRT”) and 6.18.35-rt5-ll (low-latency, “LL”).
- For each Dom0 kernel, the tests were repeated with and without static pinning of the DomU vCPUs, for a total of 4 Dom0 configurations:
 - NRT-4vcpu
 - NRT-4vcpu-pinned
 - LL-4vcpu
 - LL-4vcpu-pinned
- **“Noisy” DomU:** a dedicated VM (ubuntu-24.04-linux-6.18.35-noisyguest-big), started before each test cycle and kept active for the entire duration of the tests on the current configuration. It generates load with two independent stress-ng processes:
 - 16 CPU workers (`--cpu 16`)
 - 16 memory workers of 1 GB each (`--vm 16 --vm-bytes 1G`)
 - both without timeout (continuous load).
- **“Regular” DomU** (under measurement): two variants for each Dom0 configuration:
 - DomU with non-RT kernel (guest-nrt-hvm)
 - DomU with RT rt5 kernel (guest-rt5-hvm)
 - in the pinned versions when applicable.
- **Measurement:** cyclicttest executed inside the regular DomU with `--mlockall`, priority 99, 1 thread, affinity 1, interval 50 μ s, duration 5 minutes, histogram up to 1000 μ s.
- Each kernel-DomU combination was executed only once (no statistical repetition) .

The entire flow is orchestrated by a bash script (`cyclicttest_noisy_domu.sh`) which, for each of the 4 kernel configurations:

- reboots the Dom0 onto the target kernel via grub-reboot,
- starts the noisy DomU and launches stress-ng on it,
- then sequentially starts the two regular DomUs,
- executes cyclicttest on each,
- retrieves their histogram and destroys the VM before moving to the next.

At the end of the two DomUs, the noisy DomU is destroyed and the process moves to the next kernel configuration. Before each run, the Dom0 caches are cleared (`drop_caches`) to reduce residual caching effects between tests.

The result of the campaign is a set of 8 histogram files (4 kernel configurations $\times$ 2 DomU types), each with the complete latency distribution, min/avg/max, overflow count, and cycles in which they occurred.

I. Collected data

The following table summarizes the values extracted from the footer of each histogram. Highlighted (bold) is the

TABLE XXXIV
XEN SUMMARY DATA

Dom0 kernel config.	Pinning	DomU / DomU kernel	Avg (μs)	Max (μs)	Overflow	Samples
NRT-4vcpu	No	guest-nrt-hvm (non-RT)	33	62566	4	5,003,768
NRT-4vcpu	No	guest-rt5-hvm (RT)	36	312	0	4,807,301
NRT-4vcpu	Yes	guest-nrt-pinned-hvm (non-RT)	33	336	0	5,041,620
NRT-4vcpu	Yes	guest-rt5-pinned-hvm (RT)	33	87	0	5,061,753
LL-4vcpu (rt5-ll)	No	guest-nrt-hvm (non-RT)	32	167	0	5,085,072
LL-4vcpu (rt5-ll)	No	guest-rt5-hvm (RT)	32	74	0	5,098,348
LL-4vcpu (rt5-ll)	Yes	guest-nrt-pinned-hvm (non-RT)	33	502	0	5,052,370
LL-4vcpu (rt5-ll)	Yes	guest-rt5-pinned-hvm (RT)	33	123	0	5,035,559

only test with overflows and maximum values out of scale compared to all the others.

In all 8 tests, the minimum recorded latency is 3 μs , a stable and consistent value regardless of kernel, pinning, or DomU — indicating that the “idle” path (no interference) is not influenced by the noisy neighbor, unlike the tail of the distribution.

1) 1- Only one critical case out of eight: The combination **NRT-4vcpu (Dom0 without RT patch) + no pinning + non-RT DomU** is the only one to present a truly critical behavior: maximum latency of 62,566 μs (~62.5 ms) and 4 histogram overflows, recorded at cycles 8,323, 20,698, 24,840, and 37,984. All other 7 configurations remain under a millisecond of maximum latency and at zero overflows. It is also the only test with a significantly lower number of samples than the others (4.81M against the expected 5.0-5.1M in 5 minutes at a 50 μs interval), consistent with lost/delayed cycles during spikes.

2) 2- *vCPU pinning effect*: Pinning has a very marked impact only in the absence of an RT kernel on Dom0: on the NRT-4vcpu configuration, it completely eliminates the catastrophic outlier (max 62,566 $\rightarrow$ 336 μs on the non-RT DomU; 312 $\rightarrow$ 87 μs on the RT DomU) and zeros out the overflows. On the LL kernel, instead, the effect is opposite and more contained: pinning slightly increases the maximums compared to the same test without pinning (non-RT DomU: 167 $\rightarrow$ 502 μs ; RT DomU: 74 $\rightarrow$ 123 μs), while remaining orders of magnitude below the critical threshold observed on NRT.

3) 3- *DomU kernel effect (RT rt5 vs non-RT)*: With the same Dom0 configuration, the DomU with the RT rt5 kernel almost always shows a shorter latency tail compared to its non-RT twin:

- NRT-4vcpu (62,566 $\rightarrow$ 312 μs)
- NRT-4vcpu-pinned (336 $\rightarrow$ 87 μs)
- LL-4vcpu (167 $\rightarrow$ 74 μs)

Even in LL-4vcpu-pinned, the RT DomU remains better (502 $\rightarrow$ 123 μs), although both values are higher than the unpinned variant of the same Dom0 kernel.

4) 4- *RT kernel on Dom0 and RT kernel on DomU: overlapping benefits*: The two mechanisms (low-latency kernel on Dom0, RT kernel on DomU) act in the same direction and partially overlap: any LL-4vcpu configuration, with or without pinning, always stays within a few hundred μs of maximum, regardless of the DomU. It is the total absence of both protections — no RT kernel on either the Dom0 or the DomU, no pinning — that produces the only truly out-of-scale result of the campaign.

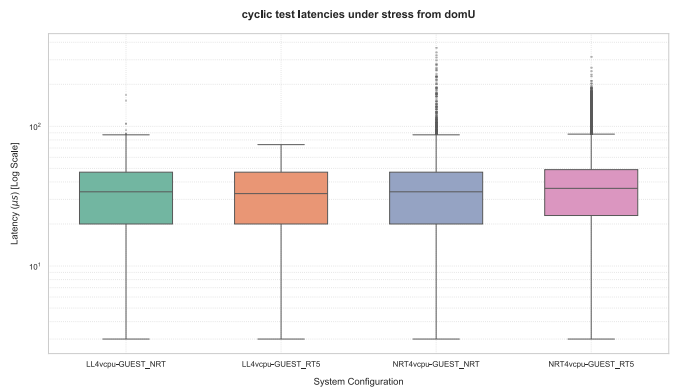


Fig. 32. 4vcpu HVM on Xen

J. Conclusions

- The concrete risk introduced by the noisy neighbor (CPU+memory stress-ng) manifests severely only in the configuration devoid of any mitigation (non-RT Dom0, no pinning, non-RT DomU): all other tested combinations keep the maximum latency under a millisecond.
- **vCPU pinning** is the most effective and economical countermeasure when an RT-patched kernel is not

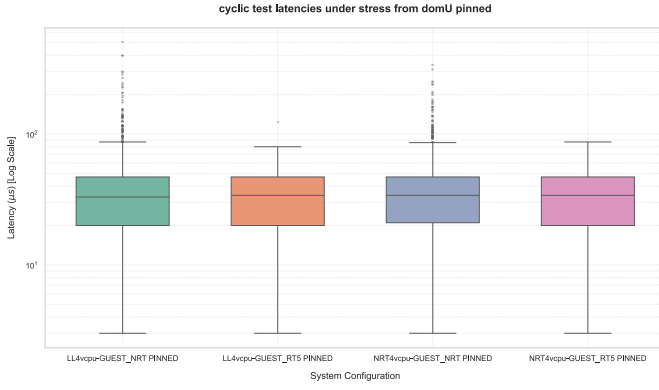


Fig. 33. 4vcpu pinned HVM on Xen

available on Dom0: on its own, it brings the maximum latency down from 62.5 ms to a few hundred μs .

- A **low-latency kernel on Dom0 (rt5-ll)** inherently guarantees stable behavior regardless of pinning, suggesting that, having such a kernel, static pinning is not strictly necessary merely for robustness against the noisy neighbor.
- An **RT kernel on the DomU** systematically reduces the latency tail compared to the non-RT kernel, under the same Dom0 conditions, confirming that the two levers (Dom0 and DomU) are complementary and not alternatives.

K. TACLe BENCHMARK

This section of the documentation illustrates the rationale behind the selection of the benchmarks extracted from the **TACLeBench** version 1.9 suite and the methodology adopted for their execution within our architecture. The goal is to provide a heterogeneous workload to accurately validate execution latencies and performance stability in environments with strict real-time requirements.

1) *1. Benchmark Selection:* We selected a representative program for each of the main TACLeBench categories, ensuring optimal coverage of the different execution patterns.

- **Kernel Benchmark (matrix1):** Isolates the most computationally intensive portions of code to evaluate raw CPU performance and cache efficiency.
- **Sequential Benchmark (huff_enc):** Evaluates sequential processing and memory access patterns. Data compression (325 SLOC, David Bourgin) is excellent for measuring latency variations in single-threaded execution.
- **Test Benchmark (test3):** An artificial stress test for WCET (Worst-Case Execution Time, 4235 SLOC, Universität des Saarlandes) analysis. It pushes the execution engine to its limits to measure time safety margins and system robustness.
- **Parallel Benchmark (Debie):** Aerospace observation tool (6615 SLOC, Tidorum Ltd) consisting of 8 tasks. Essential for testing synchronization mechanisms and preemption in multi-tasking scenarios.

- **Application Benchmark (lift):** An elevator controller (361 SLOC, Martin Schoeberl). Verifies that the latency metrics from synthetic tests guarantee stability in a real cyber-physical control application.

2) *2. Execution Methodology and Test Scenarios:* To rigorously analyze the system's behavior and the impact of the virtualization architecture, **all 5 selected benchmarks were executed on the LL-RT (Low-Latency Dom0, Real-Time DomU) configuration.**

For each benchmark, we defined a test matrix composed of four operational scenarios. In all baseline configurations, **4 vCPUs were assigned to Dom0**. The analyzed variables concern the introduction of a stress load (noise) of varying size originating from another DomU and the application of vCPU pinning (crucial for avoiding context migrations and stabilizing latencies).

To ensure strict real-time conditions and accurate latency measurements, the execution procedure was carefully standardized across all test scenarios. The process involved specific compilation flags, scheduler modifications, and rigid execution parameters designed to eliminate typical operating system interference.

Compilation and Preparation Each of the benchmarks were modified to support automatically changing scheduling priority and repeated executions (the source code is available in this repository). We then compiled them using -O2 optimizations and linked with the necessary POSIX real-time and threading libraries:

```
gcc -O2 -o test3 test3.c -lpthread -lrt
```

Benchmark Execution Parameters The TACLe benchmarks were executed via the command line with a strict set of arguments to maintain deterministic behavior. The fundamental flags applied across the tests included:

- **--mlockall:** Locks the process's memory pages directly into RAM, preventing any unpredictable latency spikes caused by page faults or disk swapping.
- **--priority=99:** Enforces the highest real-time priority internally for the benchmark's execution thread.
- **--affinity=1:** Applies CPU pinning, locking the execution strictly to CPU core 1. This is a critical step to prevent costly context migrations across different processor cores.

Below are the exact execution commands utilized for the targeted benchmarks:

```
# Executing test3 (pseudo-cyclicttest) for 10,000
↳ loops and outputting a latency histogram
sudo ./test3 --mlockall --priority=99 --affinity=1
↳ --loops=10000 --histogram=1000000
↳ --histfile=results/results_test3_baseline.log
```

Noise Generation Strategy To evaluate the architectural robustness and jitter expansion during the noisy scenarios, interference was artificially injected into the system. This was achieved using **stress-ng** to spawn multiple aggressive workers, intentionally taxing the CPU cores and the virtual

memory subsystem to simulate severe cache thrashing and scheduling pressure:

```
stress-ng --cpu 16 --vm 16 --vm-bytes 1G --timeout
→ 10m
```

Domain configuration To clearly understand the extent of the interference of a different DomU stress workload on the critical guest, we designed two test scenarios:

- **Small Noise:** a small DomU with just 2 vCPUs and 4 GB of RAM. This would introduce a small amount of noise, thus testing if at least some isolation is provided.
- **Big Noise:** a more beefy DomU configuration with 16 vCPUs and 16 GB of RAM. In this case, the interference introduced is more severe, pushing the limits of the isolation mechanism.
- **Big Noise with pinning:** the same of the previous setup but with added static vCPU pinning. This would determine how much the scheduling algorithm affects the isolation of the domains.

3) *TACLeBench Baseline Execution Analysis:* Establishing this baseline is a critical first step for comparing the determinism and worst-case execution time (WCET) latencies, a Type 1 hypervisor utilizing Dom0 and DomU architectures. The following analysis evaluates the raw execution logs to quantify system determinism before introducing vCPU pinning.

a) *Baseline Latency Summary:*

b) *Workload-Specific Behavior:* **DEBIE**

- The DEBIE benchmark represents the heaviest and most unpredictable workload in the dataset, with a massive spread between the minimum (23,921 μs) and maximum (31,048 μs) execution times.
- The absolute jitter of over 7 milliseconds indicates significant preemption, cache misses, or scheduling overhead during execution.
- This high variability makes DEBIE an excellent candidate for stress-testing how well future DomU or virtualized configurations handle long-running, computationally complex real-time tasks.

Huffenc

- The **huffenc** benchmark represents a short-lived task executed 1,000,000 times.
- It shows excellent stability with an average latency of 17 μs and a worst-case peak of 41 μs .
- The absolute jitter is strictly bounded at 26 μs , derived from a minimum latency of 15 μs .

Lift

- The **lift** benchmark shows a moderate average latency of 24 μs , sitting noticeably above the minimum latency (19 μs) across one million loops.
- The maximum latency peak of 254 μs represents a significant outlier, yielding an absolute jitter of 235 μs .

- While the majority of executions cluster near 19-24 μs , this tail latency shows that even on the unisolated baseline scheduler, **lift** is subject to occasional severe delays.

Matrix1

- The **matrix1** execution is extremely lightweight, registering a 0 μs average latency, which indicates that standard execution times fall below the microsecond resolution threshold of the testing configuration.
- The absolute worst-case execution time caps at just 9 μs .
- Because this benchmark is practically instantaneous, it is almost entirely cache-bound and will be highly sensitive to hypervisor memory management disruptions.

Test3

- The **test3** benchmark represents a heavier execution profile containing 10,000 loops, an average latency of 8,328 μs , and a maximum of 8,414 μs .
- The tighter jitter (128 μs) relative to its extended execution time suggests a steady computational loop that is less affected by micro-interruptions compared to the highly variable DEBIE benchmark.

c) *Real-Time Systems Assessment:* This baseline demonstrates a standard, unisolated environment where lightweight tasks (**matrix1**, **huffenc**) execute with near-perfect determinism, while heavier tasks (DEBIE) suffer from severe scheduling jitter. **lift** also shows a notable outlier tail (235 μs jitter, 254 μs peak) despite a low average, indicating it is more exposed to occasional scheduling delays than its average-case behavior would suggest. The outliers observed in **lift** (254 μs) and **huffenc** (41 μs) are the specific OS noise artifacts that isolation mechanisms aim to eliminate. When transferring these workloads to virtualized setups, tracking the expansion of these maximum latency tails will directly quantify the scheduling interference introduced by the virtualization layer.

4) *TACLeBench Small Noise Execution Analysis:* This phase of testing evaluates the determinism and worst-case execution time (WCET) latencies of KVM and Xen (a Type 1 hypervisor utilizing Dom0 and DomU architectures) under a “Small Noise” configuration. The following analysis interprets the execution logs to quantify how minor system disturbances impact the predictability of the hypervisor scheduling.

a) *Small Noise Latency Summary:*

b) *Workload-Specific Behavior:* **DEBIE**

- The DEBIE benchmark remains the heaviest workload, with execution times ranging from a minimum of 23,873 μs to a maximum of 31,022 μs .
- The absolute jitter sits at 7,149 μs , confirming its high sensitivity to preemption and scheduling overhead even in a small noise environment.

Huffenc

TABLE XXXV
XEN BASELINE LATENCY SUMMARY

Benchmark	Total Loops	Min Latency (μs)	Avg Latency (μs)	Max Latency (μs)	Absolute Jitter (Max - Min)
DEBIE	10,000	23,921	27,175	31,048	7,127 μs
huffenc	1,000,000	15	17	41	26 μs
lift	1,000,000	19	24	254	235 μs
matrix1	1,000,000	0	0	7	7 μs
test3	10,000	8,286	8,328	8,414	128 μs

TABLE XXXVI
XEN SMALL NOISE LATENCY SUMMARY

Benchmark	Total Loops	Min Latency (μs)	Avg Latency (μs)	Max Latency (μs)	Absolute Jitter (Max - Min)
DEBIE	10,000	23,873	27,118	31,022	7,149 μs
huffenc	1,000,000	15	17	741	726 μs
lift	1,000,000	19	20	43	24 μs
matrix1	1,000,000	0	0	10	10 μs
test3	10,000	8,286	8,331	8,885	599 μs

- The **huffenc** benchmark maintains a stable average latency of 17 μs across 1,000,000 iterations, close to its baseline behavior.
- However, the maximum latency spikes sharply to 741 μs — well above both the baseline (41 μs) and even the big noise scenario (445 μs) — producing an absolute jitter of 726 μs . This result has been confirmed directly against the raw execution logs: it is a genuine tail-latency event rather than a data or transcription error, and it stands out as the single largest WCET recorded for **huffenc** across every tested scenario, including heavier noise conditions.

Lift

- Under the small noise configuration, the **lift** benchmark exhibits improved determinism versus baseline, with an average latency of 20 μs .
- The maximum latency peak is 43 μs , representing a dramatically tighter jitter (24 μs) compared to the unisolated baseline’s 235 μs .

Matrix1

- The **matrix1** execution remains extremely lightweight and cache-bound, registering a 0 μs average latency.
- The absolute worst-case execution time is capped at 10 μs .

Test3

- The **test3** benchmark executed 10,000 loops with an average latency of 8,331 μs .
- The maximum latency reached 8,885 μs , pushing the absolute jitter to 599 μs . The jitter expansion here is heavily influenced by specific micro-interruptions captured during the run.

c) Real-Time Systems Assessment: Evaluating the small noise scenario reveals that lightweight and highly repetitive tasks (**matrix1**, **lift**) can maintain extreme determinism, with jitter boundaries narrowing significantly (such as **lift** dropping to a 43 μs max peak from 254 μs at baseline). **huffenc**, however, produces a confirmed tail-latency outlier (741 μs WCET) under this “small” noise level — exceeding even its own big-noise worst case (445 μs) — a counterintuitive but log-verified result that highlights how a single rare preemption event can dominate the worst-case metric for short, high-frequency tasks, independent of the nominal “size” of the background noise. Heavier workloads like **DEBIE** continue to exhibit substantial variance.

5) TACLeBench Big Noise Execution Analysis: This phase of the evaluation investigates the determinism and worst-case execution time (WCET) latencies of a Xen DomU in the presence of another, big-sized DomU affected by significant stress workload. The following analysis examines the execution logs to quantify how significant system disturbances and heavy background noise degrade the predictability of the scheduler prior to applying isolation techniques.

a) Big Noise Latency Summary:

*b) Workload-Specific Behavior: **DEBIE***

- Under heavy noise, the **DEBIE** benchmark suffers massive scheduling disruptions, pushing the worst-case execution time to 57,314 μs .
- The absolute jitter explodes to 32,687 μs , meaning the variance in execution time is larger than the minimum latency itself (24,627 μs).

Huffenc

- Under heavy noise, the **huffenc** task’s average latency rises to 27 μs (up from 17 μs at baseline, a +58.78% increase), and the maximum latency spikes to 445 μs

TABLE XXXVII
XEN BIG NOISE LATENCY SUMMARY

Benchmark	Total Loops	Min Latency (μs)	Avg Latency (μs)	Max Latency (μs)	Absolute Jitter (Max - Min)
DEBIE	10,000	24,627	28,196	57,314	32,687 μs
huffenc	1,000,000	15	27	445	430 μs
lift	1,000,000	17	38	1518	1498 μs
matrix1	1,000,000	0	0	51	51 μs
test3	10,000	8,287	9,182	14,786	6,499 μs

— a **+987.19% increase in WCET** versus baseline, the largest percentage WCET degradation of any lightweight benchmark in this scenario.

- This results in an absolute jitter of 430 μs , confirming that even high-frequency, short-lived tasks are heavily preempted in a noisy environment.

Lift

- The **lift** benchmark’s determinism collapses entirely under the big noise configuration, with the maximum latency spiking to 1,518 μs — a full millisecond, and nearly 6x the baseline’s already-elevated 254 μs peak.
- The average latency also rises substantially to 38 μs (up from 24 μs at baseline, a +55.89% increase), and the absolute jitter of 1498 μs confirms severe, sustained scheduling delays rather than an isolated outlier.

Matrix1

- Although the average latency remains at 0 μs due to the task’s cache-bound, lightweight nature, the maximum latency expands to 45 μs .
- An absolute jitter of 45 μs on a task that typically executes instantaneously highlights severe micro-interruptions and cache thrashing induced by the noise.

Test3

- The **test3** benchmark registers an average latency of 9,182 μs and a severe worst-case peak of 14,786 μs .
- The absolute jitter jumps to 6,499 μs , derived from a minimum execution time of 8,287 μs , proving that medium-weight computational loops cannot maintain steady execution states when sharing unisolated CPU resources with heavy noise.

c) Real-Time Systems Assessment: The results of these tests effectively demonstrates the catastrophic loss of determinism across all workloads while the system is under stress because of another DomU. Even ultra-lightweight tasks like **matrix1**, **huffenc** and **lift** experience significant latency spikes — **lift** reaches a full millisecond of worst-case latency (1,000 μs), and **huffenc** shows the steepest relative WCET degradation of the suite (+987.19% versus baseline) — while heavy tasks like **DEBIE** become entirely unpredictable. This demonstrates the ineffectiveness of the isolation boundaries of the native Xen architecture.

6) TACLeBench Big Noise Pinned Execution Analysis:

This phase of the evaluation investigates the determinism and worst-case execution time (WCET) latencies of a Xen DomU in the presence of another, big-sized DomU affected by significant stress workload. Because of the poor performance of the unpinned configuration, exhibiting excessively high latencies, CPU pinning was introduced to restrain the interference. The following analysis examines the execution logs to quantify how this pinning mitigates system disturbances.

a) *Big Noise Pinned Latency Summary:*

b) *Workload-Specific Behavior:* **DEBIE**

- The **DEBIE** benchmark ranges from a minimum of 24,034 μs to a maximum of 41,266 μs .
- Pinning the execution restrains the absolute jitter to 17,232 μs , which is a massive improvement over the unpinned big noise scenario, though it remains sensitive to preemption.

Huffenc

- The **huffenc** task executes with an average latency of 17 μs and a peak maximum latency of 41 μs , both back in line with baseline behavior.
- CPU pinning effectively bounds the absolute jitter at 26 μs , pulling the WCET down from 445 μs (unpinned Big Noise) — and dramatically down from the 741 μs outlier observed under Small Noise — back to baseline levels, restoring a high degree of stability to this lightweight loop.

Lift

- Under the pinned configuration, the **lift** benchmark regains much of its determinism, maintaining an average latency of 20 μs .
- The maximum latency reaches 57 μs , yielding an absolute jitter of 38 μs — dramatically reduced from the 983 μs jitter seen under unpinned big noise, though still above the small noise scenario’s 24 μs .

Matrix1

- The **matrix1** execution is almost perfectly insulated by the pinning, maintaining a 0 μs average latency and an absolute worst-case execution time of just 11 μs .

TABLE XXXVIII
XEN BIG NOISE PINNED LATENCY SUMMARY

Benchmark	Total Loops	Min Latency (μs)	Avg Latency (μs)	Max Latency (μs)	Absolute Jitter (Max - Min)
DEBIE	10,000	24,034	27,274	41,266	17,232 μs
huffenc	1,000,000	15	17	41	26 μs
lift	1,000,000	19	20	57	38 μs
matrix1	1,000,000	0	0	11	11 μs
test3	10,000	8,632	8,701	10,037	1,405 μs

- This 11 μs absolute jitter confirms that cache thrashing and severe scheduling interruptions are heavily mitigated.

Test3

- The **test3** benchmark registers an average latency of 8,701 μs and a worst-case peak of 10,037 μs .
- With an absolute jitter of 1,405 μs , this computational loop shows a massive recovery in predictability compared to the unpinned execution.

c) *Real-Time Systems Assessment*: The “Big Noise Pinned” baseline demonstrates the critical importance of CPU pinning when operating in highly congested environments. By binding tasks to specific cores, the hypervisor scheduler prevents the catastrophic latency spikes observed in the unpinned tests — **huffenc**’s WCET falls from 445 μs (and from the 741 μs Small Noise outlier) back to 41 μs , and **lift**’s WCET falls from 1,000 μs back to 57 μs once pinned. Lightweight tasks (**matrix1**, **lift**, **huffenc**) return to near-baseline determinism, and heavier workloads (**DEBIE**, **test3**) see their jitter margins compressed significantly.

7) *Max Latency Summary (μs)*: To quickly evaluate system stability, the following table exclusively reports the peak values (**Max Latency**). This format allows for an at-a-glance comparison of the Worst-Case Execution Time across the four operational scenarios, directly highlighting the impact of noise and the effectiveness of CPU pinning in containing interference.

Furthermore, alongside the WCET summary, this section now includes a detailed breakdown of the percentage increments for both the average latency (expressed as Mean $\pm$ Standard Deviation) and the maximum latency (WCET). This addition provides a precise quantitative analysis of the performance degradation induced by the heavy background noise within the control domain (Dom0) compared to the baseline execution for each individual benchmark.

a) **DEBIE: PERCENTAGE INCREMENTS (Stress vs. Baseline)**:

- Average Increment: +3.76%
- WCET Increment: +84.60%

b) **HUFFENC: PERCENTAGE INCREMENTS (Stress vs. Baseline)**:

- Average Increment: +58.78%
- WCET Increment: +987.19%

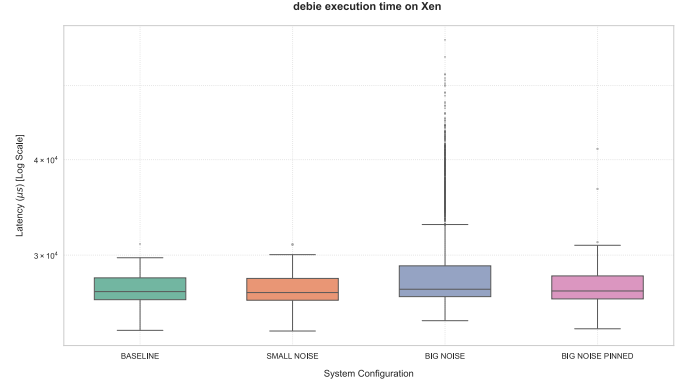


Fig. 34. TACLe benchmark - debie execution time on Xen

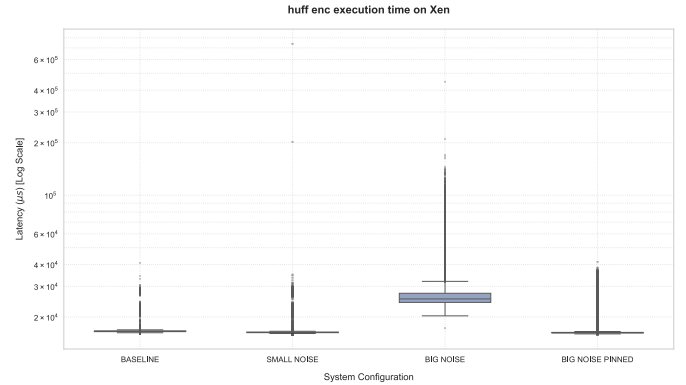


Fig. 35. TACLe benchmark - huff_enc execution time on Xen

c) **LIFT: PERCENTAGE INCREMENTS (Stress vs. Baseline)**:

- Average Increment: +55.89%
- WCET Increment: +498.77%

d) **MATRIX1: PERCENTAGE INCREMENTS (Stress vs. Baseline)**:

- Average Increment: +46.12%
- WCET Increment: +586.20%

e) **TEST3: PERCENTAGE INCREMENTS (Stress vs. Baseline)**:

- Average Increment: +10.26%
- WCET Increment: +75.73%

TABLE XXXIX
XEN WCET PEAK VALUES SUMMARY

Benchmark	Baseline	Small Noise	Big Noise	Big Noise Pinned
DEBIE	31,048	31,022	57,314	41,266
huffenc	41	741	445	41
lift	254	43	1,518	57
matrix1	7	10	51	11
test3	8,414	8,885	14,786	10,037

TABLE XL
XEN DEBIE EXECUTION TIME STATISTICS

METRIC	BASELINE	SMALL NOISE	BIG NOISE	BIG NOISE PINNED
Average $\pm$ SD	27175.71 $\pm$ 1309.25 μs	27118.01 $\pm$ 1312.34 μs	28196.64 $\pm$ 3277.83 μs	27274.32 $\pm$ 1328.13 μs
WCET (Max)	31048 μs	31022 μs	57314 μs	41266 μs

TABLE XLI
XEN HUFF_ENC EXECUTION TIME STATISTICS

METRIC	BASELINE	SMALL NOISE	BIG NOISE	BIG NOISE PINNED
Average $\pm$ SD	16756.76 $\pm$ 1304.86 ns	16533.80 $\pm$ 1568.45 ns	26606.56 $\pm$ 5118.09 ns	16568.86 $\pm$ 1794.49 ns
WCET (Max)	40959 ns	740560 ns	445304 ns	41370 ns

TABLE XLII
XEN LIFT EXECUTION TIME STATISTICS

METRIC	BASELINE	SMALL NOISE	BIG NOISE	BIG NOISE PINNED
Average $\pm$ SD	24464.53 $\pm$ 2225.55 ns	20106.23 $\pm$ 1479.67 ns	38136.97 $\pm$ 11983.88 ns	20254.04 $\pm$ 2140.08 ns
WCET (Max)	253587 ns	43321 ns	1518396 ns	57119 ns

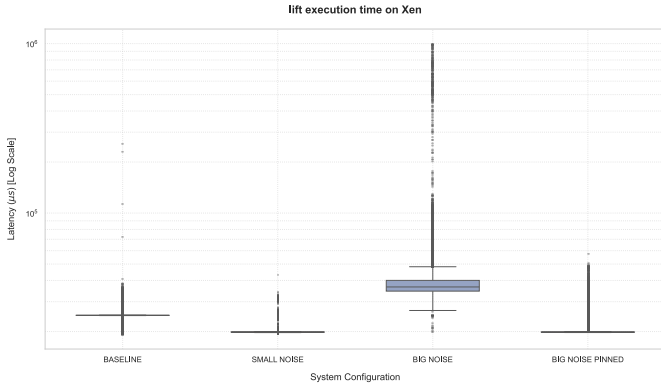


Fig. 36. TACLe benchmark - lift execution time on Xen

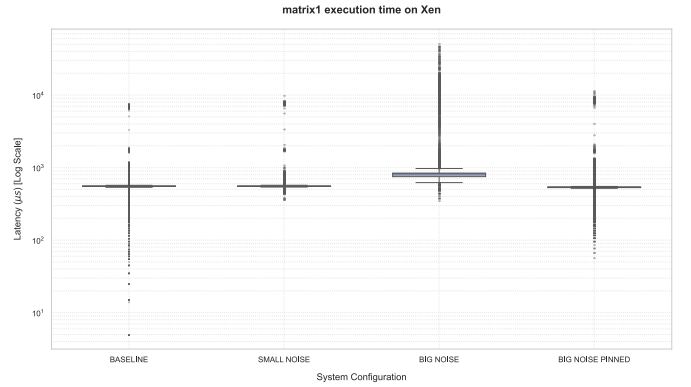


Fig. 37. TACLe benchmark - matrix1 execution time on Xen

8) *TACLeBench with stress on Dom0*: As previously mentioned, following the tests conducted, new studies have been added: the latter compares the temporal behavior (execution latency) of five benchmarks from the TACLe suite — debie, huff_enc, lift, matrix1, test3 — executed inside an HVM DomU on Xen, a Type 1 hypervisor, under two configurations of the low-latency Dom0 kernel (“LL”, Linux 6.18.35-rt5-ll): without vCPU pinning and with pinning. The goal is to understand to what extent stress on Dom0 actually matters to the performance of a DomU,

comparing it to the effects another noisy DomU had.

Data collection was automated with the provided bash script, which orchestrates the entire end-to-end experiment:

- **Dom0**: Xen 4.17-amd64 on Linux 6.18.35, 22 allocated vCPUs; kernel switching managed via grub-reboot across four variants (NRT, NRT-pinned, LL, LL-pinned).
- **DomU**: HVM Ubuntu 24.04 DomU, in two configuration variants — unpinned and pinned — corresponding

TABLE XLIII
XEN MATRIX1 EXECUTION TIME STATISTICS

METRIC	BASELINE	SMALL NOISE	BIG NOISE	BIG NOISE PINNED
Average $\pm$ SD	556.31 $\pm$ 154.19 ns	557.07 $\pm$ 162.41 ns	812.91 $\pm$ 437.85 ns	545.04 $\pm$ 180.80 ns
WCET (Max)	7470 ns	9550 ns	51259 ns	11321 ns

TABLE XLIV
XEN TEST3 EXECUTION TIME STATISTICS

METRIC	BASELINE	SMALL NOISE	BIG NOISE	BIG NOISE PINNED
Average $\pm$ SD	8328.16 $\pm$ 38.48 μ s	8331.15 $\pm$ 39.52 μ s	9182.95 $\pm$ 1317.37 μ s	8701.25 $\pm$ 43.68 μ s
WCET (Max)	8414 μ s	8885 μ s	14786 μ s	10037 μ s

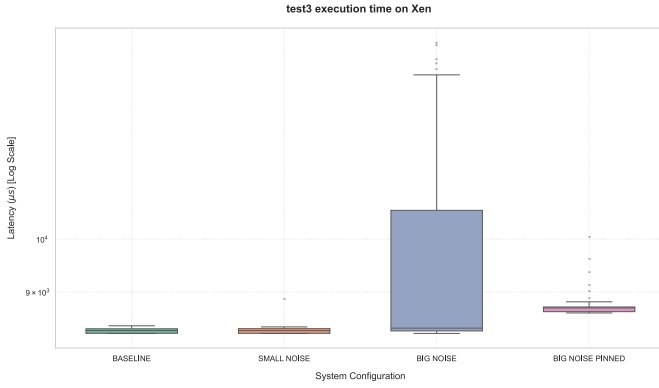


Fig. 38. TACLe benchmark - test3 execution time on Xen

to the type of Dom0 kernel under test.

- **Background workload (“always-on”):** `stress-ng --cpu 22` and `stress-ng --vm 12 --vm-bytes 2G`, started on Dom0 before each run and kept active for the entire duration of the scenario, to simulate a Dom0 under realistic pressure.
- **Sequential stressors** planned by the script: baseline (no additional stressor), cache, interrupts, rawsock — launched one at a time alongside the background workload.
- **Automation:** creation/destruction of the DomU via `xl`, collection of log files from the DomU’s virtual disk after shutdown, process and cache cleanup between runs.

Methodological note: the 10 provided files are execution time histograms of the TACLe benchmarks themselves (not of `cyclictest`, which is the workload shown in the script). The file names do not report which sequential stressor scenario was active during capture — this is information to be retrieved in order to confidently attribute the observed differences solely to pinning and not to a mix of different conditions between the two runs.

a) *Statistics extracted per file:*

b) *Results:* The following graph summarizes, for each benchmark, the percentage variation of the mean, p99, and maximum when going from LL to LL-pinned (negative

values = improvement).

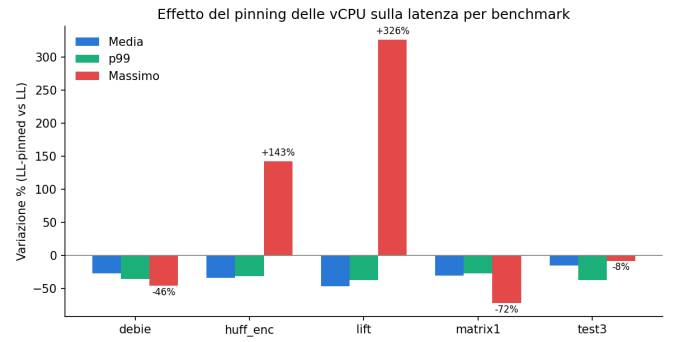


Fig. 39. Effetto del pinning delle vCPU sulla latenza per benchmark

Percentage variation of the mean, p99, and maximum (LL-pinned vs LL) for the five TACLe benchmarks: debie, huff_enc, lift, matrix1, test3.

c) *% Variation LL-pinned vs LL:*

d) *1- Pinning systematically improves typical latency:* Across all five benchmarks, without exception, vCPU pinning reduces both the mean latency (from -15% to -46%) and the p99 (from -27% to -37%). This confirms the expected effect: fixing the DomU vCPUs to dedicated physical cores eliminates the variability introduced by the Dom0 scheduler and inter-core migrations, making the temporal behavior more predictable in the typical case.

e) *2- Tail anomaly: pinning worsening the worst case:* On debie, matrix1, and test3, pinning also improves the worst-case (lower maximum). But on lift and huff_enc the opposite happens: the maximum latency increases drastically (+326% on lift, +143% on huff_enc), and huff_enc-pinned registers a histogram overflow, a sign that at least one sample exceeded even the maximum expected bucket — the actual peak could therefore be even higher than reported.

This is the most significant result to document: pinning is not a unilateral guarantee of improvement. It reduces variance in the common case, but if the pinned core is not completely isolated from external interruptions (housekeeping IRQs, Dom0 timers, one of the stressors), when that rare interference occurs its impact is concentrated entirely

TABLE XLV
XEN STATISTICS EXTRACTED PER FILE

Benchmark	Config	Min (μs)	Average (μs)	p99 (μs)	Max (μs)	Overflow
debie	LL	30212.0	37133.0	46259.0	72372.0	0
debie	LL-pinned	23976.0	27275.0	29703.0	39230.0	0
huff_enc	LL	15.8	25.0	38.9	496.2	0
huff_enc	LL-pinned	15.9	16.5	26.7	1204.0	1
lift	LL	29.7	37.6	52.3	211.6	0
lift	LL-pinned	19.7	20.2	32.9	902.5	0
matrix1	LL	0.65	0.79	0.94	47.0	0
matrix1	LL-pinned	0.52	0.55	0.69	13.1	0
test3	LL	8322.0	9789.0	13279.0	17705.0	0
test3	LL-pinned	8228.0	8277.0	8335.0	16260.0	0

TABLE XLVI
XEN VARIATION LL-PINNED VS. LL

Benchmark	$\Delta Average$	$\Delta p99$	ΔMax
debie	-26.5%	-35.8%	-45.8%
huff_enc	-34.1%	-31.4%	+142.6%
lift	-46.3%	-37.0%	+326.5%
matrix1	-30.7%	-26.6%	-72.1%
test3	-15.4%	-37.2%	-8.2%

on a single dedicated core instead of being distributed, producing an isolated spike much larger than what would happen with free scheduling.

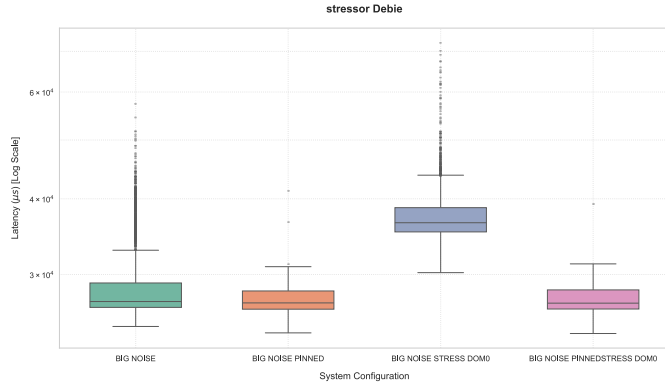


Fig. 40. TACLe benchmark - debie stress dom0

f) 3- Consistency between scale and behavior: The order of magnitude of latency varies greatly between benchmarks (from hundreds of nanoseconds for matrix1 to tens of milliseconds for debie), reflecting the different computational complexity of the TACLe workloads. The mean/p99 improvement pattern with pinning is maintained regardless of the scale, which reinforces the idea that it is a structural effect of pinning itself and not an artifact linked to the duration of the single benchmark.

L. PV AND PVH DOMUS

In their previous work, Abeni and Faggioli concluded that in Xen, virtualization technology played a major role in scheduling latencies due to a priority inversion bug in

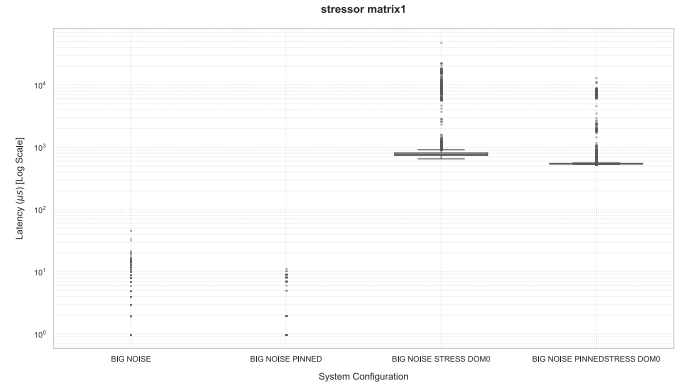


Fig. 41. TACLe benchmark - matrix1 stress dom0

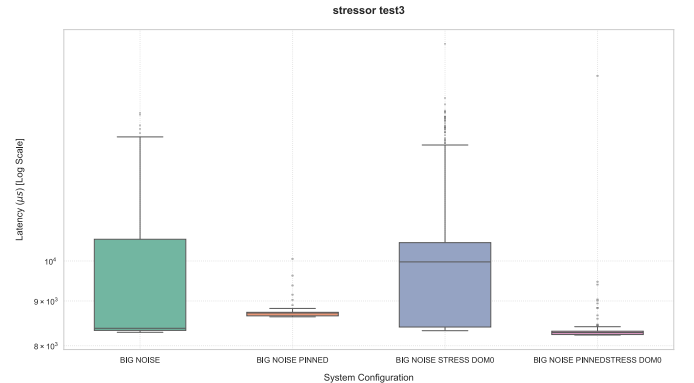


Fig. 42. TACLe benchmark - test3 stress dom0

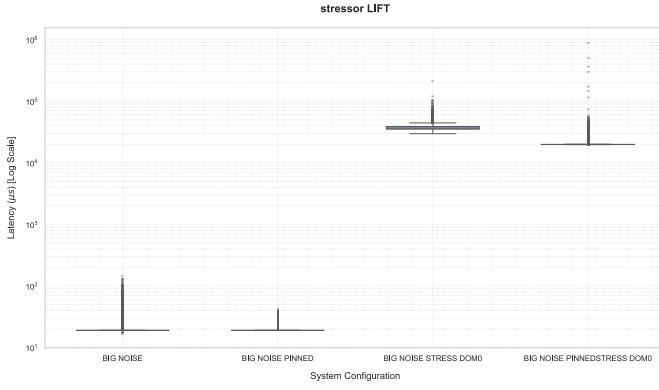


Fig. 43. TACLe benchmark - lift stress dom0

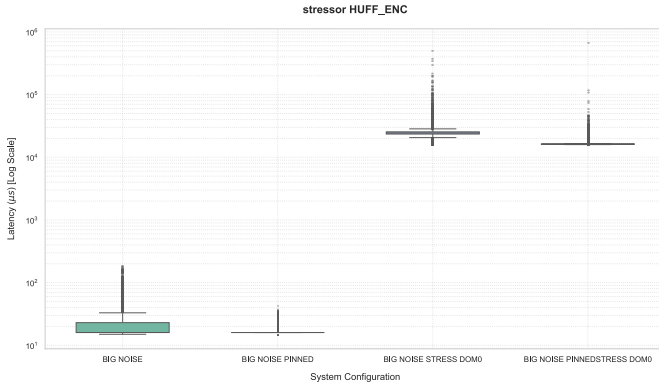


Fig. 44. TACLe benchmark - huff_enc stress dom0

how the Device Model operated. Specifically, they noted that the QEMU process acting as the DomU DM did not execute with high priority, allowing it to be preempted by the workload.

We previously attempted to reproduce this issue using a newer version of Xen (4.17.3) but did not observe the same trend. We also tried assigning maximum priority to the QEMU process in Dom0, which yielded no noticeable improvement. Consequently, we concluded that this priority inversion bug has been fixed for HVM DomUs.

To further verify this conclusion, we extended our analysis to PV and PVH guests. In this section, we reproduce the most relevant scenarios for guests running on these different virtualization technologies and analyze their resulting behaviors.

1) RT Kernel and PARAVIRTUALIZATION: While configuring the PV guest, we encountered the same issue observed during the initial setup process. This time, because the output logs were routed to the terminal, we were able to identify the cause. The logs revealed a **soft CPU lockup**, confirming that the issue stemmed from the **PREEMPT_RT** patch. We hypothesize that this failure relates to how the patch modifies low-level mechanisms—such as replacing spinlocks with mutexes—which introduces compatibility issues with paravirtualization. This also explains why the **PREEMPT_RT** patch failed in Dom0, as it is inherently a PV

guest. Additionally, we attempted to configure Dom0 as a PVH guest; however, this setup resulted in continuous automatic reboots. Without access to system logs or graphical output to diagnose the root cause, we were unable to troubleshoot the error and ultimately abandoned this configuration. Consequently, for the subsequent tests, we replaced the RT kernel with the Low-Latency kernel in the DomU as well.

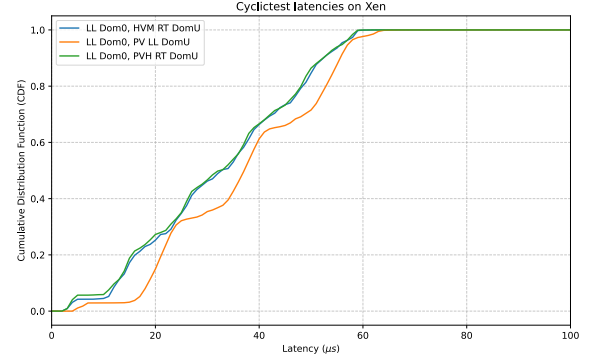


Fig. 45. Comparing HVM, PV and PVH guests - Credit2 Scheduler (No Noise)

2) LOW LATENCY DOM0 AND LOW LATENCY DOMU (PV DomU): This section analyzes the results obtained from a 5-minute execution of the **cyclictest** utility within a Xen virtualized environment, explicitly assessing a Paravirtualized (PV) guest. The configuration features a “Low Latency” kernel deployed on both the privileged domain (Dom0) and the unprivileged user domain (DomU). This test evaluates the baseline performance of the dynamic scheduler without static vCPU pinning and without any artificial stress workload applied to Dom0.

a) Nominal Performance and Average Latency: The data obtained from the **cyclictest** execution reveals the baseline virtualization overhead and scheduling behavior in an undisturbed, unpinned PV configuration. The average latency recorded during the test was $37 \mu s$, significantly higher than any other baseline average latency we recorded. The absolute minimum latency achieved was $4 \mu s$.

b) Worst-Case Execution Time (WCET) Analysis: The analysis of the Worst-Case Execution Time (WCET) provides insight into the system’s baseline ability to bound execution delays. Over the duration of the test, the absolute maximum latency recorded was $525 \mu s$.

c) Observations on the Baseline PV Unpinned Environment: The empirical data collected from this test yields the following observations regarding the behavior of the Low Latency PV DomU running on a Low Latency Dom0 without vCPU pinning in a quiet environment:

- **Bounded WCET:** The maximum latency was contained at $525 \mu s$.
- **Baseline Jitter:** The average latency of $37 \mu s$ confirms the presence of scheduling jitter.

3) LOW LATENCY DOM0 AND REAL-TIME DOMU (PVH DomU): This section analyzes the results obtained

from a 5-minute execution of the `cyclictest` utility within a Xen virtualized environment, explicitly assessing a PVH guest. The configuration features a “Low Latency” kernel deployed on the privileged domain (Dom0) and a Real-Time (`PREEMPT_RT`) kernel on the unprivileged user domain (DomU). This test evaluates the baseline performance of the dynamic Credit2 scheduler without static vCPU pinning and without any artificial stress workload applied to Dom0.

a) Nominal Performance and Average Latency: The data obtained from the `cyclictest` execution reveals the baseline virtualization overhead and scheduling behavior in an undisturbed, unpinned configuration. The average latency recorded during the test was $32 \mu s$. The absolute minimum latency achieved was $3 \mu s$. The histogram data indicates a broad distribution of execution latencies, demonstrating the variability introduced by dynamic scheduling even in the absence of host-level contention.

b) Worst-Case Execution Time (WCET) Analysis: The analysis of the Worst-Case Execution Time (WCET) provides insight into the system’s baseline ability to bound execution delays. Over the duration of the test, the absolute maximum latency recorded was $76 \mu s$. The system successfully avoided any histogram overflows.

c) Observations on the Baseline PVH Unpinned Environment: The empirical data collected from this test yields the following observations regarding the behavior of the RT PVH DomU running on a Low Latency Dom0 without vCPU pinning in a quiet environment:

- **Bounded WCET:** The maximum latency was contained at $76 \mu s$, indicating that the combination of the RT guest kernel and the Low Latency host kernel provides a stable upper bound when the host is not under load.
- **Baseline Jitter:** The average latency of $32 \mu s$ and the wide spread of nominal execution times confirm the presence of significant scheduling jitter. This variability highlights the inherent impact of the dynamic Credit2 hypervisor scheduler, even without resource contention from a `stressdom0` workload.

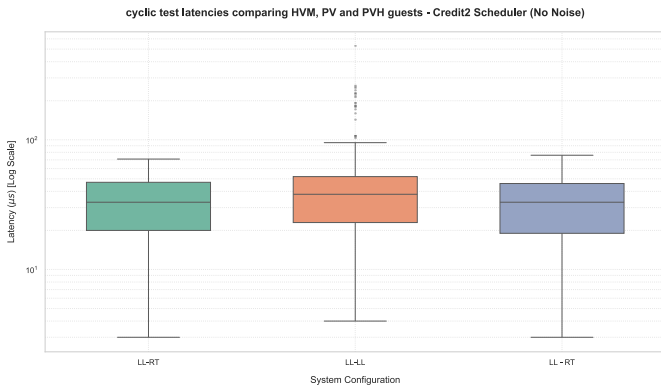


Fig. 46. Comparing HVM, PV and PVH guests - Credit2 Scheduler (No Noise)

M. IMPACT OF THE STRESS WORKLOAD ON DIFFERENT VIRTUALIZATION TECHNOLOGIES

To assess the influence of the underlying virtualization architecture on system determinism, this phase of testing introduces a background stress workload in the privileged domain (Dom0) while executing the `cyclictest` probe within PV and PVH guests. Building upon our earlier findings—which demonstrated that modern Hardware Virtual Machine (HVM) configurations successfully manage latency bounds without suffering from historical QEMU-induced preemption anomalies—this evaluation aims to compare how alternative virtualization models respond to resource contention. Ultimately, the analysis highlights that while PVH and HVM achieve highly similar performance levels, the PV architecture exhibits the worst performance among all three technologies.

Experimental analysis reveals that moving away from full hardware virtualization fundamentally alters the system dynamics, highlighting key architectural trade-offs when employing PV and PVH configurations:

- **Absence of the Device Model:** Unlike HVM guests, PV and PVH architectures do not rely on a QEMU instance running in Dom0 for hardware emulation. While our previous tests confirmed that modern Xen deployments resolve the historical priority inversion bugs associated with QEMU, evaluating PV and PVH guests allows us to observe system behavior completely isolated from Device Model interactions.
- **Constraints on Kernel Optimization:** While paravirtualization removes the Device Model variable, it introduces strict constraints regarding real-time optimizations. As established during the initial setup, the inherent incompatibility of the `PREEMPT_RT` patch with PV guests necessitated a fallback to a Low Latency kernel. This dynamic fundamentally shifts the performance bottleneck from hypervisor-level interference to guest-level scheduling limitations, severely penalizing the PV configuration.
- **Comparative Resilience to Contention:** Evaluating these paravirtualized environments under Dom0 stress provides a direct contrast to the HVM data. The empirical results reveal that the intrinsically lighter virtualization footprint of the PV architecture cannot compensate for the lack of rigorous, real-time guest optimizations, resulting in the poorest latency bounds. Conversely, the PVH architecture overcomes these limitations, demonstrating a resilience to contention and overall performance metrics that are remarkably similar to those of HVM.

In this section, we present a detailed comparative analysis of these configurations, quantifying the execution latencies of PV and PVH guests under stress to determine their overall viability for predictable, latency-sensitive applications compared to their HVM counterparts.

1) *LOW LATENCY DOM0 AND LOW LATENCY DOMU (PV DomU):* This section details the analysis of a 5-

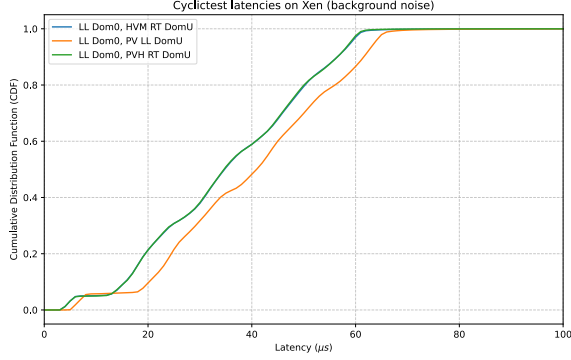


Fig. 47. Comparing HVM, PV and PVH guests - Credit2 Scheduler (Background Noise)

minute execution of the `cyclictest` utility within a Xen virtualized environment, specifically evaluating a Paravirtualized (PV) guest. This configuration features a “Low Latency” kernel deployed on both the privileged domain (Dom0) and the unprivileged user domain (DomU). Crucially, this test was conducted without static vCPU pinning and while Dom0 was subjected to a significant background stress workload (`stressdom0`). The objective is to evaluate the latency characteristics and the impact of host-level contention when both domains utilize low-latency optimizations in an unpinned PV environment.

a) Nominal Performance and Average Latency: The data obtained from the `cyclictest` execution reveals the baseline virtualization overhead and scheduling jitter under stress conditions. The average latency recorded during the test was $40\ \mu s$. The absolute minimum latency achieved was $4\ \mu s$. The histogram data indicates a broad distribution of execution latencies, demonstrating the variability introduced by the dynamic Credit2 scheduler when managing host-level contention without vCPU pinning.

b) Worst-Case Execution Time (WCET) Analysis: The analysis of the Worst-Case Execution Time (WCET) provides insight into the system’s ability to bound execution delays under stress. Over the duration of the test, the absolute maximum latency recorded was $312\ \mu s$.

c) Observations on the Stressed PV Unpinned Environment: The empirical data collected from this sustained test yields the following observations regarding the behavior of the Low Latency PV DomU running on a Low Latency Dom0 without vCPU pinning under `stressdom0`:

- **Bounded WCET:** The maximum latency was contained at $312\ \mu s$, indicating that the dual Low-Latency kernel setup provides a degree of stability under stress, preventing extreme multi-millisecond spikes despite the lack of pinning.
- **Stress-Induced Jitter:** The average latency of $40\ \mu s$ and the wide spread of nominal execution times confirm the presence of significant scheduling jitter. This highlights the impact of dynamic hypervisor scheduling and resource contention from the `stressdom0` work-

load on a PV guest.

2) LOW LATENCY DOM0 AND REAL-TIME DOMU (PVH DomU): This section details the analysis of a 5-minute execution of the `cyclictest` utility within a Xen virtualized environment, explicitly assessing a PVH guest. The configuration features a “Low Latency” kernel deployed on the privileged domain (Dom0) and a Real-Time (`PREEMPT_RT`) kernel on the unprivileged user domain (DomU). Crucially, this test was conducted without static vCPU pinning and while Dom0 was subjected to a significant background stress workload. The objective is to evaluate the latency characteristics and virtualization overhead introduced by the hypervisor when managing a PVH guest under these specific, unpinned stress conditions.

a) Nominal Performance and Average Latency: The data obtained from the `cyclictest` execution reveals the baseline virtualization overhead and scheduling jitter inherent in this unpinned configuration. The average latency recorded during the test was $35\ \mu s$. The absolute minimum latency achieved was $3\ \mu s$. The histogram data indicates a broad distribution of execution latencies, demonstrating the variability introduced when dynamic scheduling is utilized under host-level contention.

b) Worst-Case Execution Time (WCET) Analysis: The analysis of the Worst-Case Execution Time (WCET) provides insight into the system’s ability to bound execution delays under stress. Over the duration of the test, the absolute maximum latency recorded was $157\ \mu s$.

c) Observations on the PVH Unpinned Environment: The empirical data collected from this sustained test yields the following observations regarding the behavior of the RT PVH DomU running on a Low Latency Dom0 without vCPU pinning:

- **Bounded WCET:** The maximum latency was contained at $157\ \mu s$, indicating that the combination of the RT guest kernel and the Low Latency host kernel provided a degree of stability, preventing the extreme, multi-millisecond spikes that can occur in less optimized configurations.
- **Hypervisor-Induced Jitter:** The average latency of $35\ \mu s$ and the wide spread of nominal execution times confirm the presence of significant scheduling jitter. This variability highlights the impact of dynamic hypervisor scheduling and resource contention from the `stressdom0` workload when vCPUs are not statically pinned to dedicated physical cores.

N. COMPARATIVE ANALYSIS OF PV AND PVH ARCHITECTURES UNDER STATIC ALLOCATION

Following the initial investigations into dynamic scheduling behavior, this section presents a targeted comparative analysis of execution latencies between Paravirtualized (PV) and Hardware Virtual Machine with PV drivers (PVH) configurations. To eliminate the jitter introduced by complex fair-share algorithms and evaluate the highest degree of determinism achievable, these tests employ

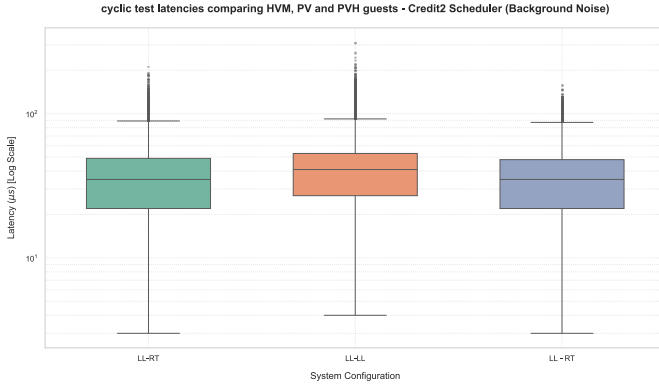


Fig. 48. Comparing HVM, PV and PVH guests - Credit2 Scheduler (Background Noise)

strict vCPU-to-pCPU pinning across both the privileged domain (Dom0) and the unprivileged user domain (DomU), effectively emulating the deterministic behavior of an offline NULL scheduler.

The evaluation is conducted under ideal, unstressed conditions, isolating the system from external background workloads. By analyzing both PV and PVH DomU architectures in a quiet environment, we establish a clean baseline for the inherent virtualization overhead that persists even when static hardware allocation is employed.

This comparative approach aims to quantify the efficacy of strict hardware isolation in bounding the Worst-Case Execution Time (WCET). By observing the system’s baseline predictability, the analysis highlights the specific architectural nuances between PV and PVH environments, demonstrating how effectively vCPU pinning manages nominal execution latencies prior to the introduction of external stress factors.

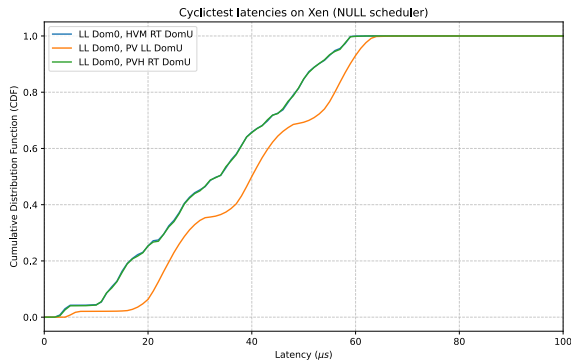


Fig. 49. Comparing HVM, PV and PVH guests - Null Scheduler (No Noise)

1) LOW LATENCY DOM0 AND LOW LATENCY DOMU (PV DomU): This section details the analysis of a 5-minute execution of the `cyclictest` utility within a Xen virtualized environment, specifically evaluating a Paravirtualized (PV) guest. The configuration utilizes a “Low Latency” kernel for both the privileged domain (Dom0) and the

unprivileged user domain (DomU). Crucially, this test evaluates the system emulating the static NULL scheduler with explicit vCPU pinning, and it is conducted in a quiet environment without any background `stressdom0` workload. The objective is to assess the baseline latency and determinism of a PV guest when fully optimized through static hardware allocation.

a) Nominal Performance and Average Latency: The data obtained from the `cyclictest` execution reveals the baseline virtualization overhead in this optimized, pinned PV configuration. The average latency recorded during the test was $39 \mu s$. The absolute minimum latency achieved was $5 \mu s$. While pinning isolates the workload, the histogram indicates a spread in nominal execution times, confirming that baseline virtualization jitter persists even with a static scheduler.

b) Worst-Case Execution Time (WCET) Analysis: The analysis of the Worst-Case Execution Time (WCET) highlights the system’s ability to bound execution delays under ideal, unstressed conditions. Over the duration of the test, the absolute maximum latency recorded was $492 \mu s$.

c) Observations on the Baseline PV Pinned Environment (NULL Scheduler): The empirical data collected from this sustained test yields the following observations regarding the behavior of the Low Latency PV DomU running on a Low Latency Dom0 with static pinning using the NULL scheduler:

- **Bounded WCET:** The maximum latency was contained at $492 \mu s$. While higher than fully patched RT configurations, this demonstrates a stable upper bound for a PV guest utilizing Low Latency kernels in a pinned environment.
- **Baseline Jitter:** The average latency of $39 \mu s$ indicates that the static allocation provided by the NULL scheduler does not completely eliminate the inherent scheduling jitter associated with the virtualization layer itself.

2) LOW LATENCY DOM0 AND REAL TIME DOMU (PVH DomU): This section details the analysis of a 5-minute execution of the `cyclictest` utility within a Xen virtualized environment, specifically evaluating a PVH guest. The configuration utilizes a “Low Latency” kernel deployed on the privileged domain (Dom0) and a Real-Time (`PREEMPT_RT`) kernel on the unprivileged user domain (DomU). This test evaluates the system emulating the static NULL scheduler with explicit vCPU pinning, and it is conducted in a quiet environment without any background stress workload. The objective is to assess the baseline latency and determinism of a highly optimized PVH guest when fully isolated through static hardware allocation.

a) Nominal Performance and Average Latency: The data obtained from the `cyclictest` execution reveals the baseline virtualization overhead in this optimized, pinned PVH configuration. The average latency recorded during the test was $33 \mu s$. The absolute minimum latency achieved was $3 \mu s$. The histogram indicates a tight distribution of nominal

execution times, confirming that pinning effectively shields the guest and provides high consistency.

b) Worst-Case Execution Time (WCET) Analysis: The analysis of the Worst-Case Execution Time (WCET) highlights the system’s ability to rigidly bound execution delays under ideal, unstressed conditions. Over the duration of the test, the absolute maximum latency recorded was 71 μs .

c) Observations on the Baseline PVH Pinned Environment (NULL Scheduler): The empirical data collected from this sustained test yields the following observations regarding the behavior of the RT PVH DomU running on a Low Latency Dom0 with static pinning using the NULL scheduler:

- **Strictly Bounded WCET:** The maximum latency was contained at 71 μs . This demonstrates an exceptionally stable upper bound for a PVH guest, proving that the combination of RT guest kernels, Low Latency host kernels, vCPU pinning, and the NULL scheduler provides hard real-time characteristics.
- **Baseline Jitter:** The average latency of 33 μs and the tight spread of execution times indicate a high level of consistency in the static allocation provided by the NULL scheduler.

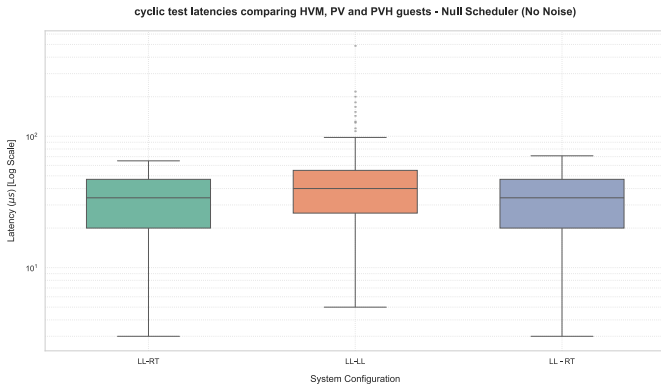


Fig. 50. Comparing HVM, PV and PVH guests - Null Scheduler (No Noise)

O. IMPACT OF THE STRESS WORKLOAD ON PV AND PVH ARCHITECTURES UNDER STATIC ALLOCATION

Building upon the baseline established in the ideal, unstressed environment, this phase introduces a severe background stress workload (**stressdom0**) into the privileged control domain. The primary objective is to evaluate the resilience of static vCPU pinning—acting as a surrogate for the offline NULL scheduler—when the host system is heavily saturated with competing processes.

By subjecting both the Paravirtualized (PV) and Hardware Virtual Machine with PV drivers (PVH) guests to this intense host-level contention, we can determine whether strict hardware isolation alone is sufficient to prevent latency spikes from degrading guest determinism. This evaluation specifically tests the boundaries of system

predictability, observing the extent to which stress-induced jitter manages to bypass the static allocation and affect the isolated virtual CPUs.

The subsequent analyses reveal a distinct divergence in architectural resilience under load. While vCPU pinning provides a foundational level of stability for both configurations, the empirical data highlights that the PV architecture remains susceptible to measurable interference from the host. Conversely, the PVH architecture demonstrates exceptional isolation, successfully shielding its critical execution paths and maintaining rigid Worst-Case Execution Time (WCET) boundaries despite the intense resource contention within Dom0.

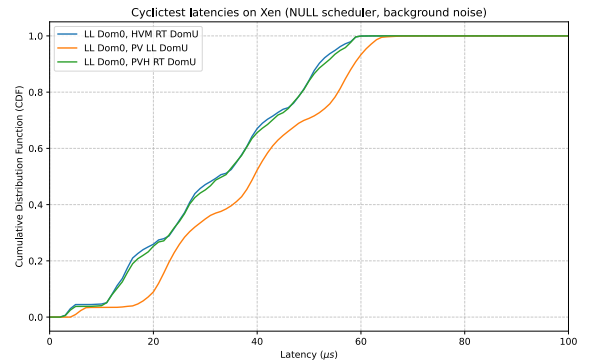


Fig. 51. Comparing HVM, PV and PVH guests - Null Scheduler (Background Noise)

1) LOW LATENCY DOM0 AND LOW LATENCY DOMU (PV DomU): This section analyzes the results obtained from a 5-minute execution of the **cyclictest** utility within a Xen virtualized environment, explicitly assessing a Paravirtualized (PV) guest. The configuration features a “Low Latency” kernel deployed on both the privileged domain (Dom0) and the unprivileged user domain (DomU). Crucially, this test evaluates the performance of the NULL scheduler with explicit vCPU pinning while Dom0 is subjected to a significant background stress workload (**stressdom0**). The objective is to evaluate the latency characteristics and the ability of the NULL scheduler’s static allocation to manage host-level contention in an optimized, pinned PV environment.

a) Nominal Performance and Average Latency: The data obtained from the **cyclictest** execution reveals the baseline virtualization overhead and scheduling behavior under stress conditions when utilizing the static NULL scheduler with pinning. The average latency recorded during the test was 39 μs . The absolute minimum latency achieved was 4 μs . The histogram data indicates a broad distribution of execution latencies, demonstrating the variability introduced by the **stressdom0** workload, even when a static, pinned scheduler is employed.

b) Worst-Case Execution Time (WCET) Analysis: The analysis of the Worst-Case Execution Time (WCET) provides insight into the system’s ability to bound execution

delays under stress. Over the duration of the test, the absolute maximum latency recorded was $285 \mu s$.

c) *Observations on the Stressed PV Pinned Environment (NULL Scheduler)*: The empirical data collected from this sustained test yields the following observations regarding the behavior of the Low Latency PV DomU running on a Low Latency Dom0 with vCPU pinning under `stressdom0` using the NULL scheduler:

- **Bounded WCET**: The maximum latency was contained at $285 \mu s$. This indicates that the combination of the dual Low-Latency kernel setup and the pinned NULL scheduler provides a degree of stability under stress, establishing a tighter bound than unpinned configurations.
- **Stress-Induced Jitter**: The average latency of $39 \mu s$ and the wide spread of nominal execution times confirm the presence of significant scheduling jitter. This highlights that even with a statically pinned scheduler, resource contention from the `stressdom0` workload on a PV guest still significantly impacts latency stability.

2) *LOW LATENCY DOM0 AND REAL-TIME DOMU (PVH DomU)*: This section analyzes the results obtained from a 5-minute execution of the `cyclictest` utility within a Xen virtualized environment, evaluating a Hardware Virtual Machine with Paravirtualized drivers (PVH) guest. The configuration features a “Low Latency” kernel deployed on the privileged domain (Dom0) and a Real-Time (PREEMPT_RT) kernel on the unprivileged user domain (DomU). Crucially, this test evaluates the performance of the NULL scheduler with explicit vCPU pinning while Dom0 is subjected to a background stress workload (`stressdom0`). The objective is to evaluate the latency characteristics and the ability of the NULL scheduler’s static allocation to maintain determinism under host-level contention in an optimized, pinned PVH environment.

a) *Nominal Performance and Average Latency*: The data obtained from the `cyclictest` execution reveals the baseline virtualization overhead and scheduling behavior under stress conditions when utilizing the static NULL scheduler with vCPU pinning. The average latency recorded during the test was $33 \mu s$. The absolute minimum latency achieved was $3 \mu s$. The data indicates a concentrated distribution of nominal execution times, demonstrating a high level of consistency despite the background load.

b) *Worst-Case Execution Time (WCET) Analysis*: The analysis of the Worst-Case Execution Time (WCET) provides insight into the system’s ability to rigidly bound execution delays under stress. Over the duration of the test, the absolute maximum latency recorded was strictly capped at $71 \mu s$.

c) *Observations on the Stressed PVH Pinned Environment (NULL Scheduler)*: The empirical data collected from this sustained test yields the following observations regarding the behavior of the RT PVH DomU running on a Low Latency Dom0 with vCPU pinning under `stressdom0` using the NULL scheduler:

- **Strictly Bounded WCET**: The maximum latency was contained at $71 \mu s$. This demonstrates that the combination of the PREEMPT_RT guest kernel, the Low-Latency host kernel, static vCPU pinning, and the NULL scheduler provides exceptional stability and successfully shields the critical guest execution path from severe preemption spikes.
- **Resilience to Host Stress**: The average latency of $33 \mu s$ and the rigid WCET boundary confirm that this highly optimized, static PVH configuration effectively mitigates the severe scheduling jitter typically induced by host-level resource contention.

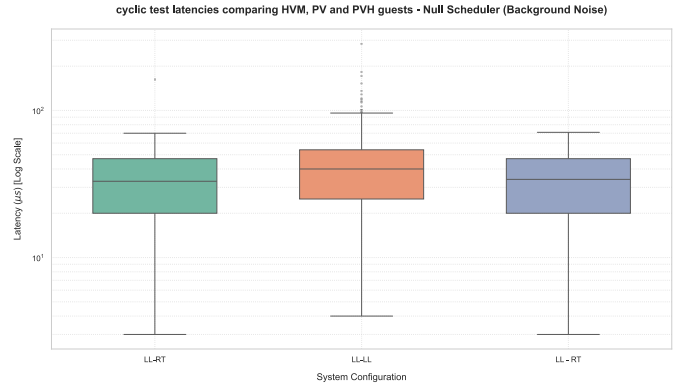


Fig. 52. Comparing HVM, PV and PVH guests - Null Scheduler (Background Noise)

P. SUMMARY OF RESULTS (PV AND PVH)

To synthesize the findings from our latency evaluations, the following table aggregates the Worst-Case Execution Time (WCET) results recorded across the three evaluated Xen virtualization modes: Hardware Virtual Machine (HVM), Hardware Virtual Machine with Paravirtualized drivers (PVH), and fully Paravirtualized (PV) guests.

This section provides a quantitative breakdown of the percentage increments for both the average latency (Mean $\pm$ Standard Deviation) and the maximum latency (WCET) across Xen’s three virtualization modes: HVM, PV, and PVH. The analysis evaluates the performance degradation caused by a heavy stress workload introduced in the control domain (Dom0).

The following tables illustrate the system’s behavior under the default dynamic scheduler. These results highlight the performance impact of the Dom0 stress workload when no hardware isolation mechanisms are applied. ### HVM LL RT

PERCENTAGE INCREMENTS (Stress vs. Baseline): * Average Increment: +8.42% * WCET Increment: +194.37%

1) **PV LL-LL: PERCENTAGE INCREMENTS (Stress vs. Baseline):** * Average Increment: +8.16% * WCET Increment: -40.57%

TABLE XLVII
XEN VIRTUALIZATION MODES WCET COMPARISON

Configuration	HVM	PVH	PV
BASILINE	71	76	525
STRESS WORKLOAD	209	157	312
vCPU PINNING BASE-LINE	65	71	492
vCPU PINNING STRESS WORKLOAD	163	71	285

TABLE XLVIII
HVM LATENCY STATISTICS

METRIC	BASILINE	STRESS WORKLOAD
Average $\pm$ SD	$32.89 \pm 15.14 \mu s$	$35.66 \pm 15.60 \mu s$
WCET (Max)	$71 \mu s$	$209 \mu s$

TABLE XLIX
PV LATENCY STATISTICS

METRIC	BASILINE	STRESS WORKLOAD
Average $\pm$ SD	$37.17 \pm 14.69 \mu s$	$40.20 \pm 16.22 \mu s$
WCET (Max)	$525 \mu s$	$312 \mu s$

TABLE L
PVH LATENCY STATISTICS

METRIC	BASILINE	STRESS WORKLOAD
Average $\pm$ SD	$32.40 \pm 15.36 \mu s$	$35.58 \pm 15.50 \mu s$
WCET (Max)	$76 \mu s$	$157 \mu s$

2) **PVH LL-RT: PERCENTAGE INCREMENTS (Stress vs. Baseline):** * Average Increment: +9.82% * WCET Increment: +106.58%

The tables below evaluate the resilience of the same configurations when strict vCPU pinning is enforced. By isolating Dom0 and DomU on dedicated physical cores, this scenario demonstrates how static hardware allocation mitigates scheduling interference under stress.

3) **HVM LL RT: PERCENTAGE INCREMENTS (Stress vs. Baseline):** * Average Increment: -1.11% * WCET Increment: +150.77%

4) **PV LL-LL (Static vCPU Pinning): PERCENTAGE INCREMENTS (Stress vs. Baseline):** * Average Increment: -2.13% * WCET Increment: -42.07%

5) **PVH LL-RT (Static vCPU Pinning): PERCENTAGE INCREMENTS (Stress vs. Baseline):** * Average Increment: +0.14% * WCET Increment: 0.00%

V. CONCLUSION

This paper presented a systematic experimental characterization of Xen and KVM as real-time hypervisors for mixed-criticality workloads. Across all tested configurations, achieving bounded worst-case latencies requires an *end-to-end* real-time approach: a **PREEMPT_RT**-patched kernel is necessary at both host and guest level, and vCPU

pinning is essential for shielding time-critical domains from background interference.

For KVM, the combination of an RT host kernel, an RT guest kernel, and vCPU pinning consistently reduces worst-case latency; however, multi-millisecond spikes under heavy stress workloads remain a concern. For Xen, the PVH guest type with the Null scheduler and strict vCPU pinning delivers the tightest latency bounds. TACLe benchmark results confirm that lightweight tasks (e.g., **matrix1**, **huff_enc**) maintain near-deterministic execution, while heavy workloads (e.g., **DEBIE**, **test3**) remain sensitive to scheduling noise across all configurations. Future work will investigate memory-bandwidth partitioning via Intel RDT and the integration of Unikraft [?] unikernels as latency-aware guest alternatives.

ACKNOWLEDGMENT

The authors wish to thank Ch.mo Prof. Marcello Cinque, Ch.moProf. Luigi De Simone, and Ing. Francesco Boccola of the University of Naples “Federico II” for their guidance and support throughout this project.

REFERENCES

- [1] L. Abeni and D. Faggioli, “An experimental analysis of the xen and kvm latencies,” in *2019 IEEE 22nd International Symposium on Real-Time Distributed Computing (ISORC)*, 2019, pp. 18–26.

TABLE LI
HVM LATENCY STATISTICS - vCPU PINNING

METRIC	vCPU PINNING BASELINE	vCPU PINNING STRESS WORKLOAD
Average $\pm$ SD	$33.09 \pm 15.18 \mu s$	$32.73 \pm 15.09 \mu s$
WCET (Max)	$65 \mu s$	$163 \mu s$

TABLE LII
PV LATENCY STATISTICS - vCPU PINNING

METRIC	vCPU PINNING BASELINE	vCPU PINNING STRESS WORKLOAD
Average $\pm$ SD	$40.00 \pm 14.71 \mu s$	$39.15 \pm 15.23 \mu s$
WCET (Max)	$492 \mu s$	$285 \mu s$

TABLE LIII
PVH LATENCY STATISTICS - vCPU PINNING

METRIC	vCPU PINNING BASELINE	vCPU PINNING STRESS WORKLOAD
Average $\pm$ SD	$33.16 \pm 15.12 \mu s$	$33.20 \pm 15.09 \mu s$
WCET (Max)	$71 \mu s$	$71 \mu s$

- [2] —, “Using xen and kvm as real-time hypervisors,” *Journal of Systems Architecture*, vol. 106, p. 101709, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1383762120300035>
- [3] L. Abeni, “Virtualized real-time workloads in containers and virtual machines,” *Journal of Systems Architecture*, vol. 154, p. 103238, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1383762124001759>
- [4] M. Cinque, L. De Simone, and D. Ottaviano, “Temporal isolation assessment in virtualized safety-critical mixed-criticality systems: A case study on xen hypervisor,” *Journal of Systems and Software*, vol. 216, p. 112147, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0164121224001924>
- [5] H. Falk, S. Altmeyer, P. Hellinckx, B. Lisper, W. Puffitsch, C. Rochange, M. Schoeberl, R. B. Sørensen, P. Wägemann, and S. Wegener, “TACLeBench: A benchmark collection to support worst-case execution time research,” in *16th International Workshop on Worst-Case Execution Time Analysis (WCET 2016)*, ser. OpenAccess Series in Informatics (OASICS), M. Schoeberl, Ed., vol. 55. Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2016, pp. 2:1–2:10.